



**UNIVERSIDAD
DE GRANADA**

TRABAJO FIN DE GRADO

DOBLE GRADO EN INGENIERÍA INFORMÁTICA Y MATEMÁTICAS

Inteligencia Artificial Explicable en medicina

Caso de estudio en Cancer de Próstata

Autor

Jose María Martín Martín

Directores

Iván Sevillano García

Francisco Herrera Triguero



**ESCUELA TÉCNICA SUPERIOR DE INGENIERÍAS INFORMÁTICA Y DE TELECOMUNICACIÓN
FACULTAD DE CIENCIAS**

Granada, julio de 2026

Yo, **Jose María Martín Martín**, alumno de la titulación Doble Grado en Ingeniería Informática y Matemáticas de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación y Facultad de Ciencias de la Universidad de Granada**, con DNI 77558885X, declaro que este Trabajo de Fin de Grado es original en el sentido de que no se ha hecho uso de fuentes sin previa citación.

A handwritten signature in black ink, appearing to read 'Jose', with a large loop above the name and a long horizontal flourish extending to the right.

Fdo: Jose María Martín Martín

Granada a 14 de julio de 2026 .

Índice

1. Introducción	1
2. Fundamentos Teóricos	2
2.1. Imágenes digitales y filtrado	2
2.1.1. Fundamentos de filtrado espacial	4
2.1.2. Operación convolución	5
2.1.3. Filtrado espacial lineal	5
2.1.4. Correlación cruzada y Convolución en imágenes	5
2.2. Probabilidad e Inferencia Estadística	6
2.2.1. Introducción	6
2.2.2. Fundamentos de probabilidad	7
2.2.3. Resultados sobre vectores aleatorios	8
2.2.4. Resultados sobre inferencia estadística	9
2.3. Optimización	13
2.3.1. Optimización basada en gradientes	13
2.3.2. Matriz Jacobiana y Hessiana	16
2.4. Fundamentos matemáticos de Machine Learning	17
2.4.1. Algoritmos de aprendizaje	17
2.4.1.1. Problema de clasificación	18
2.4.1.2. Aprendizaje supervisado	18
2.4.1.3. Aprendizaje no supervisado	19
2.4.1.4. Función de coste	20
2.4.1.5. Optimizador	21

2.5.	Fundamentos matemáticos del Deep Learning	23
2.5.1.	Perceptrón y perceptrón multicapa	23
2.5.2.	Capa de salida	26
2.5.2.1.	Unidades con función Sigmoide	26
2.5.2.2.	Unidades con función Softmax	27
2.5.3.	Capas ocultas	28
2.5.4.	Back-Propagation	30
2.5.5.	Redes neuronales convolucionales	32
2.5.5.1.	Pooling	34
2.5.6.	Transformers	36
2.5.6.1.	Scaled Dot-Product Attention	36
2.5.6.2.	Multi-Head Attention	36
2.6.	Out Of Distribution	36
2.7.	IA explicable: algoritmos LRP y CRP	37
3.	Materiales y métodos	38
3.1.	Descripción del problema clínico	38
3.1.1.	Conceptos básicos sobre el cáncer de próstata	38
3.2.	El conjunto de datos	39
3.3.	OpenOOD	40
3.4.	Modelos seleccionados	41
3.4.1.	Transfer Learning	41
3.4.2.	ResNet-50	41
3.4.3.	DeiT	44
3.5.	Fase de entrenamiento	47

3.6. Evaluación OOD	48
3.6.1. Datasets OOD	48
3.6.1.1. Far-OOD: CIFAR-10	48
3.6.1.2. Near-OOD: COVID	48
3.6.2. Métricas de evaluación OOD	49
3.6.3. Propuestas de evaluación OOD	51
3.6.3.1. Maximum Softmax Probability	51
3.6.3.2. Virtual-logit Matching	52
3.6.3.3. XAIPostProcessor	52
3.6.3.4. MahaPostProcessor	53
3.6.3.5. XAIPostProcessor (versión normalizada)	53
3.6.3.6. BestXAIPostProcessor	54
4. Resultados	55
4.1. Resultados ResNet-50	55
4.2. Resultados DeiT	57
4.3. Comparativa de resultados	58
4.4. Resultados de primeros experimentos de evaluación OOD	59
4.5. Resultados de BestXAIPostprocessor	61
5. Discusión, conclusiones y trabajos futuros	64
5.1. Discusión de resultados	64
5.2. Conclusiones	66
5.3. Trabajos futuros	67
6. Glosario de términos	68

Índice de cuadros

1.	Categorización de resultados en función del valor real y la predicción de un modelo.	49
2.	Comparación de métodos de detección OOD	59
3.	Comparación de métodos de detección OOD. Selección de mejores <i>features</i> en ResNet-50: [25 %, 50 %, 75 %, 80 %, 90 %, 100 % (equivalente a XAIPostProcessor)].	61
4.	Comparación de métodos de detección OOD. Selección de mejores <i>features</i> en DeiT: [25 %, 50 %, 75 %, 100 %].	62

Índice de figuras

1.	Ejemplo de gradiente descendente para función de variable real, [1]	14
2.	Ejemplo de puntos de silla para función de variable real. De izquierda a derecha mínimo, máximo y punto de silla. [1]	15
3.	Ejemplo de gradiente descendente y como afecta el valor de la tasa de aprendizaje.[2]	16
4.	SGD sin momentum (izda), SGD con momentum (dcha) [3]	23
5.	Ejemplo de MLP, https://commons.wikimedia.org/wiki/File:FeedForwardNN.png	25
6.	Ejemplo de perceptrón, https://commons.wikimedia.org/wiki/File:Perceptron-unit.svg	25
7.	Función Sigmoide.	27
8.	Función ReLU.	28
9.	Función ReLU con pendiente. Se le ha asignado una pendiente $\alpha_i = 0,1$ para ilustrar el comportamiento de la función.	29
10.	Función Tangente Hiperbólica.	30
11.	Ejemplo de backprop.	31
12.	Comparación capas convolucionales con capas multiperceptrón clásicas. Parte 1. . .	32
13.	Comparación capas convolucionales con capas multiperceptrón clásicas. Parte 2. . .	33
14.	Ejemplo de CNN, https://commons.wikimedia.org/wiki/File:Convolutional_Neural_Network.png	34
15.	Ejemplo de biopsia de adenocarcinoma ductal	40
16.	Ejemplo de bloque residual. Esta imagen está basada en los ejemplos de [4].	42
17.	Ejemplo de bloque bottleneck para ResNet-50. Esta imagen está basada en los ejem- plos de [4].	43
18.	Ejemplo de bloque de encoder (fondo gris). Esta imagen está basada en los ejemplos de [5] y [6].	45
19.	Esquema de <i>Vision Transformer</i> . Esta imagen está basada en los ejemplos de [5]. . .	46

20.	Ejemplo de ROC curve. https://commons.wikimedia.org/wiki/File:Roc_curve.svg#file	50
21.	Resultados de entrenamiento de ResNet-50. Comparación de evolución de función de pérdida en el conjunto de entrenamiento y validación.	55
22.	Resultados de entrenamiento de ResNet-50. Evolución de la <i>accuracy</i>	56
23.	Resultados de entrenamiento de DeiT. Comparación de evolución de función de pérdida en el conjunto de entrenamiento y validación.	57
24.	Resultados de entrenamiento de DeiT. Evolución de la <i>accuracy</i>	58
25.	Resultados de detección OOD para ResNet-50 con BestXAIProcessor en el conjunto de COVID. Las pruebas se realizan para valores diferentes de p % con el objetivo de ver cómo reducir dimensionalidad en el espacio de <i>features</i> influye en el resultado final.	61
26.	Resultados de detección OOD para DeiT con BestXAIProcessor en el conjunto de COVID. Las pruebas se realizan para valores diferentes de p % con el objetivo de ver cómo reducir dimensionalidad en el espacio de <i>features</i> influye en el resultado final.	62
27.	Prueba kde para DeiT. Estimación de funciones de densidad para cada distribución de datos.	63
28.	Prueba kde para ResNet-50. Estimación de funciones de densidad para cada distribución de datos.	64

Abstract

Prostate cancer is one of the most prevalent malignancies in men. This work addresses the problem through the use of Deep Learning models in order to classify prostatic biopsy samples. However, the main goal is to develop Out-of-Distribution detection methods based on eXplainable AI. In order to accomplish this task, this work proposes the use of DeiT and ResNet-50 models with a set of postprocessors based on the Mahalanobis distance and eXplainable AI vectors generated by the CRP algorithm. Results show that methods employing a low contribution of the CRP vector outperform state-of-the-art methods such as ViM and MSP. In contrast, incrementing the contribution of the CRP vectors yields performance that matches the state-of-the-art, resulting in an effective discriminator of Out-of-Distribution samples. Next, it is employed a smaller amount of the total features, which resulted in a significant separator for DeiT, saving up to 75 % of total features. Lastly, it is worth noting that the use of all features provided by DeiT for Out-of-Distribution detection, results in perfect separation between both datasets.

Resumen

El cáncer de próstata es una enfermedad que afecta a gran parte de la población masculina. Este trabajo aborda el problema mediante el uso de modelos de Deep Learning para clasificar muestras de biopsias prostáticas. No obstante, el principal objetivo es el desarrollo de métodos de detección Out-of-Distribution empleando vectores de IA explicable. Para ello, se han utilizado los modelos DeiT y ResNet-50 y una serie de posprocesadores basados en la distancia de Mahalanobis y vectores del algoritmo de IA explicable CRP. Los resultados muestran que los métodos propuestos que emplean una contribución reducida del vector CRP superan a métodos del estado del arte como ViM y MSP. Por el contrario, cuando se incrementa la contribución del vector CRP, el rendimiento obtenido es comparable al de los métodos del estado del arte, mostrando resultados de un separador razonable. A continuación, se propone utilizar una cantidad menor de las *features* totales, que en el modelo DeiT supuso la obtención de un separador destacable, ahorrando el 75 % de *features*. Por último, cabe destacar que el uso de la totalidad de *features* del modelo DeiT para la detección OOD, resulta en una separación perfecta de ambos conjuntos de datos.

1. Introducción

Durante los últimos años, el crecimiento de la IA ha sido exponencial. Según [7], el 78 % de las empresas encuestadas en el estudio afirmaron utilizar la IA en alguna de sus funciones en 2024, frente al 55 % en 2023. Este crecimiento se refleja en la extensión del área de aplicación de la IA a la gran mayoría de tareas que conciernen al ser humano. Este desarrollo es producto de la innovación sobre los elementos que componen los modelos de inteligencia artificial, desde nuevas arquitecturas hasta pequeñas mejoras en elementos como optimizadores. Por ejemplo, las arquitecturas basadas en mecanismos de atención [6], han dado lugar a los procesadores de lenguaje natural, herramientas muy populares a día de hoy (p.ej. ChatGPT con 700 millones de usuarios semanales en 2025 [8]). Todo esto ha provocado que la IA sea aplicable a tareas donde la correctitud es crítica, proporcio-

nando salidas con una precisión comparable a la de un especialista en tiempos considerablemente menores.

La medicina es uno de los campos donde la IA ha centrado más su atención. Esta ciencia lleva a cabo tareas donde es importante ser preciso, debido a que el objeto de estudio es la salud de las personas. En particular, el cáncer es una de las patologías que mayor atención genera en la medicina.

En España, el cáncer de próstata es el tumor más común en hombres, representando la tercera causa de muerte por cáncer [9]. Se estima que 1 de cada 8 hombres será diagnosticado con cáncer de próstata en su vida [9]. El hecho de que sea uno de los más comunes dentro de la población impulsa la necesidad de métodos para su detección, surgiendo en este escenario técnicas de visión por computador basadas en IA.

En este trabajo, se aborda un problema de **aprendizaje supervisado** (sección 2.4.1.2) y concretamente una tarea de **clasificación** (sección 2.4.1.1) sobre un conjunto de datos de biopsias de cáncer de próstata.

Aun así, el principal objetivo es responder a la pregunta **¿El uso de algoritmos de IA explicable resulta de utilidad en métodos de detección OOD?** Para ello se lleva a cabo la descripción e implementación de métodos de **evaluación OOD** (sección 2.6) basados en el uso de un algoritmo de **IA explicable (XAI)** (sección 2.7).

Se pretende que estos métodos sean efectivos en cualquier contexto donde la detección de ejemplos OOD sea necesaria, en lugar de ser de utilidad exclusiva en nuestro conjunto de datos.

El resto del trabajo se organiza como sigue. La sección 2 repasa los fundamentos teóricos que dan lugar a las herramientas utilizadas. Después, en la sección 3, se detallan los materiales y métodos empleados para realizar los experimentos planteados. A continuación, se muestran los resultados obtenidos en la sección 4. La sección 5 realiza una descripción e interpretación sobre los resultados, así como una síntesis de conclusiones fruto de la realización de este trabajo y terminará con un apartado de posibles vías de trabajos futuros. El trabajo concluye con una sección de Glosario de términos y la bibliografía empleada (secciones 6 y 7, resp.).

2. Fundamentos Teóricos

2.1. Imágenes digitales y filtrado

Definición 2.1.1. Una imagen es una función bidimensional $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ donde $f(x, y)$ representa la amplitud en el punto (x, y) (**coordenadas espaciales**). Al valor $f(x, y)$ se le conoce como **intensidad o nivel de gris** en las coordenadas (x, y) . Cuando x, y y $f(x, y)$ son valores finitos y discretos llamamos a la imagen **imagen digital**.

En particular, el valor $f(x, y)$ es proporcional a la energía emitida por una fuente física (p.ej. ondas

electromagnéticas). Como consecuencia $f(x, y)$ es un valor **finito** y **no negativo**:

$$0 \leq f(x, y) < \infty, \quad \forall (x, y) \in \mathbb{R}^2$$

Definición 2.1.2. Sea $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ una imagen en escala de grises. Esta imagen se caracteriza por dos componentes:

- $i : \mathbb{R}^2 \rightarrow \mathbb{R}$: La cantidad de iluminación **emitida** por una fuente que incide sobre la escena.
- $r : \mathbb{R}^2 \rightarrow [0, 1]$: La cantidad de iluminación **reflejada** por los objetos de la escena.

A estos componentes se les conoce como **iluminación y reflectancia**. Estas funciones se relacionan con $f(x, y)$ de la siguiente forma:

$$f(x, y) = i(x, y)r(x, y) \quad \forall (x, y) \in \mathbb{R}^2$$

donde

$$0 \leq i(x, y) < \infty \quad \forall (x, y) \in \mathbb{R}^2$$

y

$$0 \leq r(x, y) \leq 1 \quad \forall (x, y) \in \mathbb{R}^2$$

El **procesamiento de imágenes digitales** se realiza por medio de dispositivos informáticos. Cabe destacar que una imagen digital está compuesta por un número finito de elementos, donde cada uno contiene una posición y valor particular. A estos elementos los conocemos como **píxeles**. Estas imágenes digitales se obtienen tras procesos de discretización del espacio (muestreo) y de los valores de intensidad (cuantificación). Podemos considerar entonces una **imagen digital** (en escala de grises) como una función $f : \{1, 2, \dots, n\} \times \{1, 2, \dots, m\} \rightarrow \{0, \dots, k\}$, donde el conjunto $\{1, 2, \dots, n\} \times \{1, 2, \dots, m\}$ representa la discretización espacial y $\{0, \dots, k\}$ la discretización de intensidad. Además, es claro que la imagen digital se puede representar como una matriz donde:

$$A = \begin{pmatrix} f(1, 1) & f(1, 2) & \cdots & f(1, m) \\ f(2, 1) & f(2, 2) & \cdots & f(2, m) \\ \vdots & \vdots & \ddots & \vdots \\ f(n, 1) & f(n, 2) & \cdots & f(n, m) \end{pmatrix}$$

Hasta ahora, hemos tratado las imágenes en escala de grises, pero todo lo anterior es extensible al caso de imágenes a color. Para definir una imagen a color, se hace uso de 3 escalas de intensidad para cada uno de los **3 colores primarios de la luz: Rojo, Verde y Azul**. Por tanto, se combinan los niveles de intensidad en cada punto de la imagen para producir una imagen a color.

Definición 2.1.3. Imagen digital a color. Una imagen digital a color es una función que se puede ver o bien como $f : \{1, 2, \dots, n\} \times \{1, 2, \dots, m\} \rightarrow \{0, \dots, k\}^3$ o bien como $f : \{1, 2, \dots, n\} \times \{1, 2, \dots, m\} \times \{1, 2, 3\} \rightarrow \{0, \dots, k\}$, donde en cada **píxel** (x, y) tenemos 3 valores de intensidad asociados (i_r, i_g, i_b) , es decir:

$$f(x, y) = (i_r, i_g, i_b)$$

o bien

$$f(x, y, 1) = i_r, \quad f(x, y, 2) = i_g, \quad f(x, y, 3) = i_b$$

Estos valores de intensidad son asociados a la escala **RGB o Red, Green, Blue**. A estas 3 categorías también se les conoce como **canales**. Por tanto, podemos representar una imagen como la combinación de 3 matrices:

$$R = \begin{pmatrix} f(1, 1, 1) & f(1, 2, 1) & \cdots & f(1, m, 1) \\ f(2, 1, 1) & f(2, 2, 1) & \cdots & f(2, m, 1) \\ \vdots & \vdots & \ddots & \vdots \\ f(n, 1, 1) & f(n, 2, 1) & \cdots & f(n, m, 1) \end{pmatrix} \quad G = \begin{pmatrix} f(1, 1, 2) & f(1, 2, 2) & \cdots & f(1, m, 2) \\ f(2, 1, 2) & f(2, 2, 2) & \cdots & f(2, m, 2) \\ \vdots & \vdots & \ddots & \vdots \\ f(n, 1, 2) & f(n, 2, 2) & \cdots & f(n, m, 2) \end{pmatrix}$$

$$B = \begin{pmatrix} f(1, 1, 3) & f(1, 2, 3) & \cdots & f(1, m, 3) \\ f(2, 1, 3) & f(2, 2, 3) & \cdots & f(2, m, 3) \\ \vdots & \vdots & \ddots & \vdots \\ f(n, 1, 3) & f(n, 2, 3) & \cdots & f(n, m, 3) \end{pmatrix}$$

[10]

2.1.1. Fundamentos de filtrado espacial

Durante esta sección, presentamos un elemento esencial en el tratado de imágenes como son los **filtros**. El término filtro viene dado por el procesamiento en el **dominio de frecuencias** donde 'filtrado' hace referencia a aceptar, modificar o rechazar ciertas frecuencias. Por ejemplo, un filtro que permite el paso de bajas frecuencias es denominado **filtro paso bajo (lowpass)**. Este filtro aplicado a una imagen (llamado **suavizado**) realiza, precisamente, un efecto de suavizado en la imagen es decir, difumina en cierto modo la imagen. Un **filtro paso alta (highpass)** es un filtro encargado de dejar pasar las altas frecuencias. Este tipo de filtros se utilizan para acentuar bordes lo cual es conocido como **sharpening**.

El filtrado espacial modifica los píxeles de una imagen haciendo uso de una función. Esta función asigna un valor al píxel (x, y) dependiente del valor del propio píxel y sus vecinos. Si la operación aplicada a los píxeles de la imagen es lineal, hablaremos entonces de **filtro espacial lineal**. En caso contrario, se denomina **filtro espacial no lineal**.

[10]

2.1.2. Operación convolución

Podemos pensar en el concepto de convolución como la combinación de la información que proporciona una función f con otra función g que indica como debe ponderarse esta función. En el caso de función de variable real, se propone la siguiente definición:

Definición 2.1.4. Convolución Sean $f, g : \mathbb{R} \rightarrow \mathbb{R}$ dos funciones de variable real. Se define la **convolución** $f * g$ como:

$$(f * g)(t) = \int_{-\infty}^{\infty} f(a)g(t - a)da$$

A la función f generalmente se le llama **input** y a g **filtro, kernel o máscara**.

[1]

Procedemos ahora a una definición de convolución para imágenes.

2.1.3. Filtrado espacial lineal

Como habíamos comentado, el **filtro espacial lineal** realiza una operación lineal considerando el valor de cada píxel y sus vecinos. Esto se traduce en una operación de suma de productos entre una **imagen f** y un **filtro w** . El **filtro**, también llamado **kernel o máscara**, se trata de un vector o matriz cuyas dimensiones define la vecindad a considerar de cada píxel. Los coeficientes de cada kernel dependerán de la naturaleza del mismo.

2.1.4. Correlación cruzada y Convolución en imágenes

Una vez definido un concepto general de convolución, vamos a detallar como opera en el caso de imágenes.

Definición 2.1.5. Sea f una **imagen digital** y w un **kernel**. Se define la operación **Correlación cruzada** $w \circledast f$ como:

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t)f(x + s, y + t) \quad (x, y) \in \{1, 2, \dots, n\} \times \{1, 2, \dots, m\}$$

Definición 2.1.6. Sea f una **imagen digital** y w un **kernel**. Se define la operación **convolución** $w * f$ como:

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t)f(x - s, y - t) \quad (x, y) \in \{1, 2, \dots, n\} \times \{1, 2, \dots, m\}$$

En ambas operaciones, para una imagen a color, se aplica el filtro a la matriz asociada a cada canal RGB.

[10]

Estas operaciones son claves dentro del filtrado de imágenes, permitiendo aplicar filtros de todo tipo como filtros de *smoothing* o *sharpening*. Por ejemplo, dentro del contexto de la IA, una red neuronal convolucional (sección 2.5.5) podrá aprender estos filtros para tareas como eliminar ruido dentro de imágenes o delimitar precisamente objetos para tareas de clasificación.

2.2. Probabilidad e Inferencia Estadística

2.2.1. Introducción

La teoría de la probabilidad es la rama de las matemáticas que estudia la representación de afirmaciones inciertas. Provee un medio para cuantificar la incertidumbre además de axiomas para la formulación de nuevas afirmaciones inciertas. En inteligencia artificial se usa la teoría de la probabilidad en dos formas principalmente. Primero, las leyes de probabilidad guían la forma de razonar de los sistemas de IA. Segundo, hacemos uso de la probabilidad y la estadística para analizar el comportamiento de dichos sistemas de IA.

La mayoría de ramas de la informática se basan en elementos deterministas. En cambio, la rama del aprendizaje automático trabaja con cantidades inciertas y procesos estocásticos (no deterministas). En la mayoría de actividades, tenemos presente la incertidumbre de alguna manera. Podemos establecer tres posibles tipos de incertidumbre:

- **Estocasticidad inherente al sistema modelado:** Por ejemplo, la mayoría de interpretaciones de la mecánica cuántica describen los escenarios de partículas subatómicas como escenarios probabilísticos.
- **Observaciones incompletas:** Aún considerando sistemas deterministas, pueden parecer estocásticos cuando no es posible observar todas las variables implicadas en el comportamiento del sistema.
- **Modelado incompleto:** Al usar un modelo que descarta parte de la información observada, esta información descartada implica incertidumbre en las predicciones del modelo. Por ejemplo, si tenemos un robot que detecta la posición de cada objeto cercano, el robot discretizará el espacio al predecir la posición futura de estos objetos. Esta discretización hará que el robot haya introducido incertidumbre en la posición exacta de estos objetos.

[1]

2.2.2. Fundamentos de probabilidad

Definición 2.2.1 Vector aleatorio. Sea $(\Omega, \mathcal{F}, P)$ un espacio de probabilidad. Un vector aleatorio $X = (X_1, \dots, X_p)$ es una función medible:

$$X : (\Omega, \mathcal{F}) \rightarrow (\mathbb{R}^p, \mathcal{B}^p)$$

tal que

$$\omega \mapsto X(\omega) = (X_1(\omega), \dots, X_p(\omega))'$$

y $\forall B \in \mathcal{B}^p$,

$$X^{-1}(B) := \{\omega \in \Omega : X(\omega) \in B\} \in \mathcal{F}$$

Además, podemos definir la **distribución de probabilidad inducida por X** , P_X en $(\mathbb{R}^p, \mathcal{B}^p)$ como:

$$P_X[B] := P[X^{-1}(B)], \forall B \in \mathcal{B}^p$$

Definición 2.2.2 Función de distribución. Sea $(\Omega, \mathcal{F}, P)$ un espacio de probabilidad, X un vector aleatorio y P_X una distribución de probabilidad. La **función de distribución** asociada a P_X se define como:

$$F_X(x) = P_X[X_1 \leq x_1, \dots, X_p \leq x_p], \quad \forall x = (x_1, \dots, x_p) \in \mathbb{R}^p$$

Definición 2.2.3 Función de densidad. Sea F_X la **función de distribución** de un vector aleatorio X . Si existe una función f_X integrable en el sentido de Lebesgue tal que:

$$F_X(x) = \int_{-\infty}^{x_p} \cdots \int_{-\infty}^{x_1} f_X(u_1, \dots, u_p) du_1, \dots, du_p, \quad \forall x \in \mathbb{R}^p$$

entonces f_X decimos que es la **función de densidad** de la distribución, definida excepto en conjuntos de medida nula. En este caso, se dice que la distribución de X es continua. Además, si f_X es continua:

$$\frac{\partial}{\partial x_1 \dots \partial x_p} F_X(x), \quad \forall x \in \mathbb{R}^p$$

Procedemos ahora a introducir el concepto de independencia. En general para cierto $B = B_1 \times \dots \times B_p$ con $B_j \in \mathcal{B}, j = 1, \dots, p$ tenemos que:

$$P_X[B] \neq P_{X_1}[B_1] \cdot \dots \cdot P_{X_p}[B_p]$$

Definición 2.2.4 Independencia. Cuando la igualdad anterior se da para cada $B = B_1 \times \dots \times B_p \in \mathcal{B}^p$, las variables $X_1, \dots, X_p$ componentes del vector aleatorio X se dice que son **independientes**. Equivalentemente, esto sucede cuando:

$$F_X(x) = F_{X_1}(x_1) \cdot \dots \cdot F_{X_p}(x_p) \forall x = (x_1, \dots, x_p) \in \mathbb{R}^p$$

y en el caso continuo sí y solo sí

$$f_X(x) = f_{X_1}(x_1) \cdot \dots \cdot f_{X_p}(x_p) \forall x = (x_1, \dots, x_p) \in \mathbb{R}^p$$

2.2.3. Resultados sobre vectores aleatorios

Definición 2.2.5 Media. Sea $X = (X_1, \dots, X_p)$ un vector aleatorio. El **vector media** o simplemente la media de X se define como:

$$\mu_X := E[X] := \begin{pmatrix} E[X_1] \\ \vdots \\ E[X_p] \end{pmatrix} = \begin{pmatrix} \mu_1 \\ \vdots \\ \mu_p \end{pmatrix}$$

Definición 2.2.6 Matriz de covarianzas. Sea $X = (X_1, \dots, X_p)$ un vector aleatorio. La matriz de covarianzas de X se define como:

$$\Sigma_X = Cov(X) := E[(X - \mu_X)(X - \mu_X)^T] = \begin{pmatrix} \sigma_{11} & \dots & \sigma_{1p} \\ \vdots & \ddots & \vdots \\ \sigma_{p1} & \dots & \sigma_{pp} \end{pmatrix}$$

donde

$$\sigma_{ij} = E[(X_i - \mu_i)(X_j - \mu_j)^T] = \sigma_{ji}$$

En particular,

$$\sigma_{ii} = E[(X_i - \mu_i)(X_i - \mu_i)^T] = Var(X_i) = \sigma_i^2$$

Propiedades:

- Σ_X es simétrica.
- Los elementos de la diagonal son no negativos.
- La clase de matrices de covarianza (dimensión $p \times p$) coincide con la clase de matrices simétricas semidefinidas positivas (dimensión $p \times p$).

En relación con la tercera propiedad, nos centramos en el caso en el que Σ es definida positiva. En este caso, Σ es regular con $\det(\Sigma) > 0$ y por tanto $\exists \Sigma^{-1}$. En este contexto vamos a hacer la siguiente definición.

Definición 2.2.7 Distancia de Mahalanobis. Sea $X \sim (\mu, \Sigma)$ con Σ definida positiva. La **distancia de Mahalanobis** con respecto al vector media μ se obtiene como:

$$\Delta(X, \mu) := ((X - \mu)^T \Sigma^{-1} (X - \mu))^{\frac{1}{2}}$$

Esta distancia mide cuán lejos está un punto de una distribución con media y matriz de covarianzas (μ, Σ) .

[11]

2.2.4. Resultados sobre inferencia estadística

Consideremos un experimento estadístico que da lugar a resultados x , que son los valores de un vector aleatorio X . Sea F la **función de distribución** de X . En la práctica, no se conocerá completamente la función F , es decir, algún parámetro asociado a F será desconocido. La labor de un estadístico es de estimar estos parámetros o comprobar la veracidad de ciertas proposiciones. Se pueden obtener N observaciones independientes de X . Esto significa que se observan N valores $x_1, \dots, x_N$ del vector aleatorio X . Cada x_i se puede ver como el valor de un vector aleatorio X_i , $i = 1, \dots, N$ donde $X_1, \dots, X_N$ son vectores aleatorios independientes con función de distribución común F . A $X_1, \dots, X_N$ se le conoce como **muestra de tamaño N** tomada de una población. Al conjunto de valores $x_1, \dots, x_N$ se le denota como **realización** de la muestra. Pasamos a formalizar algunos de estos conceptos. [12]

Definición 2.2.8 Muestra aleatoria simple. Sea X un vector aleatorio con función de distribución F y sean $X_1, \dots, X_N$ **variables aleatorias independientes e idénticamente distribuidas (iid)** con función de distribución común F . Entonces a la colección $X_1, \dots, X_N$ se le llama **muestra aleatoria simple** de tamaño N de la función de distribución F o simplemente N observaciones independientes de X . [12]

Definición 2.2.9 Estadístico. Sea $X_1, \dots, X_N$ una muestra aleatoria simple de una variable aleatoria X y sea $T : \mathbb{R}^N \rightarrow \mathbb{R}^K$ una función medible en el sentido de Borel. Entonces a la variable aleatoria $T(X_1, \dots, X_N)$ se le conoce como **estadístico**. [12]

Procedemos ahora a definir algunos estadísticos interesantes.

Definición 2.2.10 Media muestral. Sea $(X_1, \dots, X_N)$ una muestra aleatoria simple de tamaño N . La **media muestral** es el vector:

$$\bar{X} := \frac{1}{N} \sum_{i=1}^N X_i = \begin{pmatrix} \frac{1}{N} \sum_{i=1}^N X_{i1} \\ \vdots \\ \frac{1}{N} \sum_{i=1}^N X_{ip} \end{pmatrix} =: \begin{pmatrix} \bar{X}_1 \\ \vdots \\ \bar{X}_p \end{pmatrix}$$

[11]

Definición 2.2.11 Matriz de dispersión muestral. Sea $(X_1, \dots, X_N)$ una muestra aleatoria simple de tamaño N . La **Matriz de dispersión muestral** es:

$$A := \sum_{i=1}^N (X_i - \bar{X})(X_i - \bar{X})^T$$

$$= \begin{pmatrix} \sum_{i=1}^N (X_{i1} - \bar{X}_1)(X_{i1} - \bar{X}_1)^T & \sum_{i=1}^N (X_{i1} - \bar{X}_1)(X_{i2} - \bar{X}_2)^T & \cdots & \sum_{i=1}^N (X_{i1} - \bar{X}_1)(X_{ip} - \bar{X}_p)^T \\ \sum_{i=1}^N (X_{i2} - \bar{X}_2)(X_{i1} - \bar{X}_1)^T & \sum_{i=1}^N (X_{i2} - \bar{X}_2)(X_{i2} - \bar{X}_2)^T & \cdots & \sum_{i=1}^N (X_{i2} - \bar{X}_2)(X_{ip} - \bar{X}_p)^T \\ \vdots & \vdots & \ddots & \vdots \\ \sum_{i=1}^N (X_{ip} - \bar{X}_p)(X_{i1} - \bar{X}_1)^T & \sum_{i=1}^N (X_{ip} - \bar{X}_p)(X_{i2} - \bar{X}_2)^T & \cdots & \sum_{i=1}^N (X_{ip} - \bar{X}_p)(X_{ip} - \bar{X}_p)^T \end{pmatrix}$$

Definimos ahora la **covarianza muestral** $\rightarrow S_N^2 = \frac{A}{N}$ y la **cuasivarianza muestral** $\rightarrow S_{N-1}^2 = \frac{A}{N-1}$.

[11]

Definición 2.2.12 Función de distribución muestral. Sea $(X_1, \dots, X_N)$ una muestra aleatoria simple de tamaño N de un vector aleatorio X con función de distribución F_X . Se define la **función de distribución muestral** como:

$$F_{X_1, \dots, X_N}^*(x) = \frac{\sum_{i=1}^N I_{(-\infty, x]}(X_i)}{N}$$

donde

$$I_{(-\infty, x]}(X_i) = \begin{cases} 1 & \text{si } X_i \leq x \\ 0 & \text{si } X_i > x \end{cases}$$

[13]

Teorema 2.2.13 Glivenko-Cantelli. $F_{X_1, \dots, X_N}^*$ converge uniformemente a F , es decir, para $\epsilon > 0$:

$$\lim_{N \rightarrow \infty} P\left\{ \sup_{-\infty < x < \infty} |F_{X_1, \dots, X_N}^*(x) - F(x)| > \epsilon \right\} = 0$$

[12]

Este teorema es muy útil pues nos dice que si el tamaño de la muestra crece, la función de distribución empírica se acerca a la verdadera en todos los puntos.

Procedemos ahora a definir un concepto muy importante en cuanto a estimación de parámetros. Este método lo llamamos el método de **estimación de máxima verosimilitud**. Para ilustrar el concepto definimos el siguiente ejemplo:

Ejemplo 2.2.14. Sea $X \sim b(n, p)$. Esta distribución tiene la siguiente función masa de probabilidad:

$$P(X = x) = \binom{n}{x} p^x (1-p)^{n-x}, \quad x = 0, 1, \dots, n.$$

Supongamos que en las observaciones sobre X , sabemos que $n = 2$ o $n = 3$ y que $p = \frac{1}{2}$ o $p = \frac{1}{3}$.

El objetivo es estimar el par (n, p) .

x	$(2, \frac{1}{2})$	$(2, \frac{1}{3})$	$(3, \frac{1}{2})$	$(3, \frac{1}{3})$	Probabilidad máxima
0	$\frac{1}{4}$	$\frac{4}{9}$	$\frac{1}{8}$	$\frac{8}{27}$	$\frac{4}{9}$
1	$\frac{1}{2}$	$\frac{4}{9}$	$\frac{3}{8}$	$\frac{12}{27}$	$\frac{1}{2}$
2	$\frac{1}{4}$	$\frac{1}{9}$	$\frac{3}{8}$	$\frac{6}{27}$	$\frac{3}{8}$
3	0	0	$\frac{1}{8}$	$\frac{1}{27}$	$\frac{1}{8}$

La última columna representa la probabilidad máxima en cada fila, que es para cada valor que toma X . Si se observa el valor $x = 1$, lo más probable es que $X \sim b(2, \frac{1}{2})$. Entonces el estimador que maximiza la probabilidad del valor observado es:

$$(\hat{n}, \hat{p})(x) = \begin{cases} (2, \frac{1}{3}) & \text{si } x = 0, \\ (2, \frac{1}{2}) & \text{si } x = 1, \\ (3, \frac{1}{2}) & \text{si } x = 2, \\ (3, \frac{1}{3}) & \text{si } x = 3, \end{cases}$$

El principio de máxima verosimilitud asume que la muestra es representativa de la población y elige como estimador el valor del parámetro que maximiza la función masa de probabilidad o la función de densidad f_θ .

Definición 2.2.15. Función de verosimilitud. Sea $(X_1, \dots, X_n)$ un vector aleatorio con función de densidad (función masa de probabilidad en el caso discreto) $f_\theta(x_1, \dots, x_n)$ con $\theta \in \Theta$. La función:

$$L(\theta; x_1, \dots, x_n) = f_\theta(x_1, \dots, x_n)$$

considerada como función de θ se conoce como **función de verosimilitud**. Normalmente, θ será un parámetro de varias variables. Si $X_1, \dots, X_n$ son iid con FD (FMP) f_θ :

$$L(\theta; x_1, \dots, x_n) = \prod_{i=1}^n f_\theta(x_i)$$

Definición 2.2.16. Máxima verosimilitud. Sea el espacio de parámetros $\Theta \subset \mathbb{R}^k$ y $X = (X_1, \dots, X_n)$. El **principio de estimación de máxima verosimilitud** consiste en elegir un estimador θ de $\hat{\theta}(X)$ que maximiza $L(\theta; x_1, \dots, x_n)$, es decir, encontrar una función $\hat{\theta} : \mathbb{R}^n \rightarrow \mathbb{R}^k$ tal que:

$$L(\hat{\theta}; x_1, \dots, x_n) = \sup_{\theta \in \Theta} L(\theta; x_1, \dots, x_n)$$

En el caso de encontrar un $\hat{\theta}$ que satisfaga la ecuación anterior, se le llama **estimador de máxima verosimilitud (MLE, Maximum Likelihood Estimator)**.

Observación 2.2.16. En muchos casos es conveniente usar el logaritmo de la función de verosimilitud, debido a que la función logaritmo es monótona y por tanto es equivalente encontrar el máximo de la función de verosimilitud y el del logaritmo de la función de verosimilitud:

$$\log L(\hat{\theta}; x_1, \dots, x_n) = \sup_{\theta \in \Theta} \log L(\theta; x_1, \dots, x_n)$$

Proposición 2.2.17. Sea $\Theta \subset \mathbb{R}^k$ y supongamos que $f_\theta(x)$ es una función positiva y diferenciable de θ . Si el supremo $\hat{\theta}$ existe entonces debe satisfacer las **ecuaciones de verosimilitud**:

$$\frac{\partial \log L(\hat{\theta}; x_1, \dots, x_n)}{\partial \theta_j} = 0, \quad j = 1, 2, \dots, k \quad \theta = (\theta_1, \dots, \theta_k)$$

[12]

Ejemplo 2.2.18. Sea $X_1, \dots, X_n$ una muestra aleatoria simple de una distribución normal multivariante $\mathcal{N}_p(\mu, \Sigma)$ con Σ definida positiva donde Σ y μ son desconocidos. Tenemos entonces que:

$$\begin{aligned} L(\mu, \Sigma; x_1, \dots, x_n) &= \prod_{i=1}^n f_{\mu, \Sigma}(x_i) = \prod_{i=1}^n \frac{1}{|\Sigma|^{\frac{1}{2}} (2\pi)^{\frac{n}{2}}} \exp \left(-\frac{1}{2} \sum_{i=1}^n (x_i - \mu)^T \Sigma^{-1} (x_i - \mu) \right) = \\ &= \frac{1}{|\Sigma|^{\frac{n}{2}} (2\pi)^{\frac{pn}{2}}} \exp \left(-\frac{1}{2} \sum_{i=1}^n (x_i - \mu)^T \Sigma^{-1} (x_i - \mu) \right) \end{aligned}$$

Haciendo algunas operaciones, se puede obtener una expresión equivalente de la función de verosimilitud:

$$L(\mu, \Sigma; x_1, \dots, x_n) = \frac{1}{|\Sigma|^{\frac{n}{2}} (2\pi)^{\frac{pn}{2}}} \exp \left(-\frac{1}{2} \text{tr}(\Sigma^{-1} A) - \frac{n}{2} (\bar{x} - \mu)^T \Sigma^{-1} (\bar{x} - \mu) \right)$$

y

$$\log L(\mu, \Sigma; x_1, \dots, x_n) = -\frac{pn}{2} \log(2\pi) - \frac{n}{2} \log |\Sigma| - \frac{1}{2} \text{tr}(\Sigma^{-1} A) - \frac{n}{2} (\bar{x} - \mu)^T \Sigma^{-1} (\bar{x} - \mu)$$

Donde A es la **matriz de dispersión**. Procederemos a enunciar un lema que nos ayudará a probar un resultado de gran utilidad.

Lema 2.2.19. Lema de Watson. Sean G, D matrices simétricas definidas positivas de dimensión $p \times p$ y $f(G) = -N \log(|G|) - \text{tr}(G^{-1}D)$ con $N \in \mathbb{N}$. Entonces $\exists!$ máximo de f con respecto a G y se alcanza en $G = \frac{1}{N}D$ con:

$$f\left(\frac{1}{N}D\right) = pN \log(N) - N \log(|D|) - pN$$

Por tanto, retomando el **ejemplo 18**, vamos a ver los MLE de (μ, Σ) . Teniendo en cuenta:

$$\log L(\mu, \Sigma; x_1, \dots, x_n) = -\frac{pn}{2} \log(2\pi) - \frac{n}{2} \log |\Sigma| - \frac{1}{2} \text{tr}(\Sigma^{-1}A) - \frac{n}{2} (\bar{x} - \mu)^T \Sigma^{-1} (\bar{x} - \mu)$$

únicamente el término de la derecha depende de μ por lo que el problema es equivalente a maximizar:

$$-\frac{n}{2} (\bar{x} - \mu)^T \Sigma^{-1} (\bar{x} - \mu)$$

Al ser Σ definida positiva, $x^T \Sigma x > 0 \quad \forall x \in \mathbb{R}^p - \{0\}$. luego

$$-\frac{n}{2} (\bar{x} - \mu)^T \Sigma^{-1} (\bar{x} - \mu) \leq 0$$

Por lo que este valor alcanza su máximo en 0 lo que implica que el MLE para μ es $\bar{x}$. Una vez estimado este parámetro, tenemos que:

$$\log L(\bar{x}, \Sigma; x_1, \dots, x_n) = -\frac{pn}{2} \log(2\pi) - \frac{n}{2} \log |\Sigma| - \frac{1}{2} \text{tr}(\Sigma^{-1}A)$$

Utilizando el **Lema de Watson**, identificando $G \equiv \Sigma$ y $D \equiv A$,

$$\begin{aligned} f(\Sigma) &:= 2(\log L(\bar{x}, \Sigma; x_1, \dots, x_n) + \frac{pn}{2} \log(2\pi)) = pn \log(2\pi) + 2(\log L(\bar{x}, \Sigma; x_1, \dots, x_n)) = \\ &= -n \log(|\Sigma|) - \text{tr}(\Sigma^{-1}A) \end{aligned}$$

por lo que el máximo se alcanza en $\Sigma = \frac{A}{n} = S_n^2$ que es la **covarianza muestral**. Esto es útil porque en la práctica tendremos un método para estimar la media y matriz de covarianzas en nuestro conjunto de datos.

[11]

2.3. Optimización

2.3.1. Optimización basada en gradientes

Durante el desarrollo de esta sección, se hace uso de [1]. La mayoría de los algoritmos de aprendizaje tienen como objetivo la optimización de algún elemento. La optimización es la tarea de minimizar o maximizar una función $f(x)$ alterando el valor de x . En muchos casos, se plantean los problemas de optimización en función de minimizar una función $f(x)$, ya que maximizar esa misma función es equivalente a minimizar $-f(x)$.

La función que se pretende optimizar le damos el nombre de **función objetivo**. En el caso clásico de minimización, se suele conocer a esta función como **función de coste**, **función de pérdida** o **función de error**.

Para no extender demasiado el desarrollo de este trabajo, se asume que el lector está familiarizado con el cálculo. Aun así, se muestra un pequeño resumen de ciertos conceptos con el objetivo de entender cómo se procede en optimización.

Consideramos una función de variable real $f : \mathbb{R} \rightarrow \mathbb{R}$ tal que $y = f(x)$. La **derivada** de esta función, notada por $f'(x)$ o $\frac{dy}{dx}$, nos indica la pendiente a la recta tangente a $f(x)$ en el punto x .

En otras palabras, especifica cómo debe escalarse un pequeño cambio en la entrada para obtener el cambio correspondiente en la salida:

$$f(x + \epsilon) \approx f(x) + \epsilon f'(x)$$

El uso de derivadas es muy útil para la minimización de una función porque indica como variar x para conseguir pequeños cambios en y . Por ejemplo, sabemos que $f(x - \epsilon \text{sign}(f'(x)))$ es más pequeño que $f(x)$ para un ϵ suficientemente pequeño. Podemos reducir $f(x)$ ajustando el valor de x en pequeños pasos en el sentido opuesto a la derivada. Este concepto da lugar a la técnica del **gradiente descendente**.

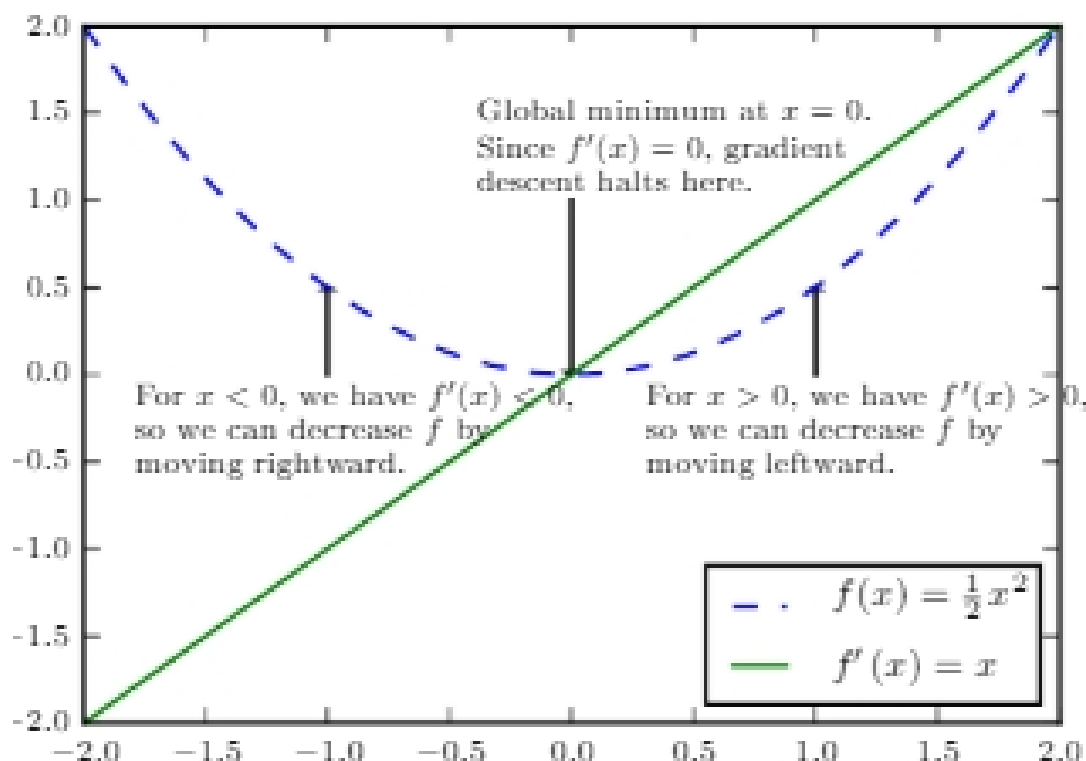


Figura 1: Ejemplo de gradiente descendente para función de variable real, [1]

Como vemos en la figura 1, tenemos un método para acercarnos al mínimo teniendo en cuenta el signo de la derivada.

Cuando $f'(x) = 0$, la derivada no proporciona información sobre en qué dirección moverse. Estos puntos son conocidos como **puntos críticos**. Un **mínimo local** (resp. **máximo local**) es un punto donde $f(x)$ es menor (resp. mayor) que el valor de la función en sus puntos vecinos. Algunos puntos críticos no son ni máximos ni mínimos locales. Estos puntos son conocidos como **puntos de silla**. En la siguiente figura podemos ver de estos tres tipos de puntos críticos:

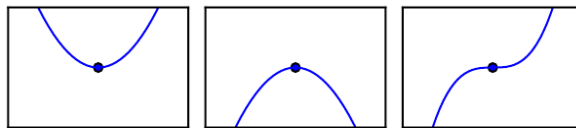


Figura 2: Ejemplo de puntos de silla para función de variable real. De izquierda a derecha **mínimo**, **máximo** y **punto de silla**. [1]

Para funciones cuya entrada sea un vector de varias variables, se hace uso de **derivadas parciales**. La derivada parcial $\frac{\partial}{\partial x_i} f(x)$ indica como varía f en el punto x si realizamos pequeñas variaciones en x_i . El **gradiente** generaliza el concepto de derivada en el caso en el que la variable de derivación es un vector. Esta operación se denota por $\nabla f(x)$ y en la posición i contiene $\frac{\partial}{\partial x_i} f(x)$. Para el caso de varias variables, los **puntos críticos** se dan cuando $\nabla f(x) = (0, \dots, 0)$.

La **derivada direccional** en la dirección de u con u vector unitario (i.e. $\|u\| = 1$) es la pendiente de la recta tangente a un punto en la dirección de u . De forma más precisa, la derivada direccional se puede representar como:

$$D_{\mathbf{u}}f(x) = \left. \frac{d}{d\alpha} f(x + \alpha \mathbf{u}) \right|_{\alpha=0}$$

Usando la regla de la cadena, tenemos que:

$$D_{\mathbf{u}}f(x) = \left. \frac{d}{d\alpha} f(x + \alpha \mathbf{u}) \right|_{\alpha=0} = u^T \nabla f(x)$$

Para **minimizar** f , debemos encontrar la dirección u en la que f disminuye de forma más rápida.

Para ello, podemos hacer uso de la derivada direccional:

$$\min_{u, u^T u = 1} u^T \nabla f(x) = \min_{u, u^T u = 1} \|u\| \|\nabla f(x)\| \cos \theta$$

donde θ es el ángulo entre u y el gradiente. Teniendo en cuenta que u es unitario y por tanto $\|u\| = 1$, Este problema es equivalente a $\min_u \cos \theta$. Como sabemos $\cos : \mathbb{R} \rightarrow [-1, 1]$ por lo que $\min_u \cos \theta = -1 \Rightarrow \theta = \pi + 2\pi k, \quad k \in \mathbb{Z}$. Por tanto, nuestra expresión se minimiza cuando u apunta en dirección contraria al gradiente. Podemos entonces disminuir f moviéndonos en dirección contraria al gradiente. Este método es conocido como **método del gradiente descendente**.

Definición 2.3.1 Gradiente descendente Sea $f : \mathbb{R}^N \rightarrow \mathbb{R}$. El algoritmo del gradiente descendente busca minimizar la función f variando x en dirección contraria al gradiente:

$$x' = x - \epsilon \nabla f(x) \quad \epsilon > 0$$

Al parámetro ϵ se le conoce como **tasa de aprendizaje** y representa el paso que damos en dirección contraria al gradiente.

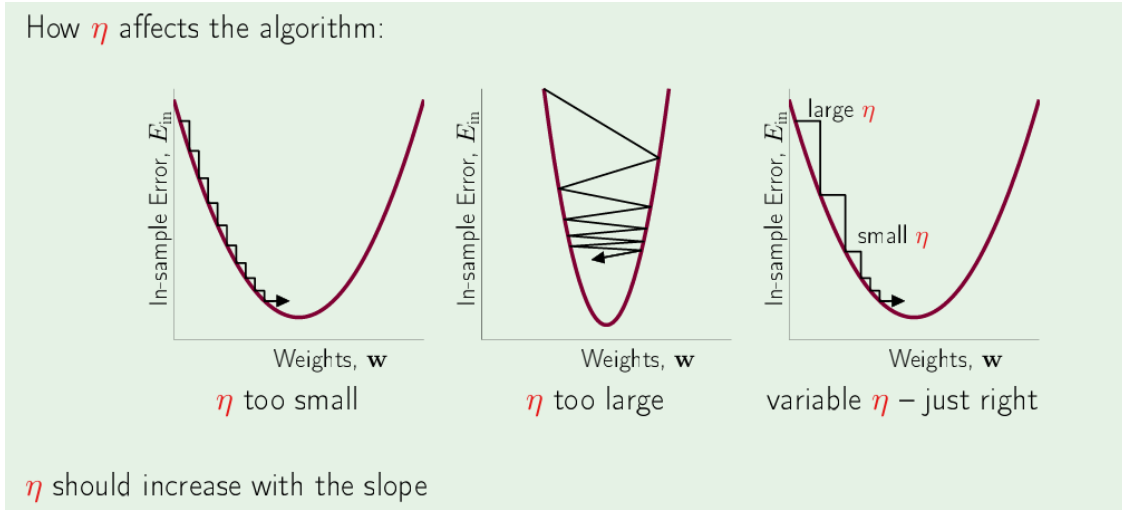


Figura 3: Ejemplo de gradiente descendente y como afecta el valor de la tasa de aprendizaje.[2]

Como podemos ver, tenemos un ejemplo de como funciona el método del gradiente descendente en una función convexa. Además, se ve la importancia de selección de un valor adecuado de la **tasa de aprendizaje**, **notada esta vez por η** , donde un valor muy pequeño o muy grande puede dar lugar a encontrar el valor del mínimo muy tarde.

2.3.2. Matriz Jacobiana y Hessiana

Definición 2.3.2, Matriz Jacobiana. Sea $f : \mathbb{R}^M \rightarrow \mathbb{R}^N$ y $f(x) = (f_1(x), \dots, f_N(x))$ con $f_i : \mathbb{R}^M \rightarrow \mathbb{R}, \quad i = 1, \dots, N$. La **matriz Jacobiana** es la matriz que contiene las derivadas parciales

de cada componente de f_i de f respecto a cada variable, es decir $J_f \in \mathbb{R}^{N \times M}$ tal que en un punto $x \in \mathbb{R}^M$:

$$J_f(x) = \begin{pmatrix} \frac{\partial f_1}{\partial x_1}(x) & \frac{\partial f_1}{\partial x_2}(x) & \cdots & \frac{\partial f_1}{\partial x_M}(x) \\ \frac{\partial f_2}{\partial x_1}(x) & \frac{\partial f_2}{\partial x_2}(x) & \cdots & \frac{\partial f_2}{\partial x_M}(x) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_N}{\partial x_1}(x) & \frac{\partial f_N}{\partial x_2}(x) & \cdots & \frac{\partial f_N}{\partial x_M}(x) \end{pmatrix} = \begin{pmatrix} \nabla f_1(x) \\ \nabla f_2(x) \\ \vdots \\ \nabla f_N(x) \end{pmatrix}$$

Además, puede ser interesante conocer la derivada de la derivada, también conocida como **segunda derivada**. Por ejemplo, considerando la función de variable real $f : \mathbb{R} \rightarrow \mathbb{R}$, la segunda derivada se trata de $\frac{d^2}{dx^2}f(x)$, denotada también por $f''(x)$. La segunda derivada nos dice cómo variará la primera derivada a medida que cambiamos la entrada. Esto es importante porque indica si un paso de gradiente provocará una mejora tan grande como la que esperaríamos basándonos únicamente en el gradiente. Podemos considerar la segunda derivada como una medida de la **curvatura** de la función.

Definición 2.3.3, Matriz Hessiana. Sea $f : \mathbb{R}^N \rightarrow \mathbb{R}$. La **Matriz Hessiana** es la que contiene en la posición (i, j) la derivada de f con respecto a x_i y x_j , es decir:

$$H_f(x) = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2}(x) & \frac{\partial^2 f}{\partial x_1 \partial x_2}(x) & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_N}(x) \\ \frac{\partial^2 f}{\partial x_2 \partial x_1}(x) & \frac{\partial^2 f}{\partial x_2^2}(x) & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_N}(x) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_N \partial x_1}(x) & \frac{\partial^2 f}{\partial x_N \partial x_2}(x) & \cdots & \frac{\partial^2 f}{\partial x_N^2}(x) \end{pmatrix}$$

2.4. Fundamentos matemáticos de Machine Learning

De nuevo, en esta sección se hace uso de [1].

2.4.1. Algoritmos de aprendizaje

Un **algoritmo de machine learning** es un algoritmo capaz de aprender a partir de datos. El concepto de aprendizaje parece reservado para entidades capaces de pensar de alguna forma, como seres humanos y otros seres vivos. Entonces, se debe especificar qué se entiende realmente por aprendizaje por parte de un modelo. Según [14], podemos definir el concepto de aprendizaje como lo siguiente:

Definición 2.4.1, Aprendizaje en programas informáticos. Un programa informático se dice que aprende de la **experiencia E** con respecto a una clase de **tareas T** y alguna forma de **medir el rendimiento P**, si su rendimiento en las tareas de **T** medido bajo **P** mejora con la experiencia **E**.

El **machine learning** permite abordar ciertas tareas que son demasiado difíciles para programas fijos diseñados e implementados por seres humanos. Las tareas relativas al machine learning normalmente describen como el modelo de machine learning debe procesar un **ejemplo**.

Definición 2.4.2, Ejemplo. Un **ejemplo** es una colección de **características** que han sido cuantitativamente medidas relativas a un objeto o evento con intención de ser procesadas por nuestro modelo de machine learning. Por lo general, un ejemplo estará representado en forma de vector $x \in \mathbb{R}^N$ o en forma de imagen $x \in \mathbb{R}^{M \times N}$.

2.4.1.1. Problema de clasificación

Existen numerosos tipos de tareas que llevan a cabo los algoritmos de machine learning. De entre todas ellas, vamos a detallar el caso de la tarea de **clasificación**, que es uno de los objetivos prácticos de este trabajo.

Definición 2.4.3, Clasificación. Una tarea de clasificación es aquella en la que el objetivo del algoritmo es generar una función $f : \mathbb{R}^{M \times N} \rightarrow \{1, \dots, k\}$ de forma que a cada **ejemplo** se le asigna una de las categorías identificadas por un número $y \in \{1, \dots, k\}$.

Existen otras variantes del problema de clasificación como por ejemplo que la aplicación devuelve una distribución de probabilidad sobre las distintas clases. Un ejemplo de clasificación es la clasificación de un objeto que aparece en una imagen.

2.4.1.2. Aprendizaje supervisado

Definición 2.4.4, Aprendizaje supervisado. El aprendizaje supervisado es la tarea que tiene como objetivo aprender la función $f : \mathbb{X} \rightarrow \mathbb{Y}$ donde $\mathbb{X}$ es el conjunto de entradas e $\mathbb{Y}$ el conjunto de salidas. [15].

Como hemos comentado anteriormente, en el caso de clasificación tenemos que aprender la función $f : \mathbb{R}^{M \times N} \rightarrow \{1, \dots, k\}$. Según [16], consideramos en este caso el **dataset** D formado por los pares $\{(x_1, y_1), \dots, (x_N, y_N)\} \subset \mathbb{X} \times \mathbb{Y}$ (asumiendo N entradas). Con este conjunto de datos, se pretende estimar el predictor $f_\theta : \mathbb{R}^{M \times N} \rightarrow \{1, \dots, k\}$ parametrizado por θ . El objetivo es, dado el conjunto de ejemplos, encontrar un parámetro θ^* de forma que:

$$f_{\theta^*}(x_n) \approx y_n \quad \forall n = 1, \dots, N$$

Utilizando un lenguaje más natural, el **aprendizaje supervisado** hace uso de un conjunto de datos de características donde, a cada ejemplo $\mathbf{x}$, se le asocia una **etiqueta o label** $\mathbf{y}$. Este tipo de aprendizaje se basa en la observación de **arrays aleatorios** $\mathbf{x}$ a los que se les asocia un valor $\mathbf{y}$, para aprender a predecir **la etiqueta y desde el ejemplo x**, por lo que el objetivo final se podría decir que es **estimar la distribución de probabilidad condicionada** $p(y|x)$.

Algunas de las tareas más comunes del aprendizaje supervisado son:

- **Clasificación:** Caso ya comentado, se pretende aprender $f : \mathbb{R}^{M \times N} \rightarrow \{1, \dots, k\}$.
- **Regresión:** Parecido a clasificación llevado al caso continuo. Se trata de estimar $f : \mathbb{R}^{M \times N} \rightarrow \mathbb{R}$, es decir, predecir un valor.
- **Problemas de salidas estructuradas:** Este caso, parecido a los casos anteriores, las salidas son estructuras más complejas que una categoría o un valor numérico.

2.4.1.3. Aprendizaje no supervisado

En el aprendizaje supervisado, se asume que para cada **ejemplo** $\mathbf{x}$ tenemos asociadas una serie de **labels** $\mathbf{y}$ y el objetivo es aprender la aplicación que asocia ejemplos a etiquetas. Pese a su evidente utilidad, se puede considerar que este tipo de aprendizaje es en esencia una forma sofisticada de ajuste de curvas.

Una tarea posiblemente más interesante puede ser la de 'dar sentido' a los datos, desconociendo qué es lo que el modelo debe proporcionar como salida.

Definición 2.4.5, Aprendizaje no supervisado. El aprendizaje no supervisado se encarga de observar **entradas** $D = \{x_n : n = 1, \dots, N\}$ sin **etiquetas** y_n con el objetivo de que el modelo sea el que proporcione información sobre los datos. [15].

En este caso, se observan **arrays aleatorios** $\mathbf{x}$ con el objetivo de determinar la distribución de probabilidad $p(x)$. Los conceptos de **aprendizaje supervisado y no supervisado** no son términos completamente definidos y separados, sino que los límites entre ellos a veces son difusos. Por ejemplo, la regla de la cadena nos dice que, para un vector $x \in \mathbb{R}^N$, la distribución conjunta se puede descomponer como:

$$p(x) = \prod_{i=1}^N p(x_i | x_1, \dots, x_{i-1}).$$

Con esta descomposición, transformamos un problema de aprendizaje no supervisado en N problemas de aprendizaje supervisado. Alternativamente, podemos resolver un problema de aprendizaje supervisado de estimar $p(y|x)$ usando técnicas de aprendizaje no supervisado para estimar la distribución conjunta $p(x, y)$ e inferir a continuación:

$$p(y|x) = \frac{p(x, y)}{\sum_{y'} p(x, y')}$$

Algunas de las tareas más comunes del aprendizaje no supervisado son:

- **Clustering:** El objetivo es agrupar los datos entre regiones (**clusters**) que contienen puntos considerados "similares".

- **Descubrir factores latentes:** Tiene como objetivo reducir la dimensionalidad proyectándolos a un subespacio de menor dimensión que capture la “esencia” de los datos. Una forma de abordar este problema es asumir que cada observación de alta dimensión $x \in \mathbb{R}^D$ fue generada por un conjunto de factores latentes (ocultos o no observados) de baja dimensión $z \in \mathbb{R}^K$.

[15]

2.4.1.4. Función de coste

En general, los algoritmos de machine learning están basados en optimización. Es por esto que se necesita una función objetivo que registre el comportamiento del modelo con objetivo de optimizarla.

Definición 2.4.6, Función de coste. La **función de coste** es una función que cuantifica el error cometido por un modelo de aprendizaje automático sobre un conjunto de datos. Dado que el predictor f_θ que se pretende estimar viene determinado por un conjunto de parámetros θ , la función de coste depende de dichos parámetros y puede expresarse como $L(\theta)$. El objetivo del proceso de entrenamiento consiste en encontrar los valores de θ que minimizan esta función.

La minimización del coste de un algoritmo de aprendizaje está estrechamente relacionada con la teoría de **máxima verosimilitud** (detallada en 2.2.4). Suponiendo un parámetro θ y una observación x , definimos la pérdida de θ sobre x como:

$$\mathbb{L}(\theta, x) = -\log(P_\theta(x))$$

Llamamos a $\mathbb{L}(\theta, x)$ la negativa del logaritmo de la función de verosimilitud en la observación x , suponiendo que los datos se distribuyen conforme a la distribución P_θ . Por tanto, queremos minimizar esta función a lo largo del conjunto de datos, es decir, encontrar:

$$\operatorname{argmin}_\theta \sum_{i=1}^N (-\log(P_\theta(x_i)))$$

donde N es el número de muestras.

[17]

Consideramos el problema de clasificación de **k clases**. Sea $\mathbb{X} \subset \mathbb{R}^d$ el espacio de características e $\mathbb{Y} = \{1, \dots, k\}$. Tenemos el conjunto de datos $D = \{(x_i, y_i) : i = 1, \dots, n\} \subset \mathbb{X} \times \mathbb{Y}$. Como hemos descrito previamente, el problema de clasificación consiste en estimar un predictor f_θ tal que $f_\theta(x_i) = \hat{y}_i$ teniendo como objetivo predecir la etiqueta real y_i . Para una **función de coste**, se mide el riesgo del predictor como $\mathbb{E}_D[\mathbb{L}(f_\theta(x), y)]$. Nótese que en este caso, hemos cambiado la notación de $\mathbb{L}(\theta, x)$ a $\mathbb{L}(f_\theta(x), y)$ ya que entendemos que en este caso, se calcula el riesgo en base al predictor f_θ y por tanto identificamos $f_\theta \equiv P_\theta$ en la **definición 2.4.6**.

Definición 2.4.7, Entropía cruzada para problema de clasificación. En las condiciones

anteriores y tomando como la función de pérdida $\mathbb{L}$ (entropía cruzada), se define la **Entropía cruzada para problema de clasificación**:

$$\mathbb{E}_D[\mathbb{L}(f_\theta(x), y)] = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^k y_{ij} \log(f_\theta(x_i)_j)$$

donde y_{ij} es el valor de la probabilidad de la etiqueta j para la muestra i y $f_\theta(x_i)_j$ es la probabilidad de que el predictor f_θ asigne a x_i la etiqueta j . Para ser más precisos con y_{ij} , consideramos que la etiqueta y_i del ejemplo x_i está en formato **one hot**, es decir, si $y_i = j$ entonces $y_{ij} = (0, \dots, \overset{j}{1}, \dots, 0)$. Entonces, la expresión $\sum_{j=1}^k y_{ij} \log(f_\theta(x_i)_j)$ realmente se traduce en calcular el logaritmo de la probabilidad que el predictor ha asignado a que la muestra x_i pertenece su verdadera clase y_i . Esto se da porque:

$$y_{ij} \log(f_\theta(x_i)_j) = \begin{cases} \log(f_\theta(x_i)_j) & \text{si } y_i = j \\ 0 & \text{si } y_i \neq j \end{cases}$$

[18]

Como otro ejemplo de función de pérdida, retomamos el **problema de regresión**. Este problema se puede ver como un problema de clasificación $f_\theta : \mathbb{X} \rightarrow \mathbb{Y}$ donde el conjunto de etiquetas se extiende al caso continuo, es decir, $\mathbb{Y} = \mathbb{R}$ con $D = \{(x_i, y_i) : i = 1, \dots, n\} \subset \mathbb{X} \times \mathbb{R}$. En este caso, normalmente se toma como función de pérdida una función que calcula la distancia entre el valor real y el proporcionado por el predictor.

Definición 2.4.8, Error cuadrático medio (MSE). Consideramos el problema de regresión descrito previamente. El **error cuadrático medio** se define como:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - f_\theta(x_i))^2$$

Que tiene como función de pérdida asociada:

$$\mathbb{E}_D[\mathbb{L}(f_\theta(x), y)] = -\frac{1}{n} \sum_{i=1}^n (y_i - f_\theta(x_i))^2$$

2.4.1.5. Optimizador

Una vez llegados a este punto, hemos definido el problema y la función objetivo a minimizar y debemos definir cómo vamos a minimizar esa función. El algoritmo que busca minimizar la función de pérdida se conoce como **optimizador**.

Un gran número de modelos de aprendizaje automático y sobre todo de aprendizaje profundo (detallado más adelante), se basan en el algoritmo **descenso estocástico de gradiente o stochastic**

gradient descent (SGD). Este algoritmo se trata de una extensión del algoritmo de descenso de gradiente visto en la sección 2.3.1. En el machine learning existe la necesidad de conjuntos de datos suficientemente grandes para una buena generalización del modelo. No obstante, esto trae consigo el problema de que conjuntos de datos muy grandes tienen un coste computacional elevado, tanto en tiempo de ejecución como en consumo de memoria.

La función de coste de un modelo de machine learning suele ser la suma sobre el conjunto de ejemplos de la función de pérdida en un ejemplo. Por ejemplo, la negativa del logaritmo de la función de verosimilitud se puede escribir como:

$$J(\theta) = \mathbb{E}_D[\mathbb{L}(f_\theta(x), y)] = \frac{1}{n} \sum_{i=1}^n \mathbb{L}(f_\theta(x_i), y_i)$$

donde $D = \{(x_i, y_i) : i = 1, \dots, n\}$, f_θ es la función a aproximar por el modelo dependiente de ciertos parámetros θ y $\mathbb{L}(f_\theta(x), y) = -\log(p(y|x; f_\theta))$ es la función de pérdida por cada ejemplo.

Para esta suma de funciones de pérdida, el gradiente descendente calcula:

$$\nabla_\theta J(\theta) = \frac{1}{n} \sum_{i=1}^n \nabla_\theta \mathbb{L}(f_\theta(x_i), y_i)$$

El coste computacional de esta operación es $O(m)$ y por tanto para un conjunto de datos muy grande, el cálculo de un solo paso de gradiente puede ser demasiado costoso. La idea fundamental de **SGD** es tratar al gradiente como una esperanza. La esperanza puede ser aproximada por un conjunto pequeño de ejemplos. En particular, en cada paso del algoritmo se selecciona un conjunto de ejemplos llamado **minibatch** $\mathbb{B} = \{x_{i1}, \dots, x_{in'}\} \subset \mathbb{X}$ tal que $|\mathbb{B}| = n'$ seleccionados de forma uniforme del conjunto de datos. El valor n' es relativamente pequeño, desde simplemente $n' = 1$ hasta varios cientos de ejemplos. De este modo estimamos el cálculo de gradientes usando una cantidad mucho menor de ejemplos de forma aleatoria (de ahí que se le llame descenso estocástico de gradiente). Por tanto el cálculo queda reducido a:

$$g = \frac{1}{n'} \sum_{x_i \in \mathbb{B}} \nabla_\theta \mathbb{L}(f_\theta(x_i), y_i)$$

y finalmente el **descenso estocástico de gradiente** realiza:

$$\theta \leftarrow \theta - \epsilon g$$

donde ϵ es la **tasa de aprendizaje** (2.3.1).

Procedemos ahora a introducir ciertos tipos de optimizadores de acuerdo a [19]. Comenzaremos con una variante del **SGD** y es el concepto de **momentum**. El momentum es una modificación de SGD que acumula una media del valor histórico de los gradientes para acelerar convergencia en valles. El método utiliza el valor de **momentum** γ (típicamente 0.9) para calcular un término de velocidad v_t de forma que:

$$v_t = \gamma v_{t-1} + \epsilon \nabla_{\theta} J_t(\theta)$$

donde $\nabla_{\theta} J_t(\theta)$ son los gradientes calculados en tiempo t . Por último:

$$\theta \leftarrow \theta - v_t$$

De forma intuitiva, este algoritmo simula el movimiento de una pelota cayendo por una pendiente donde el optimizador gana velocidad en direcciones con gradientes consistentes mientras que reduce oscilaciones en direcciones con gradientes que cambian de signo.

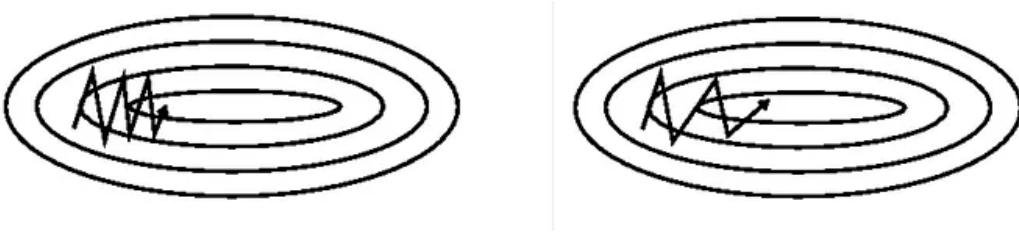


Figura 4: SGD sin momentum (izda), SGD con momentum (dcha) [3]

Seguimos con el optimizador **AdaGrad**. Este método adapta la tasa de aprendizaje en función del historial del gradiente. Cada parámetro θ_i tiene su propia tasa de aprendizaje que disminuye con la suma del cuadrado de los gradientes en dicho parámetro. La actualización para el parámetro i en tiempo t es:

$$G_{t,i} = G_{t-1,i} + (\nabla_{\theta} J_t(\theta))^2$$

$$\theta_i \rightarrow \theta_i - \frac{\epsilon}{\sqrt{G_{t,i} + \rho}} \nabla_{\theta} J_t(\theta)$$

donde ρ es una constante pequeña para evitar divisiones por 0. Intuitivamente, parámetros con gradientes grandes obtienen tasas de aprendizaje más pequeñas mientras que los que tienen gradientes más pequeños obtienen pasos más grandes. Esto es lógico, como vimos en la figura 3, si la pendiente es muy grande, una tasa de aprendizaje muy grande puede hacer que 'saltemos' el mínimo y si la pendiente es muy pequeña, una tasa de aprendizaje pequeña puede hacer que la convergencia sea muy lenta.

2.5. Fundamentos matemáticos del Deep Learning

2.5.1. Perceptrón y perceptrón multicapa

Procedemos a hacer uso de nuevo del recurso [1] para describir los fundamentos del deep learning. El deep learning es un caso particular del machine learning, y por tanto tiene elementos comunes como funciones de pérdida u optimizador. Las **redes neuronales o perceptrones multicapa (MLP)** son el concepto básico del aprendizaje profundo o **deep learning**. Del mismo modo que

en los modelos de aprendizaje automático, el objetivo es aproximar una función f . Por ejemplo, en el caso de clasificación, tenemos el objetivo de encontrar el predictor f_θ aprendiendo el parámetro θ que aproxime mejor f .

Estos modelos se basan en la propagación hacia delante porque la información fluye desde las capas iniciales que componen f_θ hasta las capas finales y, finalmente, hacia la salida y . No existe propagación hacia atrás en el sentido de que dentro de las capas del modelo, capas finales no proporcionan información a capas iniciales.

La **redes neuronales** tienen el nombre de redes porque están compuestas por varias funciones diferentes. Estos modelos están representados por un grafo dirigido acíclico describiendo como estas funciones se componen entre sí. Por ejemplo, para $f^{(1)}, f^{(2)}, f^{(3)}$ conectadas en cadena, tenemos $f(x) = f^{(3)}(f^{(2)}(f^{(1)}(x)))$. A $f^{(1)}$ la denominamos como **primera capa** y, en general, $f^{(i)}$ es la **capa i -ésima**. La longitud de la cadena de capas es la **profundidad** del modelo. Es esta la razón por la que se conoce a esta rama como **aprendizaje profundo o deep learning**. La última capa de una red neuronal se conoce como **capa de salida**.

Para un problema de aprendizaje supervisado, esperamos encontrar f_θ que aproxime f . Cada ejemplo x es acompañado por una etiqueta y . Los ejemplos de entrenamiento especifican qué debe proporcionar la capa de salidas, es decir, proporcionar un valor lo más fiel posible a y . Sin embargo, el comportamiento de las capas restantes no es especificado por el conjunto de datos, sino que es el algoritmo de aprendizaje quién debe decidir como usar cada capa para producir la salida deseada. El hecho de que el conjunto de datos no decida el comportamiento de estas capas da lugar a llamar a estas capas **capas ocultas**.

Hemos comentado el por qué las redes neuronales tienen el nombre de redes y se va a describir por qué tienen el apellido 'neuronales'. Se denominan así porque su diseño están ciertamente inspiradas en el ámbito de la neurociencia. Cada capa oculta de la red suele estar representada por un valor. La dimensión de estas capas ocultas determina el **ancho** del modelo. Cada elemento del vector se puede interpretar como una unidad análoga a una neurona. En lugar de que cada capa representa una función que espera y devuelve un vector, es más preciso pensar que cada capa contiene una serie de **unidades** llamadas **perceptrones** que trabajan en paralelo para que cada una devuelva un escalar a partir de un vector. El comportamiento de una unidad es similar al de una neurona pues cada unidad recibe información sobre múltiples unidades externas y calcula su propio valor de activación.

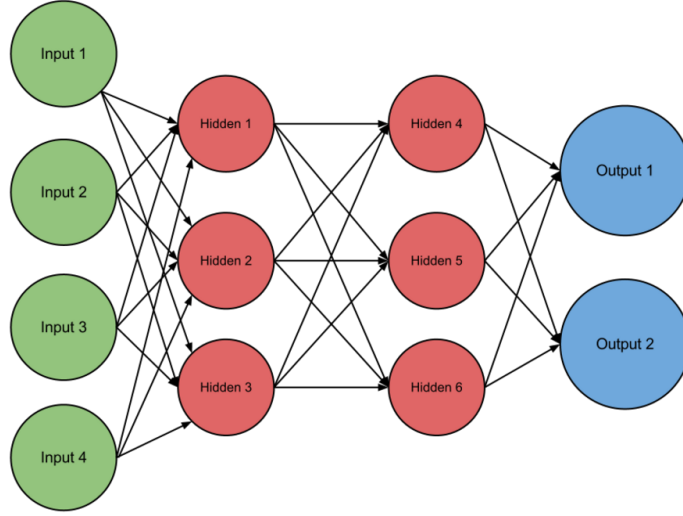


Figura 5: Ejemplo de MLP, <https://commons.wikimedia.org/wiki/File:FeedForwardNN.png>

En esta figura se muestra una posible estructura de una red neuronal compuesta por la **capa de entrada (verde)**, **capas ocultas (naranja)** y **capa de salida (azul)**. Como podemos comprobar, vemos la propagación hacia delante donde cada unidad envía datos a cada unidad de la capa siguiente, además de la estructura de grafo acíclico dirigido. En este punto, hemos descrito de forma precisa qué es una red neuronal, pero es desconocido el funcionamiento de cada unidad.

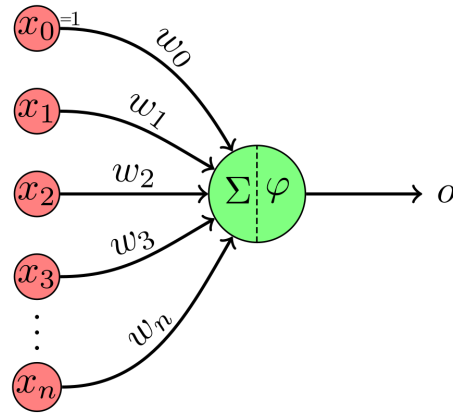


Figura 6: Ejemplo de perceptrón, <https://commons.wikimedia.org/wiki/File:Perceptron-unit.svg>

Como podemos ver en la imagen, tenemos un conjunto $I = \{x_k; k = 0, \dots, n\}$ que es el conjunto de **entradas** procedentes de una capa anterior o de datos de entrada al modelo y $W = \{w_k; k = 0, \dots, n\}$ es el conjunto de **pesos**, que son parámetros aprendidos por el modelo. Si nos fijamos en x_0 , podemos ver como $x_0 = 1$. Esta variable realmente no proviene de una unidad anterior sino que es una constante relacionada al **peso** w_0 que es un parámetro que aprende el modelo conocido como **sesgo (bias en inglés)**. Una vez definido el conjunto de inputs I y el conjunto de pesos W , se

realiza la siguiente operación:

$$z = \sum_{i=1}^n x_i w_i + w_0$$

Nótese que no hemos mencionado que esta sea la salida de la unidad y la razón de esto es que vemos que la operación es lineal. Al fin y al cabo, el objetivo es estimar una función f y no podemos asegurar la linealidad de esta función. El término φ de la imagen, desconocido hasta el momento, se encarga precisamente de cambiar esto introduciendo no linealidad al modelo. A este elemento se le conoce como **función de activación**.

2.5.2. Capa de salida

La elección de la función de coste está estrechamente relacionada con la elección de la capa de salida. En la mayoría de casos, se utiliza la **entropía cruzada** entre la distribución de los datos y la distribución del modelo. La elección de cómo representamos las salidas determina la forma de la entropía cruzada.

A lo largo de esta sección, suponemos que el modelo antes de llegar a la capa de salida, proporciona un conjunto de *features* $h = f_{\theta}(x)$. La capa de salida debe proporcionar algún tipo de transformación sobre las *features* para que la red complete su función.

2.5.2.1. Unidades con función Sigmoid

Muchos problemas consisten en predecir el valor de una variable binaria y , es decir, una clasificación binaria. La aproximación de máxima verosimilitud es definir una distribución de Bernoulli de y condicionada en x . Se dice que la variable y condicionada en x sigue una distribución de Bernoulli $y|x \sim B(p)$ si:

$$P(y = y'|x) = p^{y'}(1 - p)^{1-y'} \quad y' \in \{0, 1\}$$

Por tanto, la distribución de Bernoulli se define por un único número ya que sólo necesita predecir $P(y = 1|x)$. Para que este valor sea una probabilidad válida este valor debe caer en el intervalo $[0, 1]$. Para satisfacer esta restricción debemos diseñar correctamente las unidades de salida. Por ejemplo, podríamos obtener una probabilidad válida como:

$$P(y = 1|x) = \max\{0, \min\{1, w^T h + b\}\}$$

donde h es el conjunto de *features* de salida de la capa anterior y w y b el conjunto de pesos y el término de sesgo relativas a esta capa. El problema con esta solución es que sería problemático la aplicación del descenso de gradiente. En el caso en el que $w^T h + b$ se salga del intervalo unidad, el gradiente respecto a sus parámetros sería 0. Un gradiente con valor 0 es problemático pues el algoritmo pierde la noción de como mejorar sus parámetros, al no conocer la pendiente sobre la que descender. Para solucionar esto, debemos buscar una solución que proporcione un gradiente suficiente en el caso en el que el modelo ha hecho una predicción incorrecta. Para ello, se propone el uso de la función de activación **Sigmoid**.

Definición 2.5.1, Sigmoide. La función **Sigmoide** $\sigma : \mathbb{R} \rightarrow (0, 1)$ se define como:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

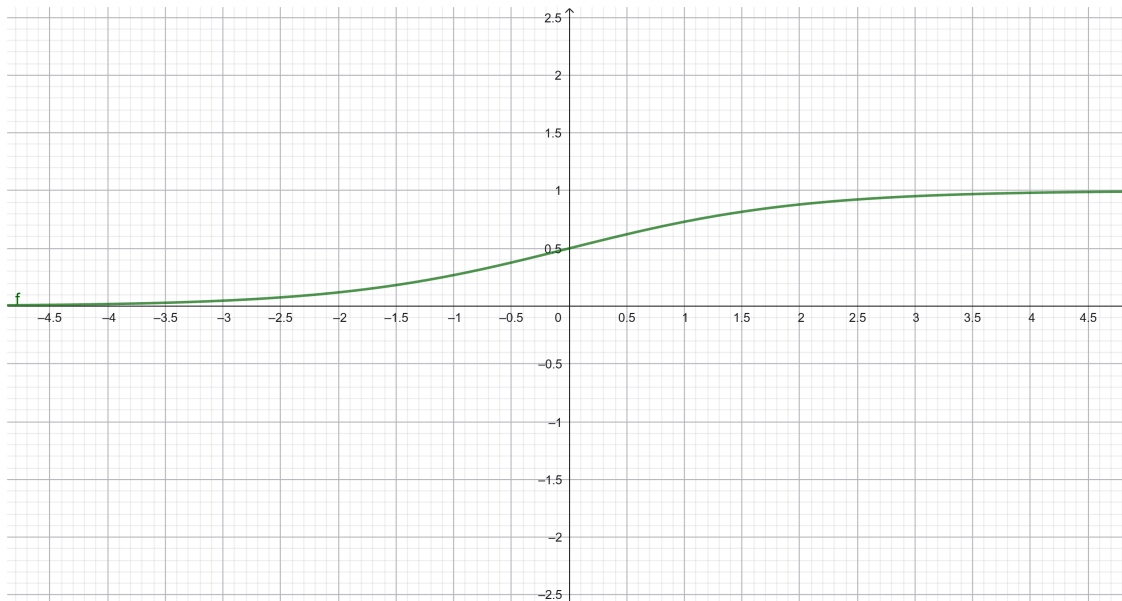


Figura 7: Función Sigmoide.

Como vemos en la imagen, la función se **satura** cuando x es muy negativo o muy positivo.

Para el problema anterior, podemos utilizar esta función es decir, la salida de la capa de salidas se puede expresar como:

$$y = \sigma(w^T h + b)$$

2.5.2.2. Unidades con función Softmax

Supongamos ahora que queremos representar la distribución de probabilidad sobre una variable discreta con n posibles valores. En este caso, podemos utilizar la función **Softmax** que es una extensión de la función Sigmoide al caso de n valores. Estas funciones suelen ser las utilizadas en las capas de salida en **problemas de clasificación**.

En este caso, necesitamos producir un vector $\hat{y}$ tal que $\hat{y}_i = P(y = i|x)$ con $i = 1, \dots, n$. Necesitamos no sólo que todos los elementos $\hat{y}_i$ sean valores entre 0 y 1 sino que la suma del vector sea 1 para que sea una distribución de probabilidad válida. Previo a la aplicación de la función de activación, obtenemos:

$$z = W^T h + b$$

donde h es el vector de *features* W la matriz de pesos y b el vector de bias de modo que $z_i = \log(P(y = i|x))$. La función Softmax se encarga de aplicar exponencial y después normalizar z para obtener y . Finalmente:

Definición 2.5.2, Softmax. La función **Softmax** $softmax : \mathbb{R}^n \rightarrow (0, 1)^n$ se define como:

$$softmax(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}} \quad i = 1, \dots, n$$

2.5.3. Capas ocultas

En esta sección, definiremos varios tipos de funciones de activación en unidades de capas ocultas. Recordemos que el principal objetivo de las funciones de activación es introducir elementos no lineales para aproximar cierta función. El diseño de unidades ocultas es un área de activa investigación y no existen demasiadas indicaciones sobre qué camino escoger.

Definición 2.5.3, ReLU. La función **ReLU** es una función $ReLU : \mathbb{R} \rightarrow [0, \infty)$ tal que:

$$ReLU(x) = \max\{0, x\}$$

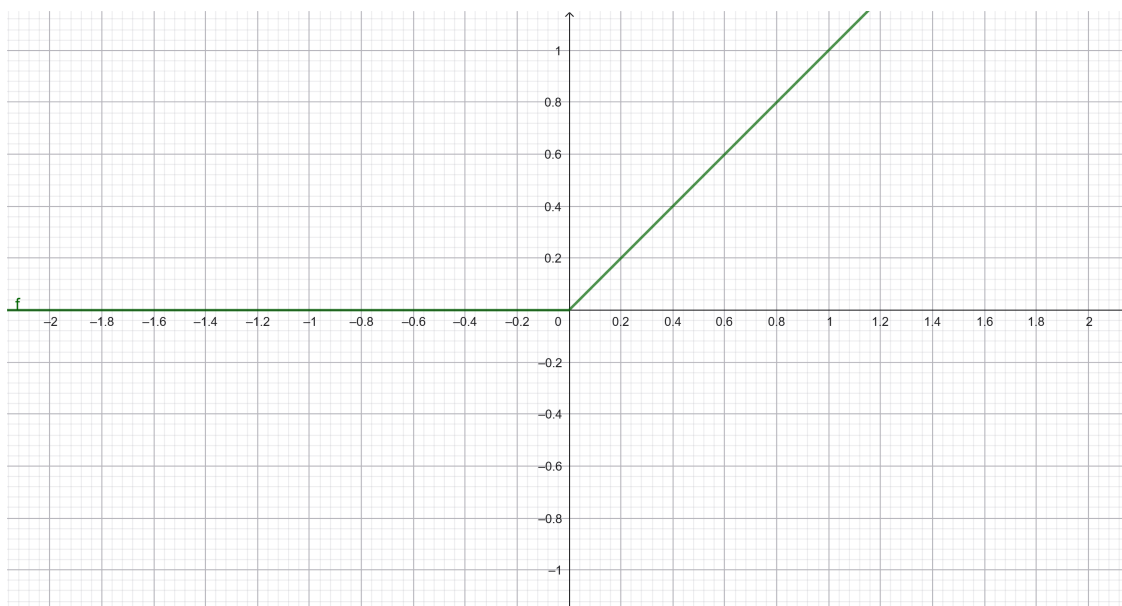


Figura 8: Función ReLU.

Las funciones de la familia **ReLU** son excelentes elecciones como función de activación. Como hemos comentado antes, es complicado determinar qué función de activación escoger en unidades ocultas. Generalmente, esto es un proceso de prueba y error en cada modelo, pero las funciones de la familia ReLU han demostrado buen rendimiento en la práctica.

Muchas de las funciones de activación realmente no son diferenciables. Por ejemplo, la función $ReLU(x) = \max(0, x)$ claramente no es derivable en $x = 0$. Puede parecer que elegir esta función es una mala decisión, ya que todo nuestro proceso de optimización está basado en gradientes. Sin embargo, en la práctica el descenso de gradiente funciona decentemente en esta tarea. Esto se da

porque típicamente, los modelos de redes neuronales no llegan al mínimo local, sino que reducen el valor de la función de coste de manera significativa.

En cuanto a variaciones de la función **ReLU**, destacamos **leaky ReLU** y **parametric ReLU** o **PReLU**. Estas variaciones añaden una pendiente α_i cuando $z_i < 0$ es decir $g(z_i, \alpha_i) = \text{ReLU}(z_i) + \alpha_i \min(0, z_i)$. En el caso de **leaky ReLU**, la pendiente es fija a un valor pequeño como $\alpha_i = 0,01$, mientras que en **PReLU** la pendiente α_i es un parámetro aprendido.

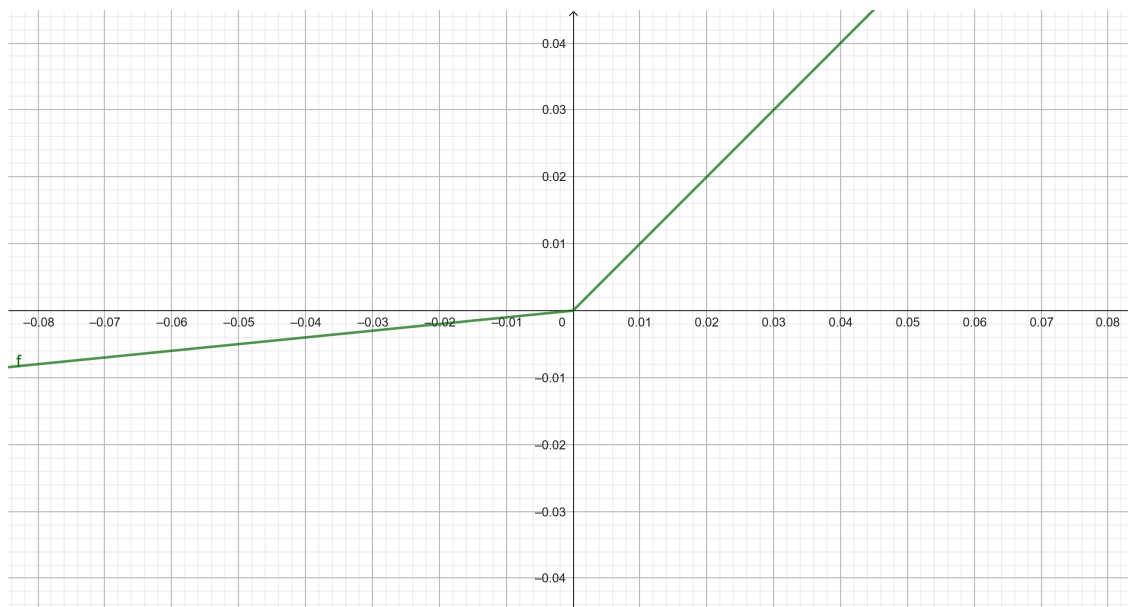


Figura 9: Función ReLU con pendiente. Se le ha asignado una pendiente $\alpha_i = 0,1$ para ilustrar el comportamiento de la función.

Las funciones de la familia ReLU, están basadas en el principio teórico de que los modelos son más fáciles de optimizar si se comportan parecido a las funciones lineales. Si comprobamos la gráfica de cualquiera de ellas, al fin y al cabo es una función lineal a trozos. En cambio, no son lineales. Por ejemplo, si ReLU fuera lineal $0 = \text{ReLU}(0) = \text{ReLU}(-1+1) = \text{ReLU}(-1) + \text{ReLU}(1) = 0+1 = 1$, lo cual no es correcto.

Otro tipo de funciones son la ya comentada **Sigmoide** o la **tangente hiperbólica** $\tanh : \mathbb{R} \rightarrow (-1, 1)$:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

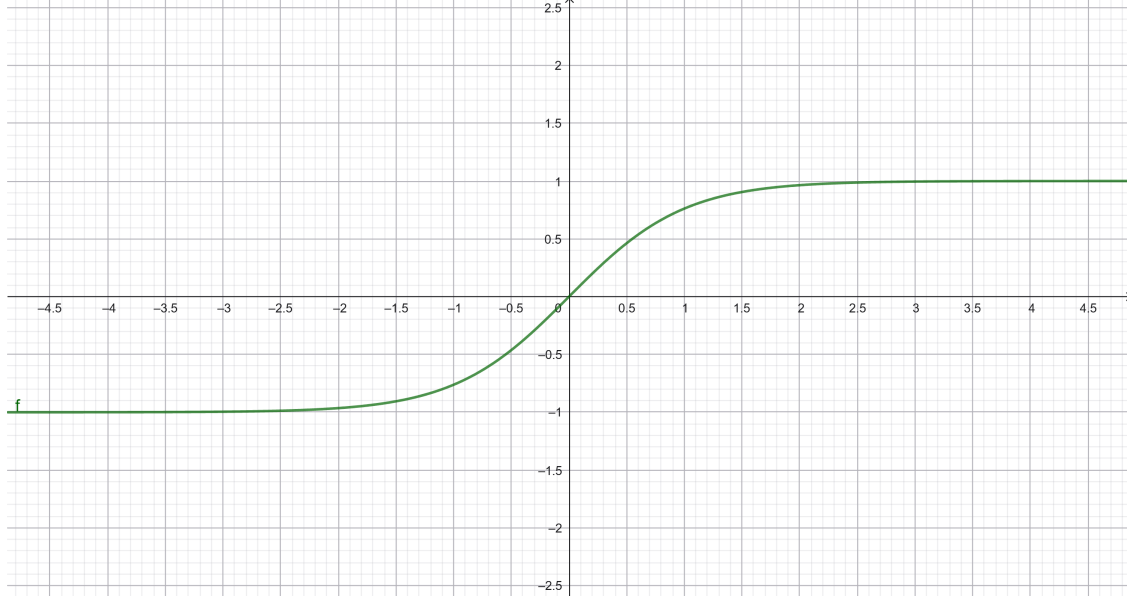


Figura 10: Función Tangente Hiperbólica.

Como podemos observar, tiene cierto parecido con la función Sigmoide en cuanto a forma, aunque esta última tiene como imagen el intervalo $(-1, 1)$. Del mismo modo que la función sigmoide, ambas se saturan en casi todo su dominio (sus rectas tangentes se aplanan). Por tanto, esto puede complicar la optimización basada en gradientes.

2.5.4. Back-Propagation

Como hemos comentado en la sección 2.5.1, las redes neuronales propagan información hacia delante, donde cada unidad envía información a cada una de las unidades de la siguiente capa. Eventualmente, obtendremos una salida y un valor de la función de coste. El algoritmo de **retro-propagación o back-propagation** [20] permite que la información viaje desde el coste pasando por toda la red para el cálculo de gradientes. Para aplicar este algoritmo, necesitamos definir un teorema muy importante.

Teorema 2.5.3, Regla de la cadena. Sean U, V abiertos no vacíos de $\mathbb{R}^n$ y $\mathbb{R}^m$ respectivamente, y sean $f : U \rightarrow V$ y $g : V \rightarrow \mathbb{R}^p$. Si f es diferenciable en un punto $a \in U$ y g es diferenciable en $b = f(a)$, entonces $g \circ f$ es diferenciable en a con:

$$D(g \circ f)(a) = Dg(b) \circ Df(a) = Dg(f(a)) \circ Df(a)$$

Por tanto, si $f \in D(U, V)$ (f es una función diferenciable de U en V) y $g \in D(V, \mathbb{R}^p)$, entonces $g \circ f \in D(U, \mathbb{R}^p)$. [21]

Consideremos un grafo computacional que describe el cálculo de un escalar $u^{(n)}$. Queremos calcular el gradiente para este valor con respecto a los n_i nodos de entrada $u^{(1)}, \dots, u^{(n_i)}$, es decir $\frac{\partial u^{(n)}}{\partial u^{(i)}}$ $\forall i \in \{1, \dots, n_i\}$. En el caso de aplicar **back-propagation** para el cálculo de gradientes, $u^{(n)}$ será el coste asociado a un ejemplo o un minibatch y $u^{(1)}, \dots, u^{(n_i)}$ los parámetros del modelo.

Vamos a suponer que los nodos del grafo están ordenados en el sentido de que se puede calcular las salidas de uno tras otro, comenzando en $u^{(n_i+1)}$ hasta $u^{(n)}$. Como definiremos próximamente, cada $u^{(i)}$ está relacionado con la operación $f^{(i)}$ y este nodo es calculado:

$$u^{(i)} = f(\mathbb{A}^{(i)})$$

donde $\mathbb{A}^{(i)}$ es el conjunto de los nodos padre de $u^{(i)}$.

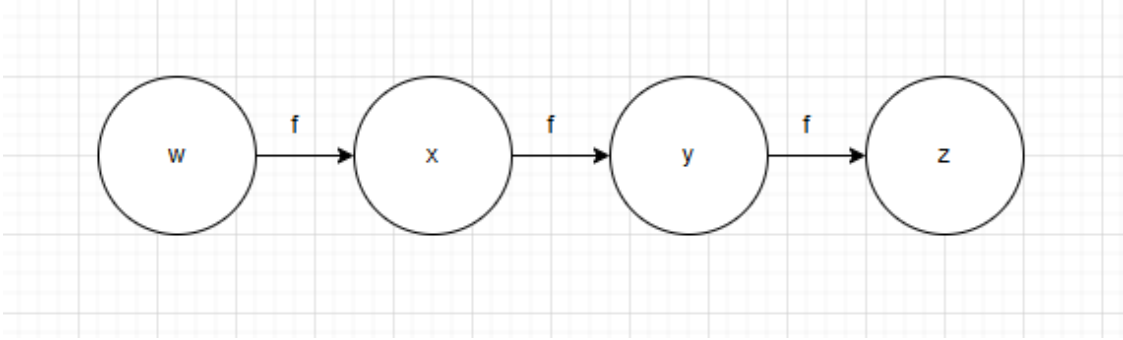


Figura 11: Ejemplo de backprop.

Supongamos que $w \in \mathbb{R}$ y que usamos la misma función $f : \mathbb{R} \rightarrow \mathbb{R}$ en cada unidad del grafo de modo que: $x = f(w), y = f(x), z = f(y)$. Para calcular $\frac{\partial z}{\partial w}$ aplicamos la regla de la cadena:

$$\begin{aligned} \frac{\partial z}{\partial w} &= \frac{\partial z}{\partial y} \frac{\partial y}{\partial x} \frac{\partial x}{\partial w} = \\ f'(y)f'(x)f'(w) &= f'(f(f(w)))f'(f(w))f'(w) \end{aligned}$$

Volviendo al caso general que hemos descrito antes del ejemplo, sabemos que el algoritmo que describe la propagación hacia delante se puede representar como un grafo G . Para realizar **back-propagation** podemos construir un grafo B dependiente de G que contiene un nodo por cada nodo de G . Los cálculos en el grafo B se realizan en orden inverso a la propagación hacia delante de G , donde cada nodo de B calcula $\frac{\partial u^{(n)}}{\partial u^{(i)}}$ asociado al nodo $u^{(i)}$ del grafo G . Esto se realiza mediante la regla de la cadena:

$$\frac{\partial u^{(n)}}{\partial u^{(j)}} = \sum_{i: j \in Pa(u^{(i)})} \frac{\partial u^{(n)}}{\partial u^{(i)}} \frac{\partial u^{(i)}}{\partial u^{(j)}}$$

donde $Pa(u^{(i)})$ es el conjunto de padres de $u^{(i)}$ en G . El algoritmo de **back-propagation** finalmente sigue los siguientes pasos:

- **1.** Inicializar $\text{grad_table}[u^{(n)}] \leftarrow 1$. Esta tabla contiene las derivadas calculadas donde $\text{grad_table}[u^{(i)}]$ contiene $\frac{\partial u^{(n)}}{\partial u^{(i)}}$.
- **2.** Iterando sobre $j = n - 1$ hasta 1 :

- $\text{grad_table}[u^{(j)}] \leftarrow \sum_{i:j \in Pa(u^{(i)})} \frac{\partial u^{(n)}}{\partial u^{(i)}} \frac{\partial u^{(i)}}{\partial u^{(j)}}$. Esta operación se hace reutilizando la tabla de gradientes de forma:

$$\text{grad_table}[u^{(j)}] \leftarrow \sum_{i:j \in Pa(u^{(i)})} \text{grad_table}[u^{(i)}] \frac{\partial u^{(i)}}{\partial u^{(j)}}$$

- **3.** Se devuelve $\{\text{grad_table}[u^{(i)}] : i = 1, \dots, n_i\}$

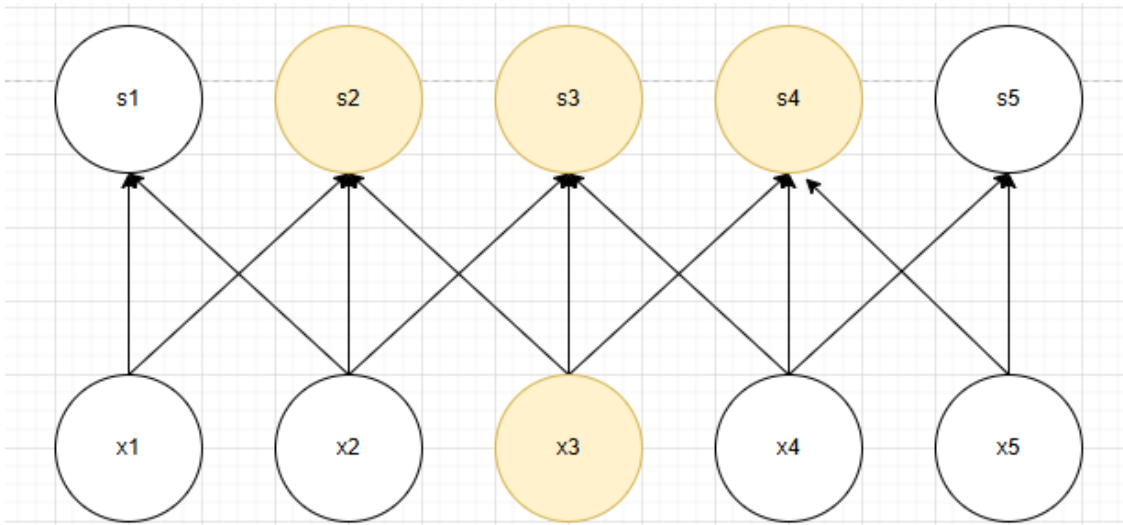
2.5.5. Redes neuronales convolucionales

Las **redes neuronales convolucionales** (**convolutional neural networks** o simplemente **CNN**), son redes neuronales especializadas en el procesamiento de datos con una estructura de rejilla. Algunos ejemplos son datos en forma de series temporales, interpretables como datos rejillas 1D o imágenes, en forma de rejillas 2D. El nombre de redes neuronales convolucionales proviene de la operación matemática **convolución**, descrita en la sección 2.1.4.

El uso de convoluciones se sustenta en tres principios que ayudan a mejorar los sistemas de machine learning: **interacciones dispersas**, **compartir parámetros** y **representaciones equivariantes**.

- **1. Interacciones dispersas:** Las capas de redes neuronales tradicionales utilizan multiplicación matricial por una matriz de parámetros, en la que existe un parámetro distinto que describe la interacción entre cada unidad de entrada y cada unidad de salida, por lo que cada unidad de salida interactúa con una de entrada. En redes convolucionales tenemos conexiones dispersas. Esto es posible por el uso de kernels más pequeños que las entradas. Por ejemplo, una imagen puede contar con millones de píxeles, pero podemos extraer *features* fundamentales usando kernels de decenas o cientos de píxeles.

Figura 12: Comparación capas convolucionales con capas multiperceptrón clásicas. Parte 1.



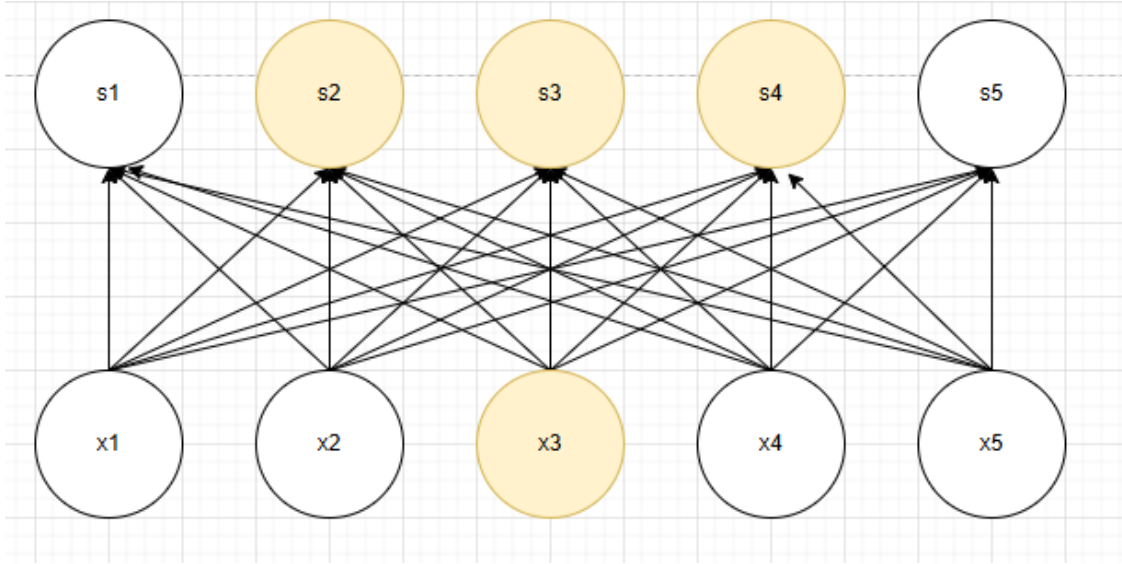


Figura 13: Comparación capas convolucionales con capas multiperceptrón clásicas. Parte 2.

Como podemos ver en la primera imagen (**capa convolucional**), tenemos la unidad x_3 y las unidades señalizadas que tienen una arista que parte de x_3 . Cuando s está formado por un kernel de amplitud 3, solo 3 salidas son afectadas por x , como vemos en el caso de x_3 . En cambio, cuando usamos multiplicación de matrices, la interacción no es dispersa, sino total, donde todas las unidades de salida están afectadas por todas las unidades de entrada. Por esta razón, las mejoras de rendimiento son sustanciales. Si tenemos m entradas y n salidas, la matriz de multiplicaciones contiene $m \times n$ parámetros y los algoritmos en la práctica tienen un tiempo de ejecución por cada ejemplo de $O(m \times n)$. Si limitamos el número de conexiones a k , entonces el enfoque de conectividad dispersa requiere únicamente $k \times n$ parámetros y $O(k \times n)$ de tiempo de ejecución.

- **2. Compartir parámetros:** Este principio se basa en el uso del mismo parámetro para más de una función de una capa. En el enfoque tradicional, cada elemento de la matriz de pesos es usado únicamente una vez en el cálculo de la capa de salidas. Dicho elemento se multiplica una vez y nunca se vuelve a hacer uso de él. En redes convolucionales, cada elemento del kernel se usa en cada posición de la entrada. En lugar de aprender un conjunto distinto de parámetros para cada posición de la entrada, se aprende un único kernel que se comparte y se aplica en todas las posiciones. Si nos fijamos de nuevo en la imagen 12, cada conexión estrictamente vertical (es decir, el arco que une x_i con s_i) representa **el mismo valor central de un kernel de tamaño 3**. En cambio, en la segunda imagen, los arcos que unen x_i con s_i no representan el mismo valor para todos los valores de i .
- **3. Representación equivariante:** De forma más precisa, podemos decir que las capas convolucionales son equivariantes con respecto a traslaciones. La propiedad de equivarianza significa que si la entrada de una función cambia, la salida cambia de forma similar. Para ser más exactos, decimos que una función f es equivariante con respecto a g si $f(g(x)) = g(f(x))$. En el caso de convoluciones, si realizamos una traslación de la entrada, la convolución será equivariante a esa traslación, es decir, si a una imagen por ejemplo aplicamos una convolución y después la desplazamos, el resultado será el mismo a desplazarla primero y después aplicar convolución.

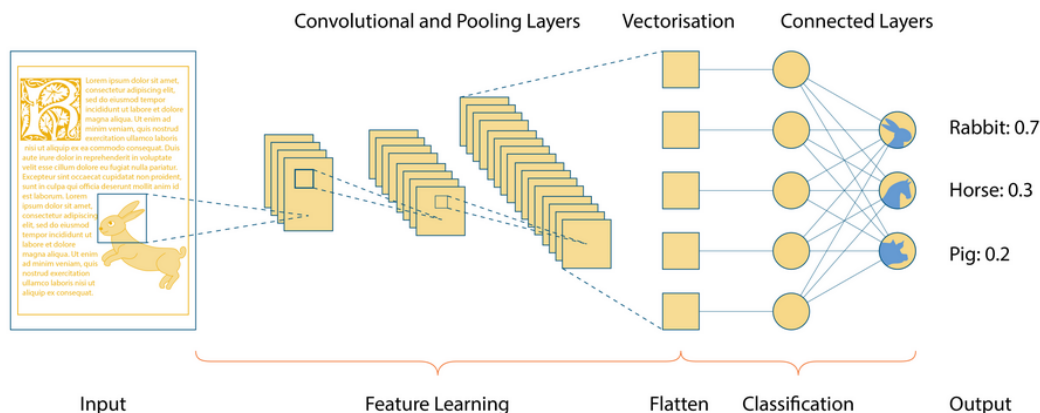


Figura 14: Ejemplo de CNN, https://commons.wikimedia.org/wiki/File:Convolutional_Neural_Network.png

En la imagen podemos apreciar un ejemplo de una **CNN**. Observando la salida, podemos ver que es una tarea de clasificación de 3 clases, donde la imagen de entrada contiene un conejo. Está compuesta por una serie de capas convolucionales y de pooling (detallaremos este término en la próxima sección). Estas capas se encargan de la **extracción de *features***, tarea que en problemas de machine learning clásico era responsabilidad del desarrollador. Como podemos ver, en cada capa se producen una cantidad de datos en forma matricial. A estos datos se le conocen como **mapas de activación o *feature maps***. La cantidad viene determinada por el **número de filtros** de la capa, mientras que la dimensión y los valores de los mapas de activación dependen del tamaño del kernel.

Posterior a esta extracción de *features*, se aplanan estos mapas de activación, es decir, se convierten a un vector. Este vector será procesado por una combinación de una o más capas densas, es decir, donde todas las unidades realizan propagación a todas las unidades de la capa siguiente. Finalmente, tenemos la clasificación del ejemplo.

2.5.5.1. Pooling

Una capa convolucional está compuesta típicamente de 3 partes:

- 1. La propia capa que realiza operaciones de convolución sobre entradas.
- 2. Una función de activación no lineal del mismo modo que en capas fully-connected.
- 3. Función de **pooling**.

Definición 2.5.4, Pooling. Una función de **Pooling** es una función p que se aplica a una vecindad V de la entrada. Tiene como objetivo hacer la representación invariante ante pequeñas traslaciones de la entrada.

Existen varios tipos de Pooling, donde destacamos los siguientes:

- **1. Max Pooling:** Toma el máximo de la vecindad, $p(V) = \max_{x \in V}(x)$.
- **2. Average Pooling:** Toma la media de la vecindad, $p(V) = \frac{1}{|V|} \sum_{x \in V}(x)$.
- **3. Pooling L^2 :** Calcula la norma L^2 , $p(V) = \sqrt{\sum_{x \in V}(x^2)}$.
- **4. Pooling basado en distancia:** Se trata de una media ponderada basada en la distancia al píxel central de la vecindad.

Consideremos el siguiente ejemplo:

$$M = \begin{pmatrix} 12 & 20 & 30 & 0 \\ 8 & 12 & 2 & 0 \\ 34 & 70 & 37 & 4 \\ 112 & 100 & 25 & 12 \end{pmatrix}$$

Consideremos ahora que aplicamos **max Pooling y average Pooling** con filtros de tamaño 2x2 y un **stride** (el tamaño de paso de aplicación del kernel):

$$\text{max Pooling}\left(\begin{pmatrix} 12 & 20 \\ 8 & 12 \end{pmatrix}\right) = \max(12, 20, 8, 12) = 20$$

Como el stride es 2, desplazamos 2 veces hacia la derecha el filtro, por tanto:

$$\text{max Pooling}\left(\begin{pmatrix} 30 & 0 \\ 2 & 0 \end{pmatrix}\right) = \max(30, 0, 2, 0) = 30$$

luego:

$$\text{max Pooling}\left(\begin{pmatrix} 12 & 20 & 30 & 0 \\ 8 & 12 & 2 & 0 \\ 34 & 70 & 37 & 4 \\ 112 & 100 & 25 & 12 \end{pmatrix}\right) = \begin{pmatrix} 20 & 30 \\ 112 & 37 \end{pmatrix}$$

De modo similar, el average Pooling tendría como resultado:

$$\text{average Pooling}\left(\begin{pmatrix} 12 & 20 & 30 & 0 \\ 8 & 12 & 2 & 0 \\ 34 & 70 & 37 & 4 \\ 112 & 100 & 25 & 12 \end{pmatrix}\right) = \begin{pmatrix} 13 & 8 \\ 79 & 20 \end{pmatrix}$$

2.5.6. Transformers

Este apartado está basado en el innovador artículo **Attention is all you need** [6]. En él se propone una nueva arquitectura que se separa de las redes neuronales convolucionales y de redes neuronales recurrentes. Esta arquitectura es el **Transformer**, que está basada en mecanismos de **atención**, abandonando recurrencia y convoluciones por completo. Vamos a detallar los mecanismos de atención.

2.5.6.1. Scaled Dot-Product Attention

Para calcular la atención, suponemos que tenemos entradas que se dividen en **consultas** Q y **claves** K , ambas de dimensión d_k y **valores** V de dimensión d_v . Se calcula el producto escalar de la matriz de consultas y de claves, se escala dividiendo por $\sqrt{d_k}$ y se aplica la función softmax a este término para obtener los pesos de los valores. Finalmente, calculamos la atención como:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

2.5.6.2. Multi-Head Attention

En lugar de aplicar una única función de atención con claves, valores y consultas de dimensión d_{model} , durante el estudio se aprendió que era beneficioso proyectar linealmente las consultas, valores y claves h veces con diferentes proyecciones lineales de dimensiones d_k , d_k y d_v respectivamente. En cada una de las proyecciones de las consultas, valores y claves, se aplica la función de atención en paralelo, generando salidas de dimensión d_v . Estas salidas se concatenan y se vuelven a proyectar, dando lugar a los valores finales. Podemos entonces expresar la función **Multi-Head Attention** como:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

$$\text{donde } \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

donde las proyecciones de matrices de parámetros $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ y $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$

2.6. Out Of Distribution

Uno de los objetivos principales del desarrollo experimental de este trabajo es la evaluación **Out-of-Distribution (OOD)**. La detección de ejemplos OOD es una tarea emergente dentro del ámbito del machine learning. En esta tarea, suponemos que no sólo tenemos un conjunto de datos como

es común en problemas de machine learning, sino que disponemos de un conjunto de datos **In-Distribution (ID)** con los que entrenaremos el modelo y el objetivo es que dicho modelo sea capaz de separar estas muestras de muestras OOD. En modelos de machine learning es crucial esta tarea pues normalmente, cualquier muestra pertenezca o no a la distribución de los datos a evaluar, obtiene una predicción por parte del modelo.

En ciertas situaciones, la ausencia de métodos de detección OOD puede llevar al modelo a hacer predicciones incorrectas y por tanto potencialmente peligrosas. Por esto, en los modelos es importante incorporar detección de ejemplos OOD para que el modelo sea robusto y fiable.

Podemos distinguir dos enfoques de aplicación de métodos OOD:

- **Basados en el entrenamiento.**
- **Métodos post-hoc:** métodos aplicados una vez finalizada la etapa de entrenamiento.

[22]

2.7. IA explicable: algoritmos LRP y CRP

Hasta el momento, hemos descrito las salidas que proporcionan los modelos de Inteligencia Artificial y cómo se obtienen. Uno de los conceptos que ha obtenido mayor relevancia recientemente es el estudio de cómo la Inteligencia Artificial toma las decisiones que en última instancia, dan lugar a las salidas. A este concepto se le conoce como **IA explicable, eXplainable AI o simplemente XAI**. Según [23], “Para un determinado público, una **Inteligencia Artificial explicable** es aquella que proporciona detalles y razones para hacer su funcionamiento claro y sencillo de entender”.

En este trabajo utilizaremos el algoritmo de IA explicable **Concept Relevance Propagation (CRP)**, basado en un algoritmo previo llamado Layer-wise Relevance Propagation. Supongamos un predictor con L layers o capas:

$$f(x) = f_L \circ \dots \circ f_1(x)$$

LRP sigue el flujo de los mapas de activaciones calculados por el modelo desde la capa final f_L hacia la inicial f_1 . Sea f_i una función, sus preactivaciones z_{ij} representan la contribución de la entrada i en la salida j . La suma de estas preactivaciones se denota z_j , es decir:

$$\begin{aligned} z_{ij} &= a_i w_{ij} \\ z_j &= \sum_i z_{ij} \\ a_j &= \sigma(z_j) \end{aligned}$$

donde a_i son las entradas, w_{ij} los pesos de la capa y σ es la función de activación que introduce no linealidad. El método LRP distribuye valores de relevancia R_j correspondiente a los valores de

a_j y recibidos de capas superiores hacia capas inferiores de forma proporcional a las contribuciones relativas de z_{ij} a z_j , es decir:

$$R_{i \leftarrow j} = \frac{z_{ij}}{z_j} R_j$$

La relevancia de unidades inferiores se obtiene, sin pérdida de información, sumando los mensajes de relevancia entrantes:

$$R_i = \sum_j R_{i \leftarrow j}$$

Este proceso garantiza la conservación de la relevancia entre una unidad j y sus entradas i .

CRP extiende el método de LRP introduciendo propagación de relevancia condicional bajo una serie de condiciones θ . Una condición se puede interpretar como un identificador de un elemento de una red neuronal, como por ejemplo las unidades de una capa. De este modo, CRP calcula la relevancia asociada a cada una de estas condiciones mediante propagación hacia atrás, permitiendo calcular la contribución de cada concepto a la predicción realizada por el modelo.

[24]

3. Materiales y métodos

3.1. Descripción del problema clínico

Como comentamos en la introducción (sección 1), la IA ha permitido numerosos avances dentro del campo de la medicina. En particular, el cáncer de próstata es un problema de gran importancia por el número elevado de afectados y por los efectos que conlleva.

En esta sección describiremos un problema de **clasificación** sobre un conjunto de datos de biopsias de cáncer de próstata con diferentes modelos de **deep learning**. Como objetivo principal, se propondrán métodos de detección **OOD** (sección 2.6) basados en el uso de **XAI** (sección 2.7).

Comenzaremos extendiendo las nociones sobre el cáncer de próstata que hemos dado hasta ahora, con el fin de obtener un mejor entendimiento del problema e introducir el conjunto de datos.

3.1.1. Conceptos básicos sobre el cáncer de próstata

El cáncer de próstata es un tumor maligno con origen en las células de la próstata, que es una glándula del aparato reproductor masculino que tiene como función la elaboración de una parte del semen.

En España, el cáncer de próstata es el tumor más común en hombres, representando la tercera causa de muerte por cáncer. Se estima que 1 de cada 8 hombres será diagnosticado con cáncer de

próstata en su vida. La mayoría de casos se dan en pacientes en edad avanzada, donde se estima que el 90 % de pacientes tienen más de 65 años y la edad media de diagnóstico es de 75 años.

El diagnóstico se suele dar en una fase temprana de la enfermedad. Los métodos más comunes de diagnóstico son:

- **Determinación de PSA en sangre:** El PSA es una sustancia producida en la próstata donde un valor alto de esta sustancia en sangre puede indicar mayor probabilidad de tumor.
- **Tacto rectal:** Se basa en la palpación de la próstata en busca de nódulos sospechosos.
- **Biopsia prostática:** En ambos métodos anteriores hemos hablado de probabilidad y sospecha respectivamente, por lo que se entienden que no son métodos totalmente precisos. En caso de cierta sospecha se procede con una biopsia prostática. Una **biopsia** es la extracción de tejido del cuerpo con fin de ser analizado bajo microscopio. En el caso de la biopsia prostática, es el único método capaz de confirmar el diagnóstico de cáncer de próstata.

[9]

En este trabajo se hará uso de imágenes de biopsias de tejido prostático, con el fin de realizar una correcta clasificación en función de los tipos de tumores, que presentaremos más adelante.

3.2. El conjunto de datos

Como hemos comentado en las secciones anteriores, el conjunto de datos está formado por biopsias de tejido prostático en forma de imagen. En el dataset [25], contaremos con **14 clases** para nuestro problema de **clasificación**, donde asignaremos la etiqueta numérica a cada clase de la siguiente forma:

- **0. Adenocarcinoma ductal** → 18.917 ejemplos.
- **1. Carcinoma intraductal** → 1.083 ejemplos.
- **2. Estroma** → 22.219 ejemplos.
- **3. Glándulas no neoplásicas** → 32.304 ejemplos.
- **4. Gleason 3** → 2.275 ejemplos.
- **5. Gleason 4 cribiforme** → 12.723 ejemplos.
- **6. Gleason 4 fusionadas** → 6.938 ejemplos.
- **7. Gleason 4 glomeruloide** → 218 ejemplos.
- **8. Gleason 4 pobremente formadas** → 1.136 ejemplos.
- **9. Gleason 5 comedonecrosis** → 499 ejemplos.

- 10. Gleason 5 sin glándulas → 10.908 ejemplos.
- 11. Inflamatorio → 714 ejemplos.
- 12. No cáncer → 1.081 ejemplos.
- 13. PIN de alto grado → 2.476 ejemplos.

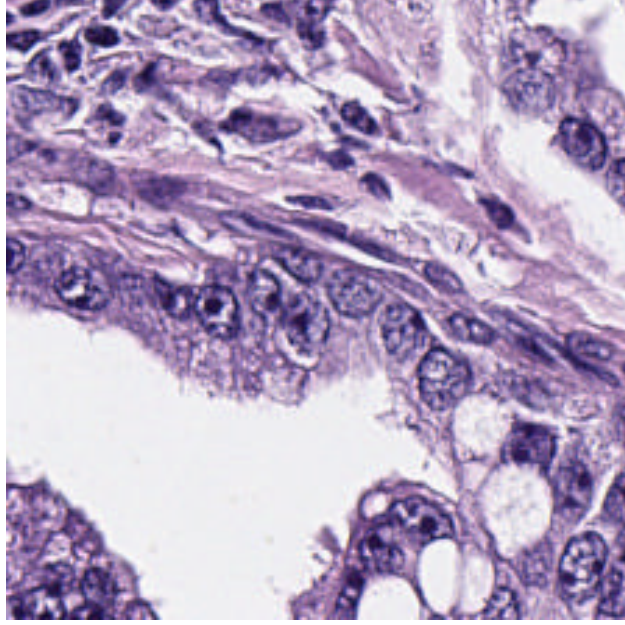


Figura 15: Ejemplo de biopsia de adenocarcinoma ductal

Estas imágenes tienen dimensión 224x224 con color y por tanto cuentan con 3 canales (RGB). Podemos definir entonces nuestro problema de clasificación como estimar un predictor $f : \mathbb{R}^{224 \times 224 \times 3} \rightarrow \{0, 1, \dots, 13\}$, mediante una función f_θ dependiente de un conjunto de parámetros θ con el objetivo de:

$$f_\theta(x_i) \approx y_i \quad (x_i, y_i) \in D = \{(x_i, y_i) : x_i \in \mathbb{R}^{224 \times 224 \times 3}, y_i \in \{0, 1, \dots, 13\}\}$$

Donde D es nuestro dataset, dividido en **113491 ejemplos de entrenamiento** y **12618 ejemplos de validación**.

3.3. OpenOOD

La detección de ejemplos OOD (sección 2.6) es fundamental en problemas de machine learning para el desarrollo de modelos robustos y seguros. Esta rama del machine learning ha sido sujeto de numerosos estudios. Sin embargo, este campo carece de un *benchmark* unificado, rigurosamente definido y exhaustivo, lo que lleva a comparaciones poco fieles y resultados no concluyentes. En este

contexto surge **OpenOOD**, que implementa más de 30 métodos desarrollados en ámbitos relevantes y propone un benchmark completo para la detección OOD. Haremos uso de este framework para llevar a cabo nuestros experimentos.

OpenOOD proporciona todo tipo de elementos para esta tarea, desde el conjunto de componentes para realizar un entrenamiento como pueden ser modelos, preprocesadores, funciones de pérdida..., pasando por datasets sobre los que entrenar nuestros modelos o evaluar detección OOD y terminando con posprocesadores que se encargan de asignar a cada ejemplo una puntuación sobre la que se decide si el ejemplo ha de pertenecer a la distribución OOD.

[26], [27]

3.4. Modelos seleccionados

En esta sección describiremos los modelos sobre los que se ha entrenado el conjunto de datos, **ResNet-50** que se trata de una red neuronal convolucional (sección 2.5.5) y **DeiT** que es un *Vision Transformer* (sección 2.5.6). Antes de detallar estos modelos, comentaremos un concepto clave.

3.4.1. Transfer Learning

El **Transfer learning** es el concepto que se basa en que lo que ha sido aprendido para un contexto se reutilice para mejorar el rendimiento en otro contexto. Esto es una práctica común en deep learning, donde los modelos generalmente son entrenados en datasets con un gran número de clases. Por tanto, los parámetros aprendidos por estos modelos en sus entrenamientos se pueden reutilizar como base para el modelo de nuestra tarea en particular y no partir de parámetros aleatorios, lo que supondría un coste en tiempo llegar a un punto parecido al que nos ofrecen los pesos de modelos preentrenados. En este trabajo, se usará el principio de **transfer learning** sobre los modelos ResNet-50 y DeiT. Para adaptarlos a nuestro problema, será necesario realizar pequeñas modificaciones en el modelo, como por ejemplo cambiar el número de salidas al **número de clases de nuestro dataset** y volver a entrenar para adaptar los pesos preentrenados a nuestro modelo. A esto se le conoce como **fine tuning**.

[1]

3.4.2. ResNet-50

ResNet-50 es un modelo de red neuronal convolucional basado en el **aprendizaje residual** introducido por [4]. Para describir mejor el concepto de aprendizaje residual, presentamos la siguiente imagen:

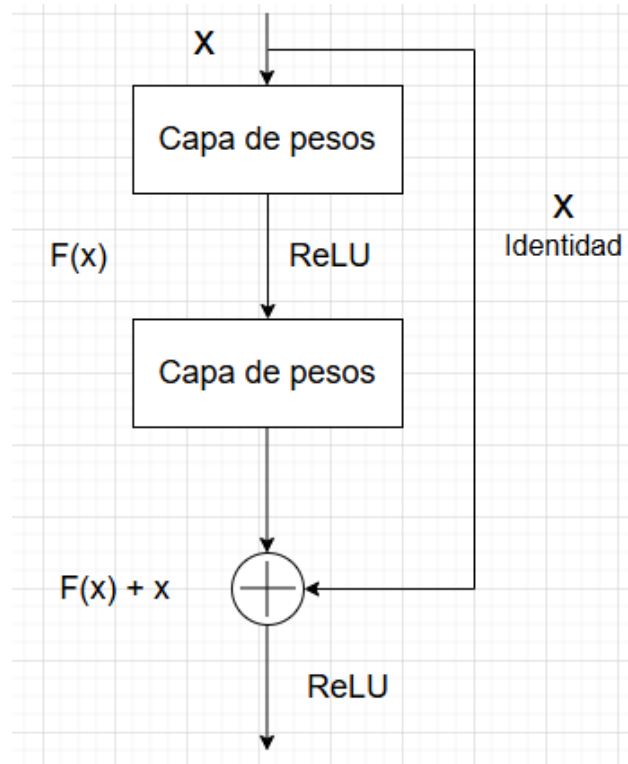


Figura 16: Ejemplo de bloque residual. Esta imagen está basada en los ejemplos de [4].

En lugar de pretender que cada conjunto de capas aprendan una función objetivo, podemos reformular el problema a uno equivalente intentando aprender una **función residual**. Supongamos que la función objetivo es $H(x)$. Ahora en lugar de aprender esta función, indicamos a las capas que la función objetivo es $F(x) = H(x) - x$, pudiendo obtener $H(x)$ como $F(x) + x$. A la función F se le conoce como **residuo o función residual** y representa la corrección que hay que aplicarle a la entrada para obtener la salida deseada. La premisa de este elemento es que es más fácil aprender la corrección que aprender la función completa. En un caso extremo donde la función objetivo fuera la identidad, sería mucho más sencillo aprender el residuo $F(x) = 0$ que intentar aproximar la función identidad a través de las capas internas.

Observando la figura anterior, nos damos cuenta de que existe una conexión que conecta antes y después de las capas de pesos. Estas conexiones se les conoce como **skip connections o shortcut connections**. En este caso, estas conexiones simplemente aplican la función identidad a la entrada, es decir, devuelven la propia entrada. Finalmente, para obtener $H(x)$, se suma la función residuo $F(x)$ aprendida por el bloque de capas de pesos a la entrada x proporcionada por la conexión. Como apunte, al sumar x al final de cada bloque residual, parece indicar que la función residuo debe producir salidas con las mismas dimensiones que x pero en la práctica tenemos que las capas convolucionales pueden y suelen reducir dimensionalidad. Para esto, se aplica a x una **proyección lineal** W_s (normalmente una convolución 1×1).

Una vez definido qué son los residuos y los bloques residuales, vamos a ver la arquitectura de ResNet-50. Antes de ello, conviene ver qué bloques residuales en particular utiliza este modelo. Estos bloques se les conoce como **bottleneck o cuello de botella** y tienen la siguiente estructura:

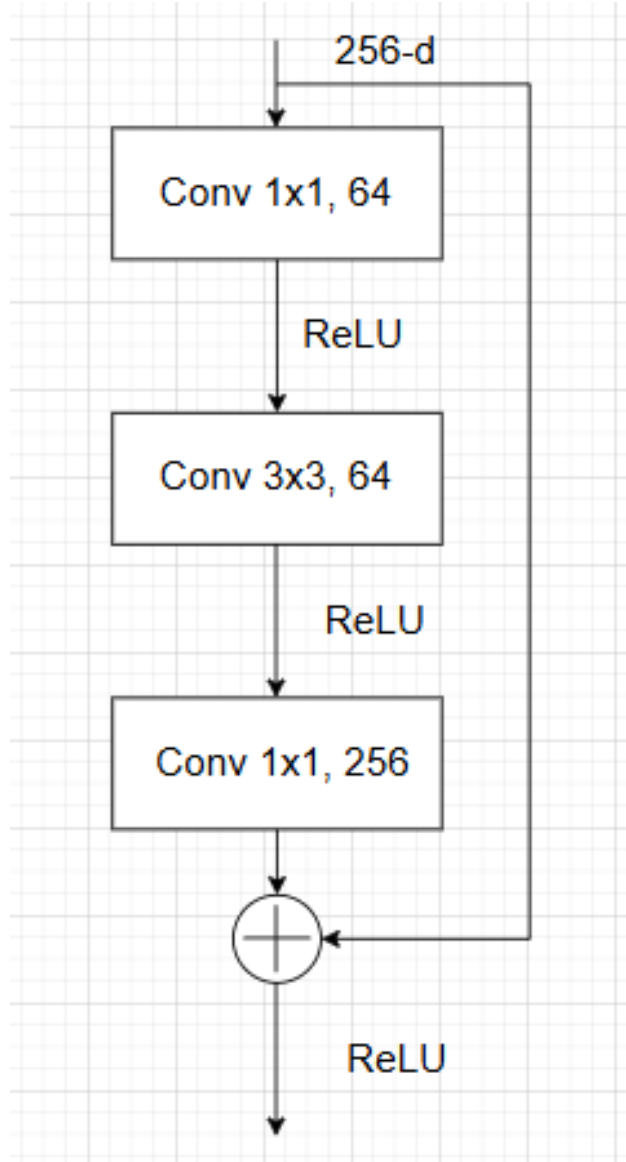


Figura 17: Ejemplo de bloque bottleneck para ResNet-50. Esta imagen está basada en los ejemplos de [4].

Como vemos, está formado por 3 capas convolucionales de 1x1, 3x3 y 1x1 respectivamente, introduciendo no linealidad con funciones ReLU y una skip connection que conecta el antes y después de estas tres capas convolucionales. Los canales de salida en este caso son de ejemplo y dichos valores variarán en función del bloque. Para estructurar esto de una forma más clara, notaremos como:

$$\text{Bottleneck}(n_1, n_2, n_3)$$

Al bloque bottleneck que tiene en las subcapas convolucionales filtros de salida de tamaño n_1, n_2 y n_3 respectivamente. Una vez definido estos bloques, la estructura de ResNet-50 es:

- **Bloque conv1:** Capa convolucional con 64 filtros de tamaño 7x7 y stride 2 (tamaño

de paso de aplicación de kernel).

- **Bloque conv2_x**: max pooling (sección 2.5.5.1) 3x3 con stride 2 seguidos de **3 bloques** Bottleneck(64, 64, 256).
- **Bloque conv3_x**: **4 bloques** Bottleneck(128, 128, 512).
- **Bloque conv4_x**: **6 bloques** Bottleneck(256, 256, 1024).
- **Bloque conv5_x**: **3 bloques** Bottleneck(512, 512, 2048).
- **Average pooling** para aplanar el mapa de *features* a un vector de dimensión 2048, **capa densa** de 14 unidades (número de clases) y **softmax**.

[4]

3.4.3. DeiT

Las redes neuronales convolucionales fueron tradicionalmente la herramienta utilizada en tareas de aprendizaje sobre imágenes. Desde que surgieron los mecanismos de atención ([6]), ha incrementado el uso de modelos basados en atención para este tipo de tareas, cuando originalmente fueron propuestos para tareas de procesamiento de lenguaje natural. [28]

En este contexto surge un modelo clave, el **Vision Transformer (ViT)** introducido por [5]. El *Transformer* original recibe un vector que representa una secuencia de token embeddings. Para las imágenes, que son datos 2D, teniendo en cuenta que la imagen se representa como $x \in \mathbb{R}^{H \times W \times C}$ dividimos esa imagen en una secuencia de **parches (o patches en inglés)** 2D de dimensión $x_p \in \mathbb{R}^{N \times (P^2 C)}$ donde (H, W) es la resolución de la imagen original, C es el número de canales, (P, P) es la resolución de cada patch y $N = \frac{HW}{P^2}$ es el número de patches. El *Transformer* usa vectores de tamaño D (dependiente del modelo usado) en todas sus capas por lo que debemos proyectar nuestra secuencia de patches a tamaño D mediante **proyecciones lineales entrenables**. Al vector resultante se le conoce como **patch embeddings**.

Al vector de embeddings de patches se le añade un nuevo componente aprendible conocido como **token de clase**. Este token una vez pasado por la estructura de **encoder** que detallaremos más adelante, contiene la representación de la imagen. Además, se añade una serie de positional embeddings a los patch embeddings para tener información posicional sobre cada patch. Estos embeddings también son entrenables. Para construir este embedding final se realiza la siguiente operación:

$$z_0 = [x_{\text{class}}; x_p^1 E; x_p^2 E; \dots; x_p^N E] + E_{\text{pos}} \quad E \in \mathbb{R}^{(P^2 C) \times D}, E_{\text{pos}} \in \mathbb{R}^{(N+1) \times D}$$

es decir, el embedding final se construye sumando el embedding posicional entrenable E_{pos} a los patch embeddings donde x_{class} es el token de clase, x_p^i es el patch i -ésimo de la imagen p y E es la proyección lineal entrenable. El **embedding** z_0 representa el embedding asociado a una imagen **antes** de pasar por mecanismos de atención y capas MLP. Continuamos introduciendo la estructura de **encoder**:

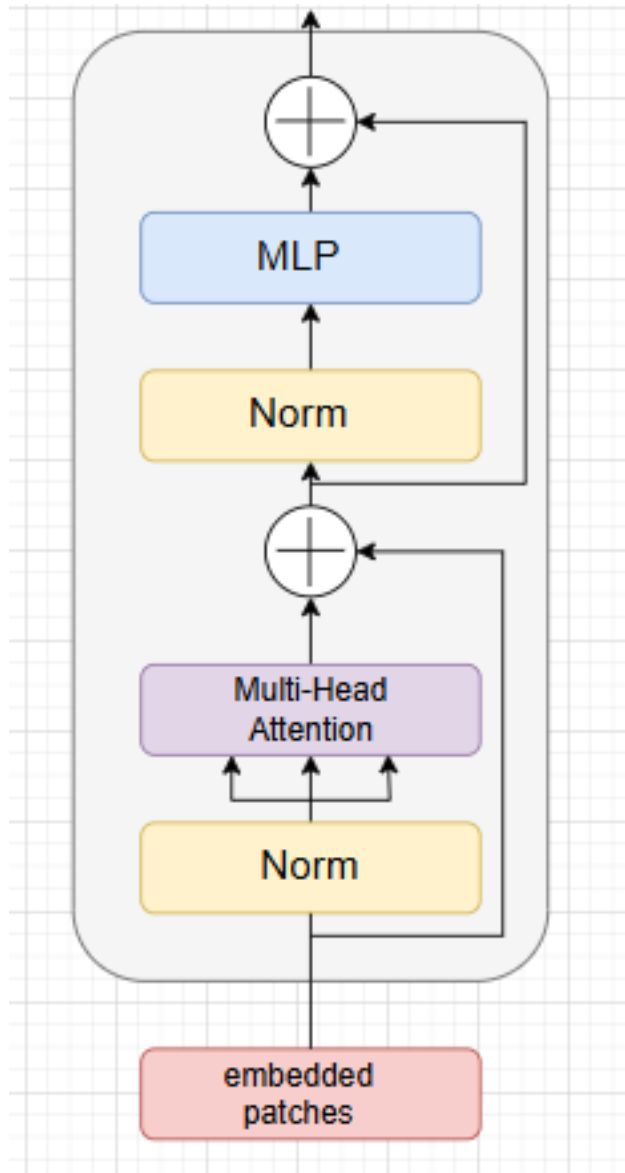


Figura 18: Ejemplo de bloque de encoder (fondo gris). Esta imagen está basada en los ejemplos de [5] y [6].

Un **encoder** en el contexto de deep learning es un conjunto de capas dedicadas a reducir dimensionalidad para obtener unos mapas de activación que concentren la información más importante del dato de entrada. Como podemos ver, un bloque de **encoder** en este contexto está formado por dos bloques residuales. El primero contiene una capa de normalización de los datos seguida de una capa de **Multi-Head Attention** (sección 2.5.6). El siguiente contiene otra capa de normalización y un perceptrón multicapa **MLP**. Dependiendo del modelo, este bloque se repite **L veces**. El MLP contiene 2 capas con función de activación GeLU, de la familia ReLU. El proceso de cálculos que sigue el encoder es el siguiente. Notando z_i al embedding tras pasar por bloque i -ésimo:

$$\begin{aligned}
z_0 &= [x_{\text{class}}; x_p^1 E; x_p^2 E; \dots; x_p^N E] + E_{\text{pos}} \quad E \in \mathbb{R}^{(P^2 C) \times D}, E_{\text{pos}} \in \mathbb{R}^{(N+1) \times D} \\
z_i^* &= \text{MSA}(\text{LN}(z_{i-1})) + z_{i-1} \quad i = 1, \dots, L \\
z_i &= \text{MLP}(\text{LN}(z_i^*)) + z_i^* \quad i = 1, \dots, L \\
y &= \text{LN}(z_L^0)
\end{aligned}$$

Donde **LN** es una **layer normalization**, **MSA** es la capa de **Multi-Head Self Attention** y z_L^0 es el token de clase tras pasar por los L bloques del encoder. Por tanto y , que es la salida total del encoder, es el token de clase normalizado tras pasar por el encoder. Este token contiene la **representación de la imagen** como habíamos introducido anteriormente. Finalmente, este token pasa por un MLP para proceder con la clasificación de la imagen asociada. La estructura de un **Vision Transformer (ViT)** se puede representar como:

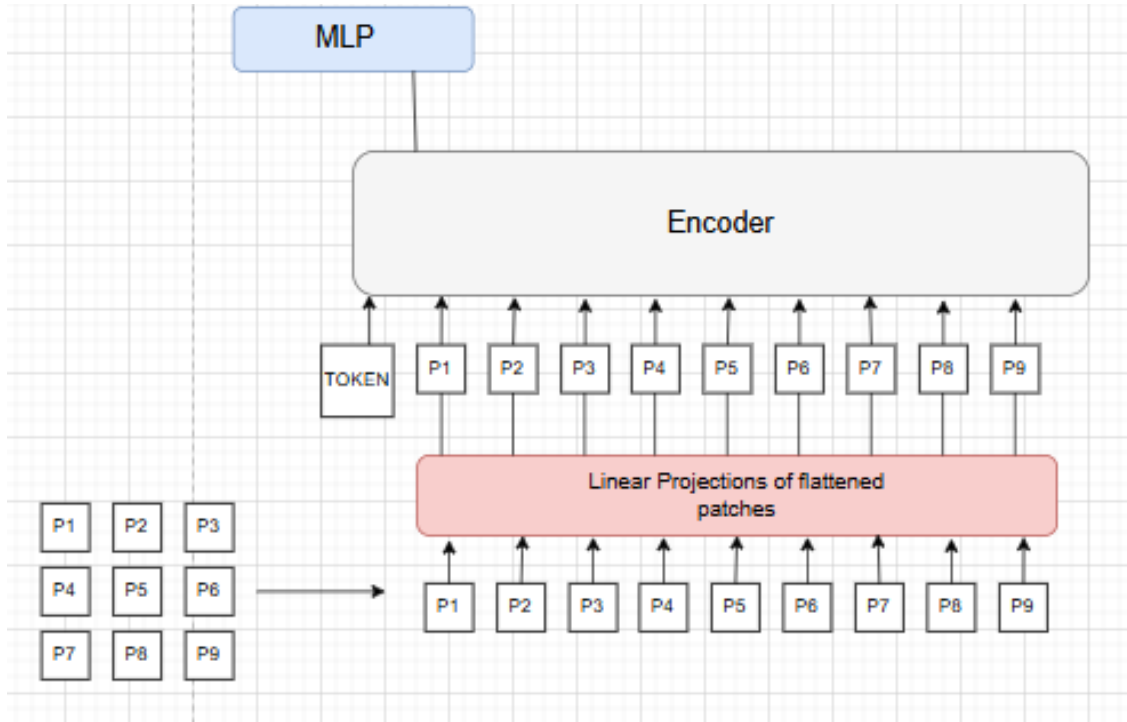


Figura 19: Esquema de *Vision Transformer*. Esta imagen está basada en los ejemplos de [5].

[5]

El modelo **ViT** proporcionó resultados excelentes en un dataset de 300 millones de imágenes. El artículo [5] concluye en que los *Transformers* no generalizan bien cuando la cantidad de datos es insuficiente. En este punto entra en juego **DeiT (Data-efficient image Transformer)**, que es una versión de **ViT** que introduce ciertas variaciones con el fin de mejorar la eficiencia de ViT.

La primera novedad es el conjunto de entrenamiento, que se opta por **ImageNet** que tiene 1.281.167 de ejemplos de entrenamiento, reduciendo así el coste de entrenar sobre un conjunto de 300 millones de ejemplos como en **ViT**. Además, se proponen cambios en el entrenamiento, desde la variación

de ciertos hiperparámetros como el uso de otro valor de la **tasa de aprendizaje**, hasta la inclusión de nuevos elementos como **label smoothing** o **stochastic depth**. Un punto clave en la mejora de eficiencia es el uso de **Data augmentation**, que es la introducción de nuevos ejemplos de entrenamiento basados en ejemplos existentes a los que se les aplica una transformación como por ejemplo una rotación o cambios de iluminación. Este principio tiene el objetivo de aumentar la diversidad del conjunto de datos y que el modelo generalice mejor.

Finalmente, el modelo usado es **deit_base_patch16_224** de la librería **timm** [29]. Este modelo tiene las siguientes *features*:

- **12 bloques en el encoder.**
- **Imágenes de tamaño 224x224** $\rightarrow (H, W) = (224, 224)$.
- **Tamaño del patch 16x16x3** $\rightarrow (P, P) = (16, 16)$ y $C = 3$ (RGB) $\Rightarrow$ el tamaño del patch es $16 \times 16 \times 3 = 768$.
- **196 patches por imagen más el token de clase** $\rightarrow (P, P) = (16, 16)$ y $(H, W) = (224, 224) \Rightarrow N = \frac{HW}{P^2} = \frac{224^2}{16^2} = 196$ más el token de clase hacen un total de **197 patches**.
- **Tamaño de cada patch tras proyección lineal** $D = 768$.

3.5. Fase de entrenamiento

En esta sección detallaremos el entrenamiento para ambos modelos. Cabe recordar que si bien es uno de los objetivos de este trabajo y paso previo necesario para otros experimentos, el objetivo principal no es buscar el modelo perfecto variando entre diferentes modelos e hiperparámetros. El objetivo principal es el desarrollo métodos de detección OOD. Ambos modelos fueron entrenados en el mismo contexto:

- **Tamaño del batch** = 64.
- **Función de pérdida:** entropía cruzada.
- **Optimizador:** SGD con **momentum** 0.9 y **weight decay** = 0.0005.
- **Número de épocas** = 50.
- **Métricas para evaluar el rendimiento:** función de pérdida en entrenamiento y validación y **accuracy** en validación:

$$accuracy = \frac{\text{Número de aciertos}}{\text{Número total de ejemplos}}$$

3.6. Evaluación OOD

Como hemos comentado en las secciones 2.6 y 3.3, la evaluación OOD es una de las tareas que ha cobrado más importancia recientemente dentro del mundo del machine learning. De forma muy resumida, consiste en que el modelo aprenda a separar los ejemplos que pertenecen verdaderamente a la distribución de los datos, denotados como datos **ID (In Distribution)**, de los ejemplos que no pertenecen a la distribución, es decir, **OOD (Out Of Distribution)**.

La capacidad del modelo de separar ejemplos ID y OOD, dota al mismo de robustez y seguridad. Por ejemplo, considerando nuestro problema, si el modelo desplegado en ámbito profesional recibe una imagen parecida a una biopsia de próstata (como podría ser una biopsia de cualquier otro tejido), el modelo proporcionará una etiqueta que puede resultar en la predicción errónea de un tumor. Por tanto esto es peligroso y ha de ser controlado.

3.6.1. Datasets OOD

En la sección 3.2 introducimos el dataset sobre el que realizar el entrenamiento. En contexto de detección OOD, este será nuestro dataset **ID**. El siguiente paso es definir qué conjunto o conjuntos de datos vamos a utilizar para evaluación OOD. Para ello definimos dos tipos, **Near-OOD** y **Far-OOD**.

3.6.1.1. Far-OOD: CIFAR-10

El término **Far-OOD** hace referencia a un conjunto de datos cuya distribución sea más alejada a los datos ID que otro dataset denominado **Near-OOD**. Para este trabajo, seleccionamos el conjunto de datos **CIFAR-10** disponible en el framework OpenOOD.

Este conjunto, descrito en [30], es bastante distante al nuestro, ya que está formado por imágenes de animales y vehículos, y por tanto se presupone lejano a la distribución de biopsias de tejido prostático. Usaremos el dataset descrito en el archivo `test_cifar10.txt` proporcionado por OpenOOD que cuenta con 9.000 ejemplos.

3.6.1.2. Near-OOD: COVID

El conjunto ID está formado por biopsias prostáticas (microscopio, tejido, color), mientras que CIFAR-10 son fotografías naturales, completamente fuera del dominio médico, por lo que se clasifica como Far-OOD. El conjunto COVID son radiografías de tórax (rayos X, escala de grises), por lo que pertenecen al mismo dominio médico que el ID, aunque la modalidad de adquisición es distinta. Por ello se clasifica como Near-OOD, debido a que es más cercano al ID que CIFAR-10, aunque no sea semánticamente idéntico.

Este conjunto de datos fue introducido por [31] y tiene varias versiones. El dataset no tiene clases como tal, aunque se diferencia entre positivos y negativos. En este experimento usaremos el conjunto

del archivo proporcionado por OpenOOD `test_bimcv.txt` que cuenta con **891 ejemplos**.

3.6.2. Métricas de evaluación OOD

Previo paso a describir algunas de estas métricas debemos definir algunos conceptos:

Cuadro 1: Categorización de resultados en función del valor real y la predicción de un modelo.

	Predicción positiva	Predicción negativa
Clase positiva	Verdadero Positivo (TP)	Falso Negativo (FN)
Clase negativa	Falso Positivo (FP)	Verdadero Negativo (TN)

En esta tabla podemos observar una relación entre lo predicho por el modelo y el valor real. En función de estos valores, definimos:

- **Sensibilidad, Recall o True Positive Rate:**

$$TPR = \frac{TP}{TP + FN}$$

- **False Positive Rate:**

$$FPR = \frac{FP}{FP + TN}$$

- **Precision:**

$$Precision = \frac{TP}{TP + FP}$$

La **curva ROC (Receiver Operating Characteristic)** se representa como la **TPR** en función de la **FPR** para distintos umbrales. Cada punto de la curva ROC representa un par (TPR,FPR) correspondiente a un determinado umbral de decisión. Como los valores de FPR Y TPR están en $[0, 1]$, una prueba que muestre una separabilidad perfecta tiene como Curva ROC la recta constantemente igual a 1. Esto básicamente implica que $TPR = 1$ es decir que $FN = 0$ y por tanto la separación es perfecta. En este contexto surge la métrica AUROC.

AUROC (Area Under the ROC curve) representa, como su propio nombre indica, el área bajo la curva ROC. Recordemos que $(TPR, FPR) \in [0, 1]^2$, luego la curva ROC está contenida en el cuadrado unidad y por tanto el área es máxima cuando la curva ROC es constantemente igual a 1.

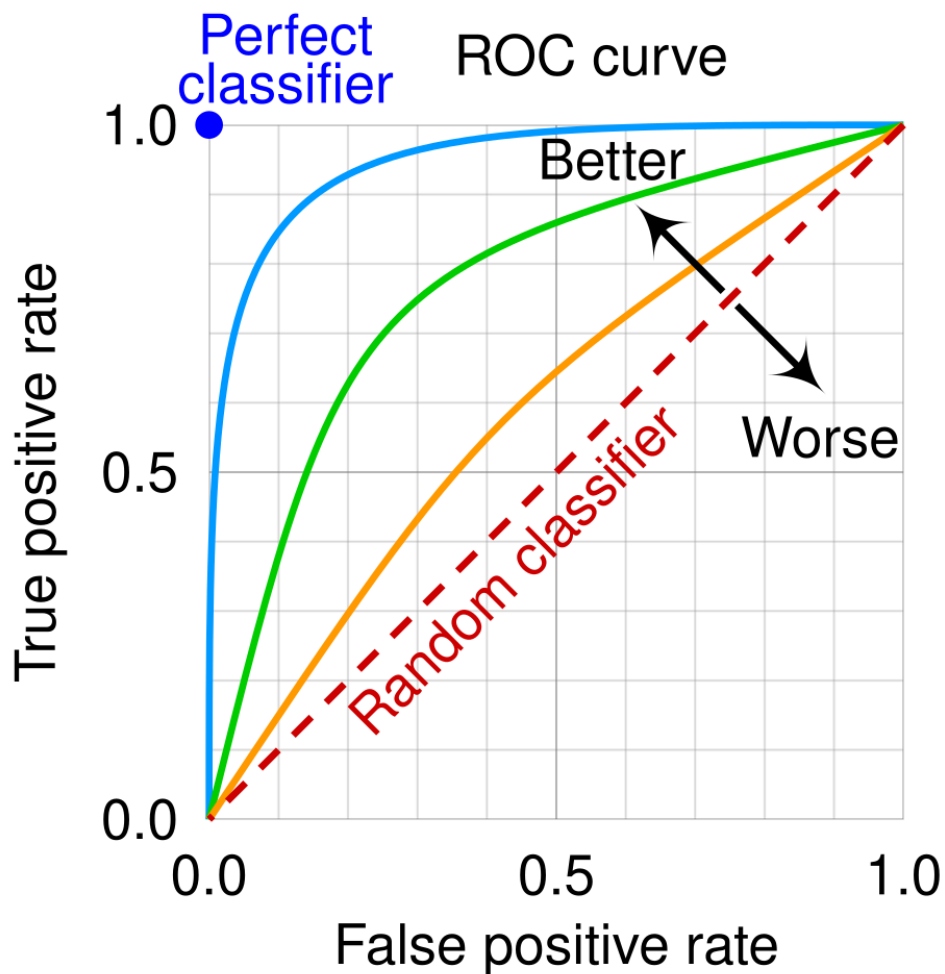


Figura 20: Ejemplo de ROC curve. https://commons.wikimedia.org/wiki/File:Roc_curve.svg#file.

Como se ilustra en la imagen, cuanto más cercano sea al cuadrado unidad, área más cercana a 1 tendrá la curva y mejor será el modelo. Por tanto, $\text{AUROC} \in [0, 1]$ y cuanto más cercano a 1 es este valor, mejor. En la práctica, AUROC suele ser como mínimo 0.5, ya que como vemos en la imagen, esto corresponde a un clasificador que asigne un valor aleatorio a cada ejemplo.

[32] [33]

La siguiente métrica es **FPR@95 (False Positive Rate at 95 % TPR)**, es decir, se calcula la FPR para el umbral en el que la TPR es de 95 %. Por tanto, al medir la tasa de falsos positivos, esta métrica es mejor cuanto más próxima al 0.

$$\text{FPR@95} = \text{FPR} \quad \text{si } \text{TPR} = 0,95$$

[34]

Similar a AUROC, presentamos **AUPR (Area Under the Precision Recall curve)**. En la curva Precision-Recall, representamos la TPR (Recall) en función de la Precision, donde cada punto es el valor (Recall, Precision) para cierto umbral. Asociada a esta curva, tenemos **AUPR_IN** y **AUPR_OUT**. En AUPR_IN, se toma como positiva la clase de ejemplos **ID**, mientras que en AUPR_OUT la clase positiva es **OOD**. Se puede interpretar como la capacidad que tiene el modelo de detectar las muestras que pertenecen a nuestra distribución. Para ambas métricas, tenemos que de nuevo los valores son mejores cuanto más cercanos a 1.

[35]

3.6.3. Propuestas de evaluación OOD

En esta sección detallaremos los métodos de evaluación OOD que se han utilizado. El objetivo principal de este trabajo es encontrar un método de detección OOD basado en IA explicable. Para llevar a cabo esta tarea, definiremos una serie de **posprocesadores**, que se encargarán de asignar una **puntuación o score** a cada muestra ID y OOD. El objetivo es obtener scores en muestras ID y OOD, de forma que sean similares entre muestras ID y entre muestras OOD pero distintos al comparar ambas clases.

Comenzaremos usando dos métodos implementados en OpenOOD, *Maximum Softmax Probability* y *Virtual-logit Matching* exclusivamente para el modelo **ResNet-50** que fue el modelo inicialmente seleccionado para este trabajo. Posteriormente introduciremos varias propuestas propias, haciendo una comparación entre todos los métodos. Estos métodos también serán exclusivos de **ResNet-50** hasta llegar a la propuesta final **BestXAIPostProcessor** que incorporará el uso de **DeiT**.

La mayoría de estas propuestas están basadas en el uso del algoritmo de **XAI CRP** (sección 2.7) y la **distancia de Mahalanobis** (sección 2.2.3). La idea principal será tomar como score el cálculo de la distancia de Mahalanobis a la distribución (μ, Σ) de los datos ID. Estos datos dependerán de las *features* y vectores de IA explicable de la distribución ID.

3.6.3.1. Maximum Softmax Probability

Sea $z = (z_1, \dots, z_k)$ el vector de logits asociado a un ejemplo x . Recordemos que la función softmax calcula:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}} \quad i = 1, \dots, k$$

El método **Maximum Softmax Probability** es un método sencillo basado en la confianza del modelo en la predicción. Si la probabilidad asociada a la predicción final es alta, el modelo se presupone seguro de su elección y por tanto parece indicar que será una muestra ID. En cambio, si la probabilidad asociada a la predicción no es muy alta, indica que el modelo no está seguro de la predicción final y por tanto podría ser una muestra OOD. Finalmente definimos el **score** de una muestra con el método MSP como:

$$\text{MSP}(x) = \max_{i \in \{1, \dots, K\}} \text{softmax}(z_i)$$

donde $z = (z_1, \dots, z_k)$ es el vector de logits asociado a la muestra x [27], [26]. Este método fue probado **únicamente en ResNet-50**, ya que era el modelo original escogido para la evaluación OOD.

3.6.3.2. Virtual-logit Matching

Además del uso de logits, este método añade el uso de *features*, que es la salida de la capa anterior a la capa de salida. En la implementación de OpenOOD, se calcula el score de la siguiente forma:

- 1. Cálculo de energía:

$$E(x) = \log \sum_i e^{z_i} \quad \text{donde } z_i \text{ es el } i\text{-ésimo logit de } x.$$

- 2. Se obtiene la matriz de pesos W y el vector de bias b y se calcula el **centro del espacio de features** como:

$$u = -W^{-1}b$$

- 3. Se calcula la matriz de covarianzas Σ . Calculamos el espacio **Null Space** N_S como el espacio de los vectores propios asociados a los valores propios más pequeños. La cantidad de valores propios depende del hiperparámetro **dim**, donde escogemos **512 valores** para obtener $feature_{size} - 512$ vectores nuestro problema.
- 4. Se proyecta la extitfeature $f(x)$ centrada con respecto a u sobre el Null Space N_S y se calcula la norma, es decir:

$$v(x) = \|(f(x) - u)N_S\|$$

A este valor se le conoce como virtual logit.

- 5. Se calcula α , que es la relación entre el valor medio del máximo de los logits en el conjunto de ejemplos y el valor medio del virtual logit:

$$\alpha = \frac{\frac{1}{N} \sum_{i=1}^N \max_{j \in \{1, \dots, k\}} (z_j^i)}{\frac{1}{N} \sum_{i=1}^N v(x_i)} \quad \text{donde } z_j^i \text{ es el logit } j\text{-ésimo del ejemplo } x_i$$

- 6. Se calcula el score:

$$ViM(x) = E(x) - \alpha v(x)$$

[27], [26]

3.6.3.3. XAIPostProcessor

Este método está basado en el uso de *features* y vectores de XAI. Esto se debe a la sustancial mejora de ViM (basado en logits y *features*) con respecto a MSP (basado en logits únicamente) de acuerdo a la tabla 2. El posprocesador realiza lo siguiente:

- 1. Para cada ejemplo $x \in \text{ID}$, calculamos sus *features* $f(x)$ y su vector CRP . Finalmente se concatenan:

$$v(x) = [f(x), \text{CRP}(x)]$$

- 2. Calculamos la distribución (μ, Σ) de estos nuevos vectores.
- 3. Finalmente se calcula el score como:

$$\text{score}(x) = -((v(x) - \mu)^T \Sigma^{-1} (v(x) - \mu))$$

Se devuelve la negativa de la distancia de Mahalanobis por una razón, y es que queremos que los valores más altos supongan más probabilidad de ID. Por tanto, como cuánto menor distancia a la distribución, mayor probabilidad de ID, mayor valor del negativo de la distancia implica mayor probabilidad de ID.

3.6.3.4. MahaPostProcessor

Para hacer una comparación sobre el uso de XAI, se propone este posprocesador, que realiza las mismas operaciones que **XAIPostProcessor** omitiendo el uso de vectores de XAI, es decir:

- 1. Calculamos la distribución (μ, Σ) de los ejemplos del conjunto ID.
- 2. Se calcula el score como:

$$\text{score}(x) = -((x - \mu)^T \Sigma^{-1} (x - \mu))$$

Se devuelve la negativa de la distancia de Mahalanobis por la misma razón que en XAIPostProcessor.

3.6.3.5. XAIPostProcessor (versión normalizada)

De acuerdo a lo expuesto en la tabla 2, las métricas en XAIPostProcessor y MahaPostProcessor resultaron idénticas. Tras comprobar que los valores del vector proporcionado por CRP eran de orden muy inferior a las *features*, se propuso un nuevo método que incluía un proceso de normalización, con objetivo de que el vector XAI y de *features* tuvieran la misma importancia en el cálculo de la distancia:

- 1. Para cada ejemplo $x \in \text{ID}$, calculamos sus *features* $f(x)$ y su vector $\text{CRP}(x)$. Finalmente se concatenan:

$$v(x) = [f(x), \text{CRP}(x)]$$

- 2. Supongamos que $v(x) \in \mathbb{R}^D$ y que tenemos N muestras, normalizamos el vector de la siguiente forma:

$$n(x_i)_j = \frac{v(x_i)_j}{\max_{z \in \{1, \dots, N\}} |v(x_i)_j|} \quad \forall j = 1, \dots, D$$

Es decir, para la variable de la posición j -ésima, calculamos el máximo en valor absoluto de esa variable en todas las muestras ID.

- 3. Calculamos la distribución (μ, Σ) de estos nuevos vectores $n(x)$.
- 4. Finalmente se calcula el score como:

$$score(x) = -((n(x) - \mu)^T \Sigma^{-1} (n(x) - \mu))$$

donde se plantea la misma normalización que en el punto 2 para cada muestra.

3.6.3.6. BestXAIPostProcessor

Este método propone el uso de los vectores de XAI calculados por el algoritmo CRP para decidir qué *features* intervendrán en el cálculo de la distancia. Por el diseño del mismo, el vector de *features* y de CRP tendrán la misma dimensión. El proceso de cálculo de scores es el siguiente:

- 1. Para cada ejemplo $x \in ID$, calculamos sus *features* $f(x) \in \mathbb{R}^D$ para cierto D dependiente del modelo.
- 2. Calculamos la distribución (μ, Σ) de *features* de los ejemplos ID.
- 3. Para el cálculo de scores, comenzamos calculando el vector de *features* $f(x)$ y el vector de XAI $CRP(x)$. En el caso de **DeiT**, seleccionaremos como *features* las propias *features* del **token de clase**, ya que este contiene la representación de la imagen.
- 4. Para cada ejemplo, escogemos los índices de las variables del $p\%$ de valores mayores en valor absoluto del vector $CRP(x)$, es decir, $\forall x \in ID \cup OOD$:

$$\text{Best_index}_x = \{j_1, \dots, j_{(\frac{p}{100}D)} : |CRP(x)_{j_1}| > |CRP(x)_{j_2}| > \dots > |CRP(x)_{j_{(\frac{p}{100}D)}}|\}$$

- 5. Calculamos unos nuevos vectores $v(x)$ tal que:

$$v(x) = \begin{cases} f(x)_j & \text{si } j \in \text{Best_index}_x \\ 0 & \text{si } j \notin \text{Best_index}_x \end{cases}$$

es decir, para el cálculo de distancias a la distribución (μ, Σ) utilizamos los índices de los $p\%$ mejores valores de CRP (en valor absoluto). Las *features* en estos índices mantendrán su valor mientras que en caso contrario se cambiará a 0.

- 6. Finalmente, calculamos la distancia de Mahalanobis.

$$score(x) = -((v(x) - \mu)^T \Sigma^{-1} (v(x) - \mu))$$

Los métodos descritos en esta sección se encuentran implementados en <https://github.com/josemartin2912/TFG/tree/main>.

4. Resultados

4.1. Resultados ResNet-50

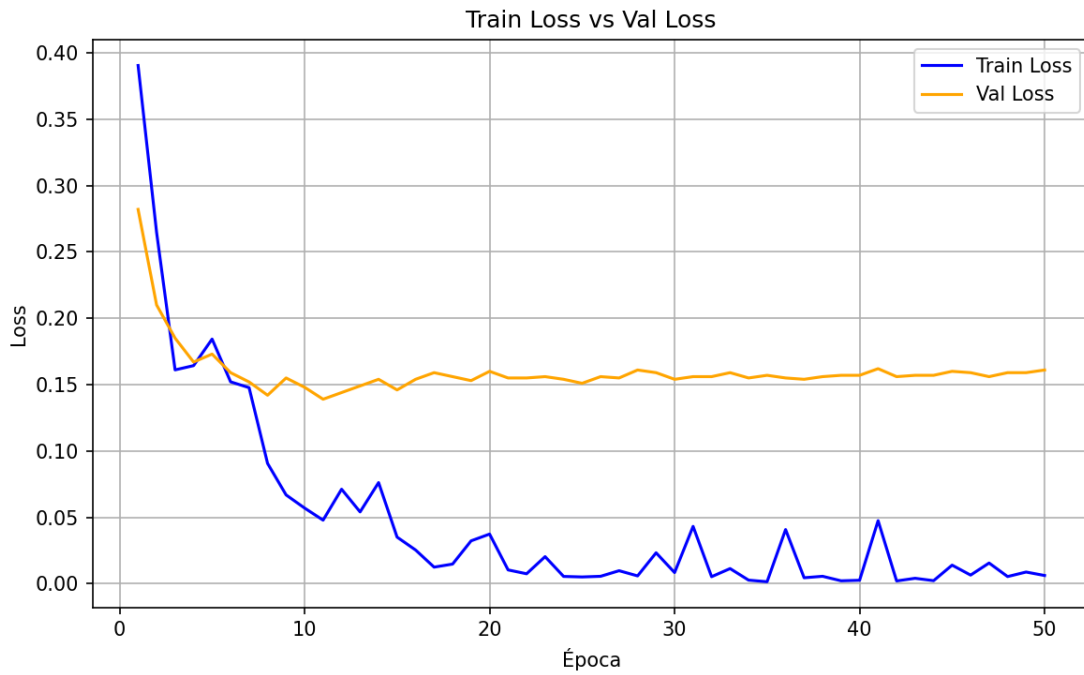


Figura 21: Resultados de entrenamiento de ResNet-50. Comparación de evolución de función de pérdida en el conjunto de entrenamiento y validación.

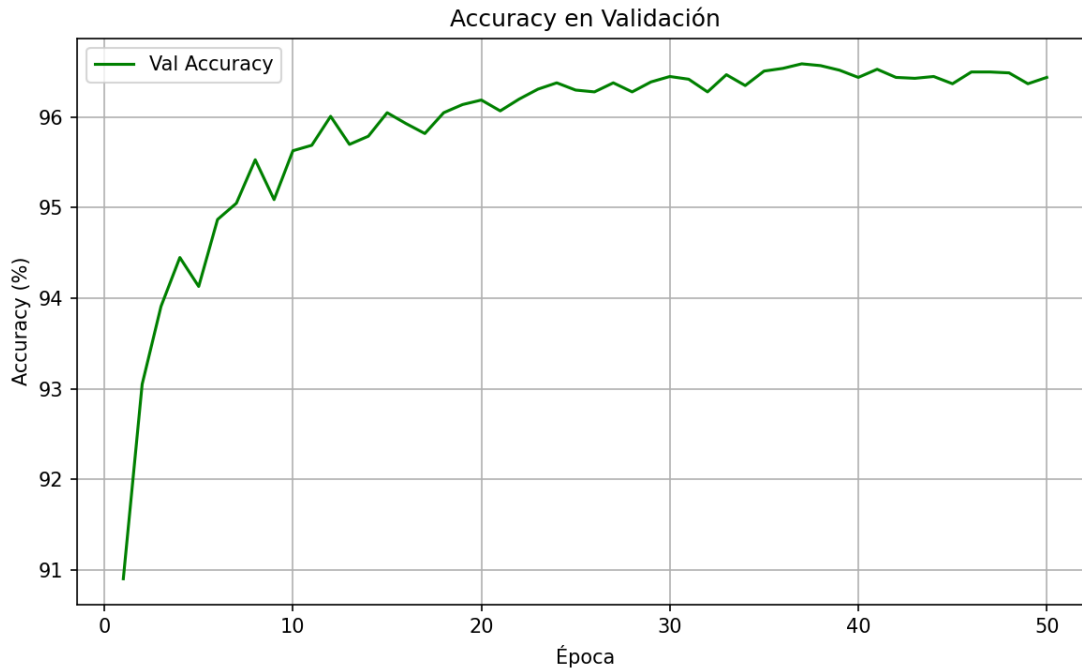


Figura 22: Resultados de entrenamiento de ResNet-50. Evolución de la *accuracy*.

Observaciones:

- El tiempo de entrenamiento fue de **11 horas 13 minutos y 59 segundos** lo que supone unos **13 minutos y 29 segundos por época**.
- Como se aprecia en la figura 21, la pérdida tanto en entrenamiento como en validación presenta valores muy bajos, lo que indica que el modelo consigue un buen aprendizaje sobre los datos. De hecho, se puede ver como en entrenamiento la función de pérdida se acerca al 0.
- Debido a la separación que hay entre las gráficas de entrenamiento y validación, y como esta última no presenta mejora tras la época 10, podría parecer que estamos ante un caso de **sobreajuste** (es decir, que el modelo aprende de memoria los datos del conjunto de entrenamiento y no es capaz de generalizar a datos que no ha visto, es decir, ejemplos de validación). Sin embargo, esto no es así pues como se ha comentado, la pérdida en entrenamiento oscila en torno al 0 y por tanto no se consigue mejora en el modelo, por lo que no se aprecia mejora en validación. Por tanto, la diferencia que se aprecia entre las gráficas de pérdida no es overfitting, sino la **brecha de generalización** que es la diferencia de rendimiento entre el conjunto de entrenamiento y validación debido a que el modelo no ha visto los datos de validación durante el entrenamiento.
- Con respecto a la figura 22, se obtienen conclusiones similares a las anteriores. El modelo presenta muy buenos resultados, dando lugar al mejor valor de *accuracy* de **96.59 en la época 37**, donde se toman los parámetros del modelo tras esta época para futuros experimentos. Este valor de *accuracy*, como se ha comentado, es un valor muy alto y es un dato muy esperanzador de cara a un posible uso profesional (siempre bajo la supervisión de un profesional).

- Además, se confirma la idea de que a partir de la época 10 la gráfica se estabiliza y por tanto el modelo deja de mejorar de forma significativa.
- En la primera imagen también se puede apreciar, pero es más notorio como en la segunda imagen se parte desde unos valores muy buenos de métricas (90% en *accuracy*). Esto es gracias al proceso de **fine tuning** de un modelo preentrenado adaptado al contexto de este trabajo (**transfer learning**, sección 3.4.1).

4.2. Resultados DeiT



Figura 23: Resultados de entrenamiento de DeiT. Comparación de evolución de función de pérdida en el conjunto de entrenamiento y validación.

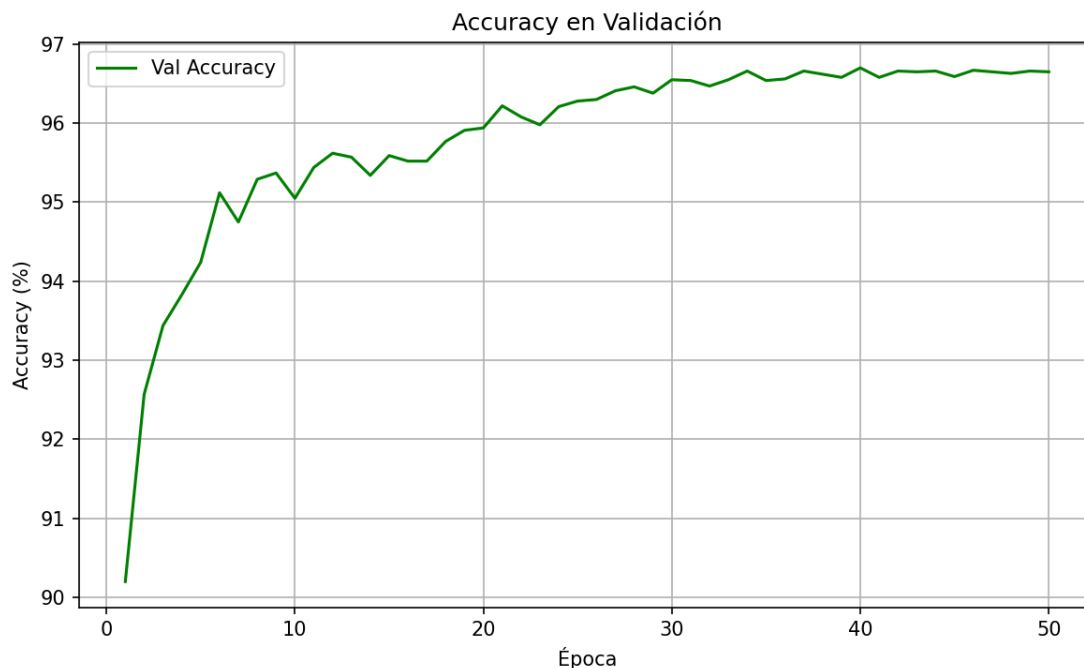


Figura 24: Resultados de entrenamiento de DeiT. Evolución de la *accuracy*.

Observaciones:

- El tiempo de entrenamiento fue de **37 h 24 min 33 s** teniendo un tiempo por época de **44 min 53 s/época**, lo cual es bastante superior al caso anterior.
- En cuanto a valores de las métricas, presentan casos muy similares al caso anterior (figuras 21 y 22), donde se alcanza el mejor valor de *accuracy* en validación de **96.7 en la época 40**. Del mismo modo que en el caso anterior, se observan comportamientos muy similares no sólo en valores sino en la evolución de las métricas. Por ejemplo, se aprecia de nuevo como gracias al fine tuning de un modelo preentrenado, se parte desde valores muy altos como 90 % en *accuracy* mejorando en pocas épocas hasta llegar cerca de 96 % en *accuracy* y estabilizándose en torno a ese valor.

4.3. Comparativa de resultados

De acuerdo a lo expuesto en estas secciones, los valores de las métricas y la evolución de las gráficas es bastante similar en ambos casos. En el caso de **ResNet-50** tenemos que el mejor modelo se alcanza en la **época 37 con un valor de 96.59 en *accuracy* y 11 horas, 13 minutos y 59 segundos de entrenamiento**. Para **DeiT**, estos valores alcanzan un **96.70 en *accuracy* en la época 40 y 37 horas, 24 minutos y 33 segundos** de tiempo de entrenamiento.

En el caso de tener que escoger un modelo exclusivamente para una tarea de clasificación, quizá la mejor opción sería **ResNet-50**, porque la poca pérdida con respecto a DeiT en *accuracy* se

compensa con el ahorro de más de 25 horas de tiempo de entrenamiento.

4.4. Resultados de primeros experimentos de evaluación OOD

En esta sección se describen los resultados de la evaluación OOD con los posprocesadores MSP, ViM, XAIPostprocessor y su versión normalizada y MahaPostprocessor en el modelo ResNet-50. En un principio se optó por el uso de este modelo, pero más adelante se describe por qué se hizo uso también del modelo DeiT.

Cuadro 2: Comparación de métodos de detección OOD

DATASET	FPR@95	AUROC	AUPR_IN	AUPR_OUT	MÉTODO OOD
COVID	91.83	53.31	94.44	6.72	MSP
CIFAR-10	14.62	95.38	97.24	91.85	MSP
COVID	2.32	99.39	99.96	89.88	ViM
CIFAR-10	0.01	100.00	100.00	100.00	ViM
COVID	0.59	99.78	99.98	93.61	XAIPostProcessor
CIFAR-10	0.09	99.98	99.98	99.97	XAIPostProcessor
COVID	0.59	99.78	99.98	93.61	MahaPostProcessor
CIFAR-10	0.09	99.98	99.98	99.97	MahaPostProcessor
COVID	2.21	98.96	99.93	74.70	XAIPostProcessor(norm)
CIFAR-10	0.08	99.98	99.98	99.97	XAIPostProcessor(norm)

Observaciones:

- En el caso de **MSP**, los resultados son bastante pobres en ambos datasets, sobre todo en el dataset COVID. Por ejemplo, se observa un valor de 91.83 en FPR@95, lo cual, como vimos en la sección 3.6.2, indica que para una tasa de verdaderos positivos de 95 %, resulta en una tasa de falsos positivos de 91.83 %. Esto quiere decir que para alcanzar un acierto del 95 % en las predicciones sobre muestras ID, el modelo predice el 91.83 % de muestras OOD de forma errónea, es decir, como muestras ID. Por tanto, la conclusión es que el umbral para FPR de 95 es muy bajo, ya que también acepta el 91.83 % de OOD. Otro ejemplo del mal rendimiento es el valor de **53.31** en AUROC, es decir, este método de detección OOD ofrece prácticamente los mismos resultados que asignar a cada muestra aleatoriamente la etiqueta ID u OOD, ya que este método da como valor AUROC=50.
- Es lógico que el conjunto de CIFAR-10 proporcione mejores resultados, por ser la distribución de sus datos aparentemente más lejana que en los datos de COVID. Aun así, se espera encontrar un método que ofrezca mejores resultados, ya que no debería ser muy complicado distinguir entre biopsias (ID) y vehículos y animales (CIFAR-10).
- Tal y como se aprecia en la tabla anterior, utilizando el posprocesador **ViM** tanto en COVID como en CIFAR-10 se obtienen resultados muy buenos, donde en **AUROC**, **AUPR_IN** y **AUPR_OUT** tienen valores cercanos a 100 % y **FPR95** cerca de 0. Por ejemplo, los resultados indican un valor de 2.32 en FPR95, lo que supone que únicamente el 2.32 % de

ejemplos OOD han sido clasificados como ID para el umbral en el que se han clasificado correctamente el 95 % de ejemplos ID. Para $AUPR_{OUT} = 89.88\%$, el dato es cercano a 100 %. Este valor indica la buena capacidad de detectar muestras OOD.

- Comparando **ViM** con **MSP**, el primero presenta métricas mucho mejores. En MSP se dan resultados que indicaban que era prácticamente igual de efectivo clasificar ID y OOD de forma aleatoria.
- Como se pudo ver anteriormente, con **XAIPostProcessor** conseguimos en el dataset COVID una mejora en las 4 métricas con respecto a **ViM** que era el mejor método hasta el momento. En el conjunto de CIFAR-10, los resultados son ligeramente peores, pero siguen siendo casi perfectos. Además, realmente debe preocupar el conjunto de COVID que es el más cercano a la distribución ID y por tanto es más interesante estudiar la separabilidad en este caso. Este posprocesador surgió por la mejora de ViM con respecto a MSP, donde el primero hacía uso de *features*. Por tanto, parece que las *features* son un buen discriminador de ejemplos OOD.
- En el caso de **MahaPostProcessor**, se dan exactamente las mismas métricas que en **XAI-PostProcessor**. Cabe recordar que este posprocesador se implementó con el objetivo de comparar el uso de *features* y XAI con simplemente *features*. De esto se deduce que el vector de IA explicable realmente no está aportando en el cálculo de la distancia de Mahalanobis. Tras realizar un simple experimento, se pudo ver como para una muestra, los valores de $f(x)$ eran del orden de entre 100 y 1000 veces más grandes que los valores de $CRP(x)$. Esto podría ser un indicio de como los valores del vector de XAI prácticamente no importan para el cálculo de la distancia.
- En la versión normalizada de XAIPostProcessor, se dota a cada variable del vector de *features* y CRP de la misma importancia en el cálculo de distancias mediante la normalización de cada variable. En este caso las métricas resultantes son relativamente parecidas a los casos sin normalizar. Sin embargo, estos datos son ligeramente inferiores a los experimentos previos.
- Aun no suponiendo una mejora, dotar de misma importancia a *features* y CRP presenta unos resultados bastante buenos, por lo que el uso de IA explicable resulta en un separador razonable en este problema.
- La diferencia más notoria es en el valor de **AUPR_OUT** donde:
 - XAIPostProcessor y MahaPostProcessor $\rightarrow 93.61$.
 - XAIPostProcessor (versión normalizada) $\rightarrow 74.70$.

Por tanto, la capacidad para detectar ejemplos OOD es sustancialmente peor que en métodos anteriores, aunque sigue siendo un valor aceptable.

- Las conclusiones que se obtienen de todos estos experimentos es que se han implementado métodos que mejoran a otros proporcionados por OpenOOD como **MSP** y **ViM**. Aun así, como el objetivo de este trabajo es el uso de XAI como parte de un posprocesador, la **versión normalizada de XAIPostProcessor** presentaba resultados ligeramente inferiores a **XAIPostProcessor** y **MahaPostProcessor**, que tenían poco o nada en cuenta los valores del vector de XAI. Pese a esto, es positivo que la versión normalizada también aporte buenos valores, porque indica que la IA explicable puede ser buen separador. Con el objetivo de realizar una aportación de valor, planteamos nuevos métodos basados en el ahorro de *features* bajo el criterio de XAI.

4.5. Resultados de BestXAIPostprocessor

En este punto, se introduce el uso de **DeiT** junto a ResNet-50, ya que los *Transformers* han demostrado buen funcionamiento operando reduciendo el número de *features*. Se obtienen los siguientes resultados en el caso de ResNet-50:

Cuadro 3: Comparación de métodos de detección OOD. Selección de mejores *features* en ResNet-50: [25 %, 50 %, 75 %, 80 %, 90 %, 100 % (equivalente a XAIPostProcessor)].

DATASET	FPR@95	AUROC	AUPR_IN	AUPR_OUT	MÉTODO OOD
COVID	0.59	99.78	99.98	93.61	XAIPostProcessor
CIFAR-10	0.09	99.98	99.98	99.97	XAIPostProcessor
COVID	2.21	98.96	99.93	74.70	XAIPostProcessor(norm)
CIFAR-10	0.08	99.98	99.98	99.97	XAIPostProcessor(norm)
COVID	24.23	85.61	98.95	17.72	25 % best ResNet-50
CIFAR-10	9.02	97.77	98.60	96.34	25 % best ResNet-50
COVID	24.08	86.37	99.00	18.62	50 % best ResNet-50
CIFAR-10	9.45	97.94	98.63	96.93	50 % best ResNet-50
COVID	12.88	93.55	99.55	33.12	75 % best ResNet-50
CIFAR-10	3.58	99.28	99.49	99.05	75 % best ResNet-50
COVID	7.51	96.81	99.78	51.85	80 % best ResNet-50
CIFAR-10	1.78	99.61	99.72	99.49	80 % best ResNet-50
COVID	1.51	99.49	99.96	88.53	90 % best ResNet-50
CIFAR-10	0.13	99.94	99.96	99.92	90 % best ResNet-50

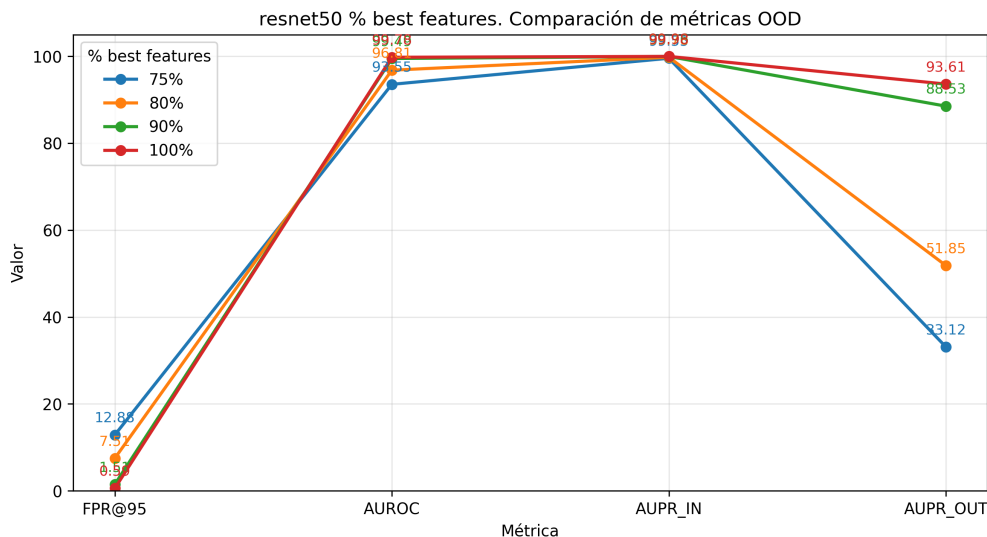


Figura 25: Resultados de detección OOD para ResNet-50 con BestXAIPostProcessor en el conjunto de COVID. Las pruebas se realizan para valores diferentes de p % con el objetivo de ver cómo reducir dimensionalidad en el espacio de *features* influye en el resultado final.

Conforme a lo visto en la imagen 25 y la tabla 3, existe un orden claro en cuanto a resultados: **100 % (XAIPostProcessor) > 90 % > 80 % > 75 %**. No parece que sea muy útil la reducción de dimensionalidad en el conjunto de *features*, ya que sobre todo considerando la métrica **AUPR_OUT**, para 75 % y 80 % los resultados son muy pobres.

En el caso de **DeiT** obtenemos:

Cuadro 4: Comparación de métodos de detección OOD. Selección de mejores *features* en DeiT: [25 %, 50 %, 75 %, 100 %].

DATASET	FPR@95	AUROC	AUPR_IN	AUPR_OUT	MÉTODO OOD
COVID	5.05	98.02	99.82	70.43	25 % best DeiT
CIFAR-10	8.31	97.66	97.93	96.38	25 % best DeiT
COVID	6.39	96.66	99.77	48.44	50 % best DeiT
CIFAR-10	8.11	97.16	98.36	92.56	50 % best DeiT
COVID	1.94	99.09	99.94	77.09	75 % best DeiT
CIFAR-10	2.23	99.35	99.61	98.71	75 % best DeiT
COVID	0.0	100.0	100.0	100.0	100 % best DeiT
CIFAR-10	0.0	100.0	100.0	100.0	100 % best DeiT

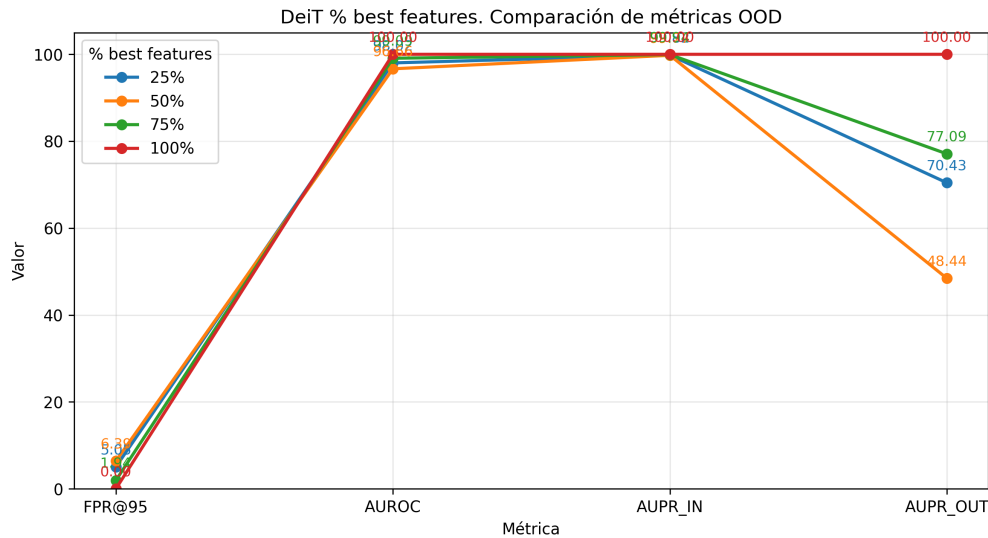


Figura 26: Resultados de detección OOD para DeiT con BestXAIPostProcessor en el conjunto de COVID. Las pruebas se realizan para valores diferentes de $p\%$ con el objetivo de ver cómo reducir dimensionalidad en el espacio de *features* influye en el resultado final.

Observaciones:

- Usando todas las *features*, (equivalente al uso de XAIPostProcessor o MahaPostProcessor), se obtiene una separabilidad perfecta de muestras ID y OOD.

- Hasta ahora, las métricas en el conjunto de CIFAR-10 habían sido uniformemente mejores que en COVID para el mismo posprocesador. En este caso, como se puede apreciar en la tabla 4, algunas métricas indican que el modelo separa mejor el conjunto de COVID que el de CIFAR-10. Esto es llamativo, pues a priori el conjunto de COVID debería ser mucho más cercano a nuestro conjunto ID y por tanto más difícil de separar.
- En este caso sí que se observa como una reducción en la dimensionalidad de *features* proporciona resultados interesantes. El modelo para el 100 % de *features* ofrece un rendimiento excelente pero, cabe destacar que no existe mucha diferencia entre el modelo para el 25 % y 75 % de *features*, confirmando la premisa de que los *Transformers* operan bien con un número reducido de *features*. El peor modelo de todos es sin duda para 50 %. Para AUROC, FPR95 y AUPR_IN no existe gran diferencia entre las 4 pruebas. Sin embargo, para 50 % el modelo ofrece un AUPR_OUT muy malo, siendo el valor de 25 % y 75 % aceptable.
- En general, DeiT ofrece mejores resultados que ResNet-50 para porcentajes parecidos, como se aprecia en las tablas 3 y 4.

Los resultados de una separación perfecta con DeiT son ciertamente sospechosos. Para confirmar que no se trata de un error se propone realizar un nuevo experimento. Este experimento es el **Kernel Density Estimation (KDE)** de la librería [36]. Se basa en estimar la función de densidad de una distribución. Se busca comprobar que efectivamente, el posprocesador separa perfectamente los ejemplos ID y OOD para DeiT y para ello representaremos las gráficas **KDE** de cada distribución de datos en base a los scores (negativa de distancia de Mahalanobis). Este experimento se realizará para ambos modelos utilizando el 100 % de *features*.

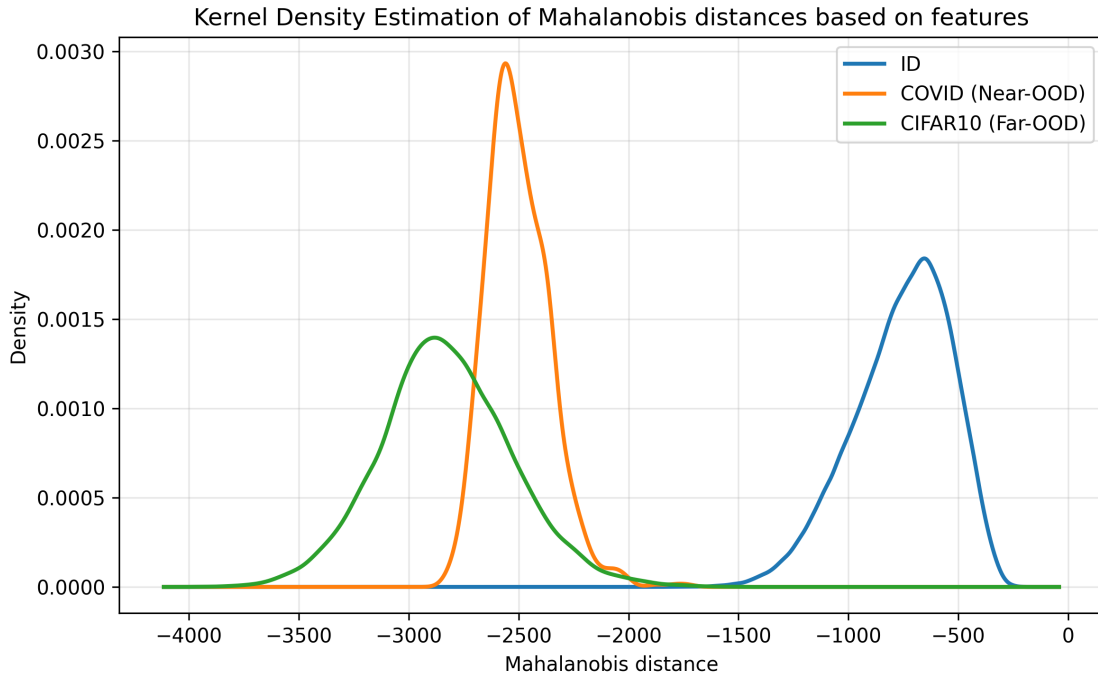


Figura 27: Prueba kde para DeiT. Estimación de funciones de densidad para cada distribución de datos.

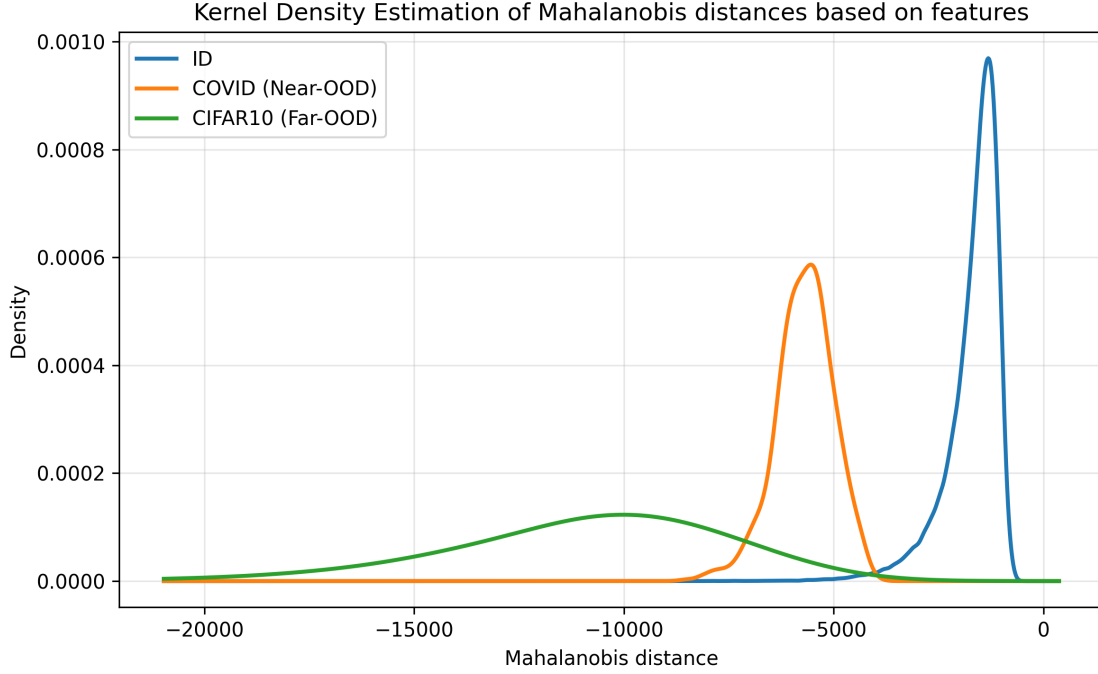


Figura 28: Prueba kde para ResNet-50. Estimación de funciones de densidad para cada distribución de datos.

Observaciones:

- De acuerdo a la figura 27, se confirma la separación perfecta con DeiT entre ID y las distribuciones OOD.
- Para ResNet-50, como se indica en la figura 28, sí que se aprecia cierto solapamiento entre la distribución ID y las distribuciones OOD, lo que concuerda con los resultados de las métricas de la tabla 3.
- Es interesante ver como las KDE en ambos modelos son bastante diferentes en cuanto a proporciones. Esto podría indicar la diferencia de extracción de *features* entre una CNN y un *Transformer*. Una posible hipótesis es la diferencia en el mecanismo de agregación entre el token de clase (DeiT) y el average pooling (ResNet-50), cuestión abierta para trabajo futuro.

5. Discusión, conclusiones y trabajos futuros

5.1. Discusión de resultados

En esta sección se discuten los resultados obtenidos en la parte experimental.

El entrenamiento proporcionó resultados de **96.59 % de accuracy** en ResNet-50 frente a **96.70 %**

en el caso de DeiT (figuras 21, 22, 23, 24). Además, la convergencia a estos valores se da relativamente rápido en ambos modelos, donde a partir de la época 10 se estabiliza la ganancia en *accuracy* e incluso se observa pérdida entre algunas épocas consecutivas. En cambio, sí que es destacable la diferencia en tiempo de entrenamiento, donde **ResNet-50** resultó en **11h 13m 59s** frente a **37h 24m 33s** en **DeiT**.

En cuanto a la detección OOD, la tabla 2 muestra como entre los dos métodos seleccionados de la literatura (MSP y ViM), el segundo ofrece resultados sustancialmente mejores. Una de las diferencias entre ambos métodos es el uso de *features* en ViM. Por tanto, los métodos propuestos involucran estas *features* en el proceso de separación de muestras OOD. El planteamiento inicial de uso de la distancia de Mahalanobis para calcular la distancia a una distribución formada por vectores de *features* y XAI (XAIPostProcessor), mejoró el rendimiento proporcionado por ViM que de por sí ofrece grandes resultados, de acuerdo a la tabla 2. En la comparación de XAIPostProcessor (*features* + XAI) con MahaPostProcessor (únicamente *features*) para comprobar la influencia del vector de XAI en la separación, se observó que el vector de XAI no contribuía realmente al cálculo de la distancia (tabla 2). Tras un análisis de magnitud de las *features* que proporcionaba ResNet-50, se comprobó como la magnitud de las *features* era entre 100 y 1000 veces más grandes que los valores aportados por el algoritmo de XAI CRP. En consecuencia, se propone una normalización de los datos (descrita en XAIPostProcessor versión normalizada). Este método permitió observar como el rendimiento en este caso era ligeramente peor que en las propuestas anteriores (2). Aun así, los resultados con respecto a **XAIPostProcessor y MahaPostProcessor** (p.ej. FPR95 0.59 % frente a 2.21 % o AUPR_OUT de 93.61 % frente a 74.70 % en la versión normalizada) son relativamente buenos, por lo que el uso del vector XAI como separador no debería quedar descartado de cara a nuevas propuestas. El hecho de no obtener una mejora usando el vector de XAI en cuanto a métricas, hizo cambiar el enfoque a buscar un método que permitiera ahorrar *features*. La tabla 3 y figura 25 indican que en ResNet-50 no aporta un beneficio claro el ahorro de *features*. Se observa como con el aumento de porcentaje de *features* utilizadas, el rendimiento mejora considerablemente, donde el único valor inferior al uso del 100 % de *features* (0.59 % FPR95 99.78 % AUROC 99.98 % AUPR_IN 93.61 % AUPR_OUT) que aporta una buena separación es 90 % (1.51 % FPR95 99.49 % AUROC 99.96 % AUPR_IN 88.53 % AUPR_OUT). Para **DeiT**, se aplicó el mismo posprocesador con la diferencia del uso de porcentajes más espaciados (25 %, 50 %, 75 % y 100 %), bajo la premisa de que los *Transformers* aportan buen rendimiento con un número reducido de *features*. En efecto, de acuerdo a la tabla 4 y figura 26, se observa que el uso del 25 % de *features* muestra unos resultados bastante buenos considerando el ahorro del 75 % de *features*. Haciendo uso del total de *features* para DeiT, se obtuvo una separación perfecta en ambos conjuntos (tabla 4 y figura 26). Para comprobar que la separación es perfecta, se realiza un experimento que consiste en estimar la función de densidad de cada distribución (ID, Near-OOD y Far-OOD) mediante KDE. Para DeiT, se observa que la figura 27 confirma la separación perfecta mientras que en ResNet-50 (figura 28) sí que se aprecia cierto solapamiento. Para concluir, cabe destacar que en el caso de DeiT, BestXAIPostProcessor proporciona algunos resultados mejores en el conjunto de COVID que en el de CIFAR-10 (para el mismo porcentaje de *features*), lo cual es llamativo porque parece romper con la idea de asignar a estos conjuntos las etiquetas de Near-OOD y Far-OOD respectivamente (tabla 4).

En cuanto al **estado del arte**, el estudio [25] está en la línea de lo expuesto en este trabajo. En dicho estudio, se hace uso de modelos como **ResNet-50 o ViT** (entre otros) para abordar un problema de clasificación sobre el mismo dataset, obteniendo resultados de 93.19 % y 96.54 % en *accuracy* respectivamente. Considerando que ViT es un modelo con una arquitectura similar a la

versión de DeiT empleada en este trabajo [28], se observa como el rendimiento es muy parecido en ambos estudios. Aun así, es llamativa la pérdida de 3 % en *accuracy* con respecto al entrenamiento en ResNet-50, que es de **96.59 %** (figura 22). En cambio, en la comparación de ViT y DeiT, se obtiene un resultado muy parecido, con valor de **96.70 % en *accuracy*** en el caso de este estudio (figura 24). En cuanto a detección OOD, no se ha encontrado en la literatura estudios que evalúen nuestro conjunto ID en comparación con otros conjuntos OOD. En cambio, el estudio [37] realiza experimentos de detección OOD usando MSP y ViM. En dicho estudio, se evalúan 4 conjuntos diferentes OOD con resultados medios para **AUROC y FPR95 de 77.25 % y 77.83 % en MSP y 90.91 % y 41.46 % en ViM** (ambos evaluados en un modelo BiT, que es una variante de la arquitectura ResNet). La tabla 2 indica que en este trabajo se obtienen valores de dichas métricas de 53.31 % y 91.83 % en MSP y de 99.39 % y 2.32 % en el caso de ViM (en el conjunto de COVID). Por tanto, ambos estudios concuerdan en la mejora sustancial que supone ViM sobre MSP en cuanto a la detección OOD, siendo destacable la gran diferencia en rendimiento en este estudio.

Este trabajo ha contado con ciertas limitaciones. Por ejemplo, el tamaño de los conjuntos de datos no es homogéneo, donde el conjunto de biopsias prostáticas cuenta con **113.491 ejemplos de entrenamiento** y **12.618 ejemplos de validación**, el conjunto CIFAR-10 con **9.000 ejemplos** y COVID con **891 ejemplos**. Por tanto, el resultado de las métricas puede estar influenciado por el número bajo de ejemplos de COVID en comparación con los otros datasets. Un aspecto relevante es el tiempo de ejecución empleado en ciertos experimentos. Previamente en esta sección se expone que el proceso de entrenamiento en **ResNet-50** resultó en **11h 13m 59s** frente a **37h 24m 33s** en **DeiT**, que son tiempos muy elevados. Además, los tiempos en detección OOD no son de la magnitud de estos últimos, pero sí que alcanzan valores superiores a los 30 minutos. Por tanto, esto supone una limitación a la hora de realizar más experimentos. Otro punto a destacar es que no se ha evaluado el rendimiento de los modelos en biopsias de tejido prostático de diversas fuentes y por tanto no se ha comprobado qué tan fiables son ante datos externos. Es posible que este estudio esté ciertamente sesgado por la elección de los conjuntos OOD. Pese a que el conjunto COVID sea del ámbito médico, existen otras pruebas de imagen en medicina que comparten más características con las biopsias de tejido prostático.

Por último, cabe destacar las posibles **implicaciones clínicas** de los resultados de este estudio. Las métricas tanto en DeiT como en ResNet-50 son excelentes en cuanto a la clasificación de ejemplos (96.70 % y 96.59 % en *accuracy* resp.) y por tanto es interesante el despliegue de estos modelos en el ámbito profesional. No obstante, se ha comentado previamente, es posible que los datos con los que trabajan los modelos sean diferentes a los empleados en otros centros. Por tanto, en caso de utilización de alguno de estos modelos, sigue siendo necesario la supervisión de un especialista, ya que no aportan un rendimiento perfecto y el caso médico requiere de una precisión tan alta como sea posible. En cuanto a la detección OOD, el uso de **BestXAIPostProcessor con el 100 % de *features*** en DeiT muestra una separación perfecta (tal y como muestran la tabla 4 y la figura 27). De cara a un uso profesional, habría que realizar estudios sobre conjuntos más cercanos al dataset que el conjunto de COVID para determinar la capacidad de separación del modelo. Los resultados para el conjunto de COVID son muy esperanzadores de cara a esta tarea.

5.2. Conclusiones

El desarrollo de este trabajo ha traído consigo numerosos aspectos a destacar:

- En la introducción se describe como objetivo responder a la pregunta de si el uso de algoritmos de XAI mejora la detección OOD. La tabla 2, aporta respuestas para esta pregunta. En general se observó que el uso de vectores de XAI no mejora respecto al uso exclusivamente de *features* pero sí que mejora o iguala el rendimiento de MSP o ViM. Por tanto, no es el mejor método evaluado pero sí que el uso de vectores de XAI implica una discriminación válida.
- Cabe destacar la utilidad de la integración de la IA en casi cualquier ámbito, como en este caso que es el de la medicina, ha sido posible construir modelos que clasifiquen las biopsias prostáticas con una precisión cercana al 97 %. Por tanto, se cumple el objetivo de aportar una propuesta que tiene un buen rendimiento en el problema de clasificación descrito en 3.2.
- Mediante el uso de métodos basados en la distancia de Mahalanobis con respecto a una distribución de *features* (p.ej. XAIPostProcessor y MahaPostProcessor), se han obtenido resultados que mejoran el rendimiento de métodos del estado del arte como MSP o ViM [37]. De acuerdo a la tabla 2 se obtienen resultados de AUROC Y FPR95 de 53.31 % y 91.83 % en MSP y de 99.39 % y 2.32 % en el caso de ViM. Para XAIPostProcessor y MahaPostprocessor, estos resultados son de 99.78 % y 0.59 %.
- La versión normalizada de XAIPostProcessor, aporta resultados cercanos a los de los posprocesadores comentados en el punto anterior (tabla 2), por lo que aunque el vector de XAI no suponga una mejora, sigue siendo un discriminador razonable y por tanto no es descartable su uso en futuros experimentos.
- El ahorro de *features* (BestXAIPostProcessor) no aporta ventajas para el modelo **ResNet-50**, donde el único valor de porcentaje de *features* que supone un discriminador razonable es 90 %, lo que no supone un gran ahorro (tabla 3 y figura 25).
- En cambio, en el caso **DeiT** sí que obtenemos un método relativamente bueno con un ahorro del 75 % (tabla 4 y figura 26).
- Haciendo uso del total de *features*, en **DeiT** hemos conseguido una separabilidad perfecta tanto en Near-OOD como en Far-OOD, como confirma la figura 27.

5.3. Trabajos futuros

Este trabajo no concluye ni por asomo las posibles vías de investigación de modelos de Deep Learning, ni de detección OOD y tampoco de nuestro dataset en particular. Por tanto, de cara a profundizar en los aspectos de este trabajo se propone lo siguiente:

- **Uso de nuevos modelos vanguardistas** tanto de CNNs como de *Transformers* como *SwinTransformer* o la versión con token de destilación de DeiT.
- **Emplear distintos datasets para la detección OOD.** En particular, parece interesante escoger un dataset todavía más cercano que el dataset de COVID a nuestro dataset ID de biopsias de tejido prostático, ya que DeiT aportó una separación perfecta y sería interesante ver en qué distribución comenzaría a detectar erróneamente ejemplos OOD.

- **Extender el problema de clasificación a uno de segmentación.** De este modo, podríamos no detectar únicamente qué tipo de cáncer es sino que también podríamos delimitar el área del mismo y aportar mayor grado de utilidad.
- **Uso de nuevos métodos de IA explicable** como pueden ser gradCAM u otros.
- **Técnicas para el cálculo de scores no basadas únicamente en la distancia de Mahalanobis.**
- **Combinar de un nuevo modo el vector de explicaciones con el de *features*.** Por ejemplo, podríamos calcular una puntuación con respecto a la distribución de *features* y otra respecto a la distribución de explicaciones y tomar la predicción de la puntuación que más confianza genere.

6. Glosario de términos

IA: Inteligencia Artificial.

OOD: Out of Distribution.

ID: In Distribution.

XAI: eXplainable AI, IA explicable.

PSA: Prostate-Specific Antigen.

iid: Independientes e idénticamente distribuidas.

FD: Función de Densidad.

FMP: Función Masa de Probabilidad.

MLE: Maximum Likelihood Estimator.

MSE: Error Cuadrático Medio.

SGD: Stochastic Gradient Descent.

MLP: Multi-Layer Perceptron.

ReLU: Rectified Linear Unit.

PReLU: Parametric ReLU.

CNN: Convolutional Neural Network.

LRP: Layer-wise Relevance Propagation.

CRP: Concept Relevance Propagation.

RGB: Escala Red, Green, Blue.

ViT: Vision Transformer.

DeiT: Data-efficient image Transformer.

MSA: Multi-Head Self Attention.

TPR: True Positive Rate.

FPR: False Positive Rate.

ROC: Receiver Operating Characteristic.

AUROC: Area Under the Receiver Operating Characteristic curve.

FPR@95: False Positive Rate at 95 % True Positive Rate.

AUPR: Area Under the Precision Recall curve.

MSP: Maximum Softmax Probability.

ViM: Virtual-logit Matching.

KDE: Kernel Density Estimation.

7. Referencias

Referencias

- [1] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. <http://www.deeplearningbook.org>.
- [2] Yaser S. Abu-Mostafa. Learning from data. lecture 9: The linear model ii, 2012. <https://work.caltech.edu/slides/slides09.pdf>.
- [3] Sebastian Ruder. An overview of gradient descent optimization algorithms, 2016. <https://www.ruder.io/optimizing-gradient-descent/>.
- [4] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition, 2015.
- [5] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly,

- Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2021.
- [6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.
 - [7] Nestor Maslej, Loredana Fattorini, Raymond Perrault, Yolanda Gil, Vanessa Parli, Njenga Kariuki, Emily Capstick, Anka Reuel, Erik Brynjolfsson, John Etchemendy, Katrina Ligett, Terah Lyons, James Manyika, Juan Carlos Niebles, Yoav Shoham, Russell Wald, Toby Walsh, Armin Hamrah, Lapo Santarasci, Julia Betts Lotufo, Alexandra Rome, Andrew Shi, and Sukrut Oak. Artificial intelligence index report 2025, 2025.
 - [8] OpenAI. How people are using chatgpt, 2025. Consultado el 8 de julio de 2026.
 - [9] Aránzazu González del Alba Baamonde and Ramón Aguado Noya. Cáncer de próstata. <https://www.seom.org/info-sobre-el-cancer/prostata>, 2025. Sociedad Española de Oncología Médica (SEOM). Accedido el 30 de junio de 2026.
 - [10] Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing*. Pearson, 4 edition, 2018.
 - [11] José Miguel Angulo Ibáñez. Multivariate statistics. unit1. multivariate normal distribution, 2023. Material docente de la asignatura Estadística Multivariante, Universidad de Granada. Acceso restringido a estudiantes.
 - [12] Vijay K. Rohatgi and A. K. Md. Ehsanes Saleh. *An Introduction to Probability and Statistics*. Wiley, 3 edition, 2015.
 - [13] Larry Wasserman. *All of Statistics: A Concise Course in Statistical Inference*. Springer, New York, 2004.
 - [14] Tom M. Mitchell. *Machine Learning*. McGraw-Hill, New York, 1997.
 - [15] Kevin P. Murphy. *Probabilistic Machine Learning: An introduction*. MIT Press, 2022.
 - [16] Marc Peter Deisenroth, A. Aldo Faisal, and Cheng Soon Ong. *Mathematics for Machine Learning*. Cambridge University Press, 2020. <https://www.cambridge.org/es/universitypress/subjects/computer-science/pattern-recognition-and-machine-learning/mathematics-machine-learning?format=PB>.
 - [17] Shai Shalev-Shwartz and Shai Ben-David. *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press, 2014.
 - [18] Zhilu Zhang and Mert Sabuncu. Generalized cross entropy loss for training deep neural networks with noisy labels. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018.
 - [19] Ansh Mittal. Optimizers in machine learning and ai: A comprehensive overview, 2025. <https://medium.com/@anshm18111996/comprehensive-overview-optimizers-in-machine-learning-and-ai-57a2b0fbcc79>.
 - [20] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

- [21] A. Roca Martínez, D. Jornet, and V. Montesinos Santalucía. *Análisis matemático*. Editorial de la Universidad Politécnica de Valencia, Valencia, Spain, 2003.
- [22] Iván Sevillano-García, Julián Luengo, and Francisco Herrera. Stood-x methodology: using statistical nonparametric test for ood detection large-scale datasets enhanced with explainability, 2025.
- [23] Alejandro Barredo Arrieta, Natalia Díaz-Rodríguez, Javier Del Ser, Adrien Bennetot, Siham Tabik, Alberto Barbado, Salvador García, Sergio Gil-López, Daniel Molina, Richard Benjamins, Raja Chatila, and Francisco Herrera. Explainable artificial intelligence (xai): Concepts, taxonomies, opportunities and challenges toward responsible ai, 2019.
- [24] Reduan Achtibat, Maximilian Dreyer, Ilona Eisenbraun, Sebastian Bosse, Thomas Wiegand, Wojciech Samek, and Sebastian Lapuschkin. From attribution maps to human-understandable explanations through concept relevance propagation. *Nature Machine Intelligence*, 5(9):1006–1019, September 2023.
- [25] Jacobo Ayensa-Jiménez, Denis Navarro, Iván Sevillano-García, Andrés Mena, Isabel Marquina, Sofía Hakim, Jorge Alfaro, Francisco Herrera, Manuel Doblaré, and Ángel Borque-Fernando. Fine-grained prostate cancer tissue classification through expert-annotated dataset and clinically interpretable deep learning. In *2026 IEEE Conference on Artificial Intelligence (CAI)*, pages 1264–1269, 2026.
- [26] Jingkang Yang, Pengyun Wang, Dejian Zou, Zitang Zhou, Kunyuan Ding, Wenxuan Peng, Haoqi Wang, Guangyao Chen, Bo Li, Yiyu Sun, Xuefeng Du, Kaiyang Zhou, Wayne Zhang, Dan Hendrycks, Yixuan Li, and Ziwei Liu. Openood: Benchmarking generalized out-of-distribution detection. *arXiv preprint arXiv:2210.07242*, 2022.
- [27] Jingyang Zhang, Jingkang Yang, Pengyun Wang, Haoqi Wang, Yueqian Lin, Haoran Zhang, Yiyu Sun, Xuefeng Du, Yixuan Li, Ziwei Liu, Yiran Chen, and Hai Li. Openood v1.5: Enhanced benchmark for out-of-distribution detection. *arXiv preprint arXiv:2306.09301*, 2023.
- [28] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention, 2021.
- [29] Ross Wightman. Pytorch image models. <https://github.com/rwightman/pytorch-image-models>, 2019.
- [30] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- [31] Maria de la Iglesia Vayá, Jose Manuel Saborit, Joaquim Angel Montell, Antonio Pertusa, Aurelia Bustos, Miguel Cazorla, Joaquín Galant, Xavier Barber, Domingo Orozco-Beltrán, Francisco García-García, Marisa Caparrós, Germán González, and Jose María Salinas. Bimcv covid-19+: a large annotated dataset of rx and ct images from covid-19 patients, 2020.
- [32] MedCalc Software Ltd. Roc curve analysis, 2025.
- [33] Wikipedia contributors. Curva roc, 2026.
- [34] TorchUncertainty Contributors. Fpr95 metric, 2024.

- [35] Galadrielle Humblot-Renaux, Sergio Escalera, and Thomas B. Moeslund. Beyond auroc & co. for evaluating out-of-distribution detection performance. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 3881–3890, 2023.
- [36] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, et al. Scipy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, 17(3):261–272, 2020.
- [37] Haoqi Wang, Zhizhong Li, Litong Feng, and Wayne Zhang. Vim: Out-of-distribution with virtual-logit matching, 2022.